作品编号：CACR2026EA16XZ




作品类别：□ 软件设计　□ 硬件制作　■ 工程实践

□ 密码应用技术　□ 其它

# 2026 年第十一届全国密码技术竞赛作品设计报告

---

**作品题目：ML-KEM 解封装实现的时间侧信道检测与常数时间防护**

2026 年 10 月 21 日

中国密码学会

# 基本信息表

编号：CACR2026EA16XZ

作品题目：ML-KEM 解封装实现的时间侧信道检测与常数时间防护

作品类别：□ 软件设计　□ 硬件制作　■ 工程实践　□ 密码应用技术　□ 其它

## 作品内容摘要：

**ML-KEM-768**（FIPS 203）的解封装在 Fujisaki-Okamoto（FO）变换下依次执行解密、重加密、密文比较与隐式拒绝，四步均接触私钥或解密消息。一次完整解封装耗时在 **50 $\mu$s** 量级，而内部消息位引起的泄漏时延仅数十纳秒，单次测量完全淹没在系统噪声中，须靠**每类 10,000 次调用、三次独立运行**及采样前预先冻结的多重检验规则才能可靠分离。同时，C 源码无分支不等于编译后的机器码无分支，单一检测工具无法互相替代，“未检出泄漏”也必须附带当前测量噪声下的最小分辨量程——**最小可检测效应（MDE）**。为此，本作品面向 ML-KEM-768 便携 C 实现构建**“检测—定位—防护—复测”**一体化平台，覆盖五类代表性被测对象：liboqs 0.10.0 消息编码分支（M1）与 pq-crystals Kyber768 秘密除法（M2）两个真实历史案例；秘密下标查表（M3）与 32 B 提前返回比较（M4）两个人工校准夹具；以及设计为常数时间的 mlkem-native 阴性对照（M5）。平台通过 CPU 锁频固定测量工作点，采样前冻结全 0 与全 1 消息密文对比组（主类对）、样本量与 Holm 检验家族；统计层采用 Welch 检验、Holm 校正、多轮逆方差合并与 MDE 预算量化每项结论；定位层联合反汇编、Memcheck、varlat、Microwalk 与 Binsec 多工具交叉验证，并按“3 实现 × 5 步骤 × 6 类来源”完成 **90 格**全流程排查，所有判定均严格绑定**同一份编译产物**。

实验表明：M1 未修复版主类对均值差为 **+32.3 ns**（仅占 57 $\mu$s 总耗时的 **0.06%**），三轮独立运行方向一致，引入官方补丁后置信区间跨 0（未检出），解封装开销仅增加 **1.51%**；M2 三处秘密除法在定位层稳定命中、修复后归零，而端到端与热点微基准均未检出，端到端 MDE 为 **24.7–67.1 ns**，实证了**定位阳性不等于计时阳性**；M4 脆弱实现显著可区分，改造为常数时间实现后未检出；M5 在本机预算下未检出，**7 种工具误报为 0**。此外，90 格全流程排查发现了 KyberSlash 官方除法补丁未消除的同源消息编码分支；同一份 M1 源码在 aarch64 架构上编译为无分支条件选择指令；同机网络各档在当前样本预算下均未检出。本作品通过“真实历史案例的官方补丁与机器码核验、人工夹具的自主常数时间改造、平台 **1,048 个调用点**审计与退出码修复”形成**三层防护闭环**，赛题 **12 项**技术要求全部满足（见附录 A）。

关键词（五个）：**ML-KEM；时间侧信道；常数时间实现；泄漏检测与定位；预注册统计**

# 目录

**第一章 作品功能与性能说明** **4**

1.1 问题背景与赛题定位 . . . . . 4

1.2 威胁模型与攻击面 . . . . . 5

1.3 作品功能与系统组成 . . . . . 6

1.4 主要工作与创新点 . . . . . 7

1.5 关键结果与性能概览 . . . . . 8

**第二章 基础知识** **11**

2.1 ML-KEM 与解封装流程 . . . . . 11

2.2 时间侧信道与常数时间 . . . . . 11

2.3 侧信道检测与定位工具 . . . . . 12

2.4 统计检验与检测预算 . . . . . 12

**第三章 设计与实现方案** **14**

3.1 解封装流程与泄漏面 . . . . . 14

3.2 泄漏机理与攻击可行性 . . . . . 17

3.3 总体架构与证据分层 . . . . . 20

3.4 被测对象与模块设计 . . . . . 22

3.5 测量层设计 . . . . . 23

3.6 统计层设计 . . . . . 26

3.7 定位层设计 . . . . . 28

3.8 防护设计与泄露渠道审计 . . . . . 30

3.9 运行结果 . . . . . 32

3.10 技术指标 . . . . . 33

**第四章 系统测试与结果** **35**

4.1 测试方案与环境 . . . . . 35

4.2 功能与语义等价测试 . . . . . 36

4.3 端到端可区分性 . . . . . 38

4.4 函数级泄漏定位 . . . . . 40

4.5 泄漏源全流程排查 . . . . . 43

4.6 防护前后复测 . . . . . 45

4.7 检测预算与稳健性 . . . . . 47

4.8 异常输入与接口边界 . . . . . 49

4.9 性能开销 . . . . . 49

4.10 编译选项与测量条件 . . . . . 52
4.11 架构与网络边界 . . . . . 55
4.12 多工具交叉验证 . . . . . 56
4.13 可复现性 . . . . . 58

**第五章 与现有工作对比及应用前景** **60**
5.1 泄漏检测与评估方法对比 . . . . . 60
5.2 应用场景与推广边界 . . . . . 62

**第六章 结论** **64**
6.1 主要结论 . . . . . 64
6.2 局限与未验证项 . . . . . 65
6.3 后续工作 . . . . . 65

**附录 A 题面要求对照** **67**

**附录 B 实验与运行总表** **69**

**附录 C 统计公式与复算说明** **73**

**附录 D 模块级完整结果** **75**

**附录 E 定位工具输出摘要** **77**

**附录 F 90 格排查全表** **78**

**附录 G 复现入口与命令** **82**

**附录 H 术语与缩略语** **83**

**参考文献** **84**

# 第一章　作品功能与性能说明

## 1.1 问题背景与赛题定位

ML-KEM（模格密钥封装机制，Module-Lattice Key Encapsulation Mechanism）解封装的难点不在算法本身，而在源码级常数时间与产物级常数时间之间那条容易被忽略的错位。ML-KEM 由 CRYSTALS-Kyber 标准化而来，最终标准见 FIPS 203[1, 2]。解封装在 Fujisaki–Okamoto 变换下依次执行解密、重加密、比较与隐式拒绝，四步都接触秘密。任何一步出现依赖秘密的分支、访存或可变时延指令，处理耗时就会随秘密值变化，秘密信息便进入攻击者可以观测的时间通道。

这些问题有真实的案例。Guo 等对采用 FO 变换的密钥封装机制（KEM）给出密钥恢复的计时攻击，说明比较与拒绝这一步的时延可以携带信息 [3]。KyberSlash 把秘密除法做成可复现的计时差异 [4]。clangover 表明编译器可以把源码里的无分支掩码还原成分支 [5]。

本作品围绕五类对象展开：liboqs 0.10.0 消息编码分支案例（M1）与 pq-crystals Kyber768 秘密除法案例（M2）对应真实历史实现；秘密下标查表人工校准夹具（M3）与提前返回比较人工校准夹具（M4）用于检验已知机制；mlkem-native 阴性对照（M5）用于观察误报。这五个对象承担三种互补角色：真实历史案例检验平台能否发现已有缺陷并核验官方补丁，人工夹具检验工具能否识别预先埋入的机制，阴性对照检查平台是否误报。

FO 变换把解密得到的消息重新加密，再与输入密文比较，比较结果决定输出真密钥还是假密钥。这条选择必须由常数时间比较与掩码完成，不能出现依赖秘密的分支。隐式拒绝的设计目的，是让失败路径与成功路径在外部不可区分。一旦比较或拒绝的路径依赖秘密，这个目的就落空：攻击者不需要私钥，只凭每次解封装的耗时，就能判断解封装走到了哪一步。

以 liboqs 0.10.0 的消息编码分支案例（M1）为例：未修复版在 57 μs 的完整解封装均值上多出 32.311 ns，95% 置信区间 [20.282, 44.339]，只占均值的 0.06%。这里比较的是全 0 消息与全 1 消息生成的两类合法等长密文，后文简称“主类对”。单次调用看不出这个差别——本机计时最小步进为 10 ns，32 ns 只相当于 3 个步进，一次调用的随机起伏就把它盖住了。把每类 10,000 次调用、三次独立运行的样本合起来，差异才从噪声里露出来。因此，报告“未检出泄漏”时，还需要交代样本量和噪声水平，并给出最小可检测效应（MDE），即在指定检验水平和检出概率下，这套测量方案预计能分辨的最小时间差。它类似计时天平的分辨能力，不能被当作真实泄漏的上限。

C 语言源码没有写条件分支，并不代表编译后的机器码也没有分支。以 M1 为例，liboqs 0.10.0 的开发者用算术掩码代替条件分支，避免执行路径随秘密消息位变化。然而，在 x86_64 上使用 clang 18 的 `-Os`（以缩小代码体积为目标的优化）编译时，这段掩码代码又被改写成了依赖消息位的条件跳转。该问题已登记为 CVE-2024-36405[6]。相同源码在 ARM 64 位架构（aarch64）上生成的是位域提取和条件选择指令，没有出现这一秘密分支。因此，常数时间评估必须检查最终生成的机器码，并针对该编译产物进行测量。

换编译器、优化级别或目标架构都会改变产物，因此每次判定都绑定实际二进制与编译条件。

单工具下结论也不可靠。dudect 与 RTLF 只回答两类耗时是否可区分 [7, 8]，Memcheck 与 Microwalk 只回答哪里依赖秘密 [9, 10]，Binsec 只回答给定边界内是否常数时间 [11]，三类结论回答的问题不同，不能互相替代。第二个被测对象 M2（pq-crystals Kyber768 历史快照里的秘密除法）是最直接的例子：定位工具找到了三处操作数含秘密的整数除法，但完整解封装的合并耗时差为 $+4.58$ ns，95% 置信区间为 $[-3.08, 12.25]$ ns，跨过 0，表示数据尚不能将均值差与零区分。单独测量除法所在函数，合并差为 $+0.044$ ns，区间 $[-0.430, 0.517]$ ns 也跨过 0。也就是说，代码里存在秘密相关指令，在当前机器和样本量下却未必能测出相应时间差。只用一个工具，M2 可能被读成存在泄漏，也可能被读成没有问题。同一份 M2，RTLF 在未修复版和已修复版均报告两类分布差异；dudect 的最大 $t$ 值落在已修复版，而采集前规定的均值检验在两个版本上均未检出显著差异。不同工具关注均值、分布形态或内部秘密依赖，结论需要结合各自检查的问题来解释。

还有一重困难在读者一侧。未检出常被当成常数时间，但阴性结论只在给定样本量、方差与检验水平下成立。不写检测预算，读者就不知道这个阴性排除了多大的效应。RTLF 在 42 个数据集上给出 30 个未检出、12 个检出 [8]，同一批数据换一种统计口径就可能换一种读法，光看“检出”还是“未检出”并不够。

M2 的检测预算为 24.7–67.1 ns，大于除法可能产生的效应。在这个预算下，未检出表示现有样本未提供足够的差异证据，不能说明除法本身没有时延。

这四类困难彼此牵连：效应相对总耗时越小，测量就越依赖严格控制与重复采样；源码层的写法只有经过实际编译产物核验，才能支撑产物层判断；不同工具给出的定位、统计与有界证明各自回答不同问题；检测预算若不公开，未检出结果便无法说明排除了多大的差异。因此，本作品构建检测、定位、防护与复测相衔接的工程流程，并将每项结论绑定到具体产物和检测预算。

题面 12 条把这条缝拆成可检查的动作：接入标准实现、公开向量验证、可重复计时、两类输入、统计检验、定位、防护、同环境复测、开销、环境实验、威胁模型与复现。任何一条只做一层，都会被后一条追问。本作品按检测、定位、防护、复测组织，让这些追问落在同一套证据上。

本作品以两类耗时是否可区分作为判定目标。每次判定绑定编译产物并按证据层登记，阴性结果同时报告该预算下的最小可检测效应。

## 1.2 威胁模型与攻击面

本作品评估的是本地白盒攻击者：有源码、能编译、能重复调用、能读完整解封装耗时，但看不到私钥、共享密钥与内部验证标志。表 1.1 把能力假设与评估边界分成六项：攻击者能观测什么，判定止于哪一步。

表 1.1　威胁模型与评估边界

| 项目 | 本作品的设定 |
| --- | --- |
| 评估者能力 | 持有源码与编译产物，可自选编译选项，可重复调用解封装，并读取每次调用的完整耗时 |
| 秘密对象 | 解封装私钥中参与解密的部分、解密得到的消息、共享密钥、比较结果与隐式拒绝标志 |
| 可观测量 | 每次完整解封装的耗时，两类合法等长密文的耗时分布 |
| 不观测的量 | 私钥内容、共享密钥、内部验证标志，不使用功耗、电磁、故障注入，不假设与被测进程共享缓存 |
| 攻击目标 | 判断两类密文的耗时分布是否可区分，并把差异归因到机制，不追求密钥恢复 |
| 范围之外 | 真实远程链路、其他架构的计时层、功耗与电磁与故障注入、密钥恢复 |

表 1.1 概括攻击者能力与作品边界。具体机制、所需观测和距密钥恢复的步骤见第 3.2 节的表 3.1。

本作品在单台 x86_64 裸金属、clang 18.1.3 上进行本地白盒评估，判断限定于指定产物与样本预算下的时间可区分性，不开展密钥恢复。M1 使用前期封存批次，环境与锁频规程相同，但不与 M2 至 M5 混合统计；aarch64 只用于机制层对照，不构成第二台测量机；网络实验是同机模拟，不代表真实远程。功耗、电磁、故障注入及跨环境合并均不在范围内。主类对、族外输入和攻击可行性分别见第 3.2 节与第 4 章。

## 1.3 作品功能与系统组成

平台围绕五个被测对象，打通“检测—定位—防护—复测”四个环节。统一计时、CPU 锁频、统计分析与定位结果登记四个公共模块，使每次运行都能追溯到具体对象、编译产物、测量环境和判定依据。

平台对外提供六项功能。首先是受控输入生成：按公开种子构造两类合法等长密文，使解密后的消息分别为全 0 和全 1，这两组主要比较对象称为“主类对”；篡改、格式破损和截断等扩展输入另行测试。其次是受控测量：固定 CPU 频率工作点，在预热后随机交错两类密文，逐次测量完整解封装并保存全部原始值。第三是统计检测：采集前写定输入类别、样本量和共同校正的检验集合（Holm 家族），再用 Welch 检验、Holm 误报校正和逆方差合并分析差异，同时报告 MDE。

另外三项是函数级定位、防护与审计、可复现。函数级定位用反汇编、秘密标记、可变时延检查、轨迹互信息与有界验证把阳性落到函数与指令。防护与审计覆盖七类手段，并在产物层核对目标机制是否真的消失。可复现把计划、产物摘要、原始表与重算脚本打包成一键入口。

五个被测对象分三类。M1 与 M2 是真实历史案例：前者是 liboqs 0.10.0 在 x86_64 上被编译器引入的消息编码分支，后者是 pq-crystals Kyber768 历史快照里的秘密除法。M3 与 M4

是人工校准夹具，分别埋入秘密下标查表与提前返回比较，用来检验检测器能否识别已知机制。M5 是 mlkem-native 阴性对照，用来检查工具会不会误报。M1 对应 FIPS 203 初始公开草案（ipd），M2 是历史 Kyber768，两者都不等同于最终标准。五类对象的结果不能互相替代。

五类对象覆盖三种角色，是为了让同一套规程同时面对真实缺陷、已知机制与阴性对照。真实案例回答能否修复，人工夹具回答检测器能否识别，阴性对照回答会不会误报。三类问题不能用一个对象回答，也不能用一类对象的结论替另一类作保。

四个共享层是计时入口、锁频控制器、统计与功效、定位工具与结果登记。计时入口统一读钟与窗口，锁频控制器统一工作点与恢复核对，统计层统一家族、合并与预算口径，定位层统一工具版本与结果字段。四段链是检测、定位、防护、复测：检测回答能否分开，定位回答差异来自哪，防护回答改什么，复测回答改动是否生效。同一份原始数据可以供多层引用，结论只在其所属层成立。总体架构见图 3.3。

平台把判定用的二进制与计时用的二进制绑定为同一个。每次运行记录源摘要、产物摘要、编译器与选项，定位工具读的产物和计时窗口跑的产物是同一份。换编译选项等于换被测对象，必须重新登记，因为产物形态一变，基于旧产物的判定就不能直接沿用。这条约束把“源码改了”与“产物变了”分开，也是后面所有分层结论的前提。

工作量按对象、工具、排查、采集、样本、环境、测试、审计、统计与复现十个维度登记，汇总见表 1.2。该表便于核查各对象的运行次数、逐格排查证据，以及采集记录能否回溯到具体运行。

表 1.2　工作量总览

| 维度 | 数量 | 说明 |
|---|---|---|
| 被测对象 | 5 个 | 2 个真实历史案例、2 个人工校准、1 个阴性对照 |
| 定位工具 | 9 种 | 45 格矩阵中 31 格有真实运行产物 |
| 全流程排查 | 90 格 | 3 实现 × 5 步骤 × 6 类，未检查 0 格 |
| 正式采集 | 177 次控制器运行 | 54 次主实验 + 6 次 M2 热点微基准 + 117 次扩展对照 |
| 主类对样本 | 每类 10,000 × 3 次 | 换种子、换物理核，原始值全量落盘 |
| 优化级别 | 12 格 x86_64 + 12 格 aarch64 | 3 种实现 × 4 级，产物判定与计时分开 |
| 网络场景 | 5 档 × 3 种实现 × 2 次 | 本地直调、回环与低/中/高三档模拟 |
| 功能测试 | 263 项 | 88 + 57 + 57 + 61 四项，只验证软件流程 |
| 泄露渠道审计 | 128 文件 / 1,048 调用点 | 122 个接触秘密，0 条秘密泄露，1 条低危已修 |
| 统计复核 | 4 个 Holm 家族 / 21 个成员 | 42 个 RTLF 数据集，每个阴性附 MDE |
| 复现材料 | 精简与完整两档 | 匿名包 5/5，42 份原始长表，逐次运行台账 |

## 1.4 主要工作与创新点

本作品把已有工具组织成以产物为中心、可审计、可分层的证据流程，不提出新的检测原语，也不把任何观察主张为新的缺陷。四项工作分别交代比较基线、具体改动、设计依据与实施代价，并共同约束结论的适用范围。

围绕上述四大挑战，本作品在证据建模、受控测量、多工具交叉定位与跨环境边界评估

四个维度开展了系统性工作：

第一条是分层证据模型：把机制存在、热点可测、端到端可测与防护有效分为四层登记，并用最小可检测效应量化未检出。相对单工具、单层结论的评估方式，它要求同一个被计时的产物同时登记定位层、热点层、端到端层与复测层。每个阴性附预算，每个未执行单列。直觉容易把定位阳性当成计时阳性，M2 正好相反：三处除法定位阳性，端到端与热点微基准都未检出。四种状态是检出、未检出、未执行、不适用。这四种状态不是标签，而是决定一条结论能不能被引用的依据。代价是采集与工具运行增加，报告必须保留四种状态。证据见表 1.3、表 4.5 与表 4.6。

第二条是预注册与可审计的锁频协议：采集前冻结检验计划，控制器校验计划摘要并核对频率恢复，使每次判定可追溯、可复算。相对于事后选检验、只看 p 值的计时实验，主类对、样本预算、停止条件与 Holm 家族在采集前一次冻结。放行前做 20 次频率探针，退出核对恢复。锁频提供可核对、可恢复的频率工作点，减少动态频率变化对实验可比性的影响。三个对象的不锁频/锁频变异系数比分别为 0.964、1.006、1.089，说明本机的相对噪声水平接近，锁频的作用不能只用标准差是否下降来评价。代价是机时与登记成本，见 3.5 节与表 5.1。

第三条是多工具交叉与 90 格排查：用 9 工具 × 5 模块矩阵和 90 格排查，把定位阳性与计时阳性分开，并识别官方修复后残留的同源分支。相对单工具评估，它要求每个存在给出函数、指令与触发输入，每个不存在写明检查手段与覆盖输入。90 格中 5 格存在、75 格不存在、10 格不适用、0 格未检查。90 格排查还发现，KyberSlash 官方修复提交后的产物仍保留与 clangover 同源的消息编码分支，说明只消除除法并不等于整个产物中已无秘密相关机制。该项是全流程排查方法的超额发现，尚未作逐指令时序归因，也不构成新的漏洞主张。代价是工具安装、版本固定与人工核对，见第 4.5、4.12 节。

第四条是产物与环境的依赖对照：把同一源码在优化级别、架构、虚拟化与网络条件下的产物形态与可检出性并列，界定结论的适用边界。相对单环境、单优化级别的评估，它把 12 格优化级别、12 格 aarch64、三台 WSL2（Windows 子系统 for Linux 2）与 5 档同机网络分开登记。时间证据不跨环境合并。直觉是分支越多越容易测到，实测 -O2 与 -O3 的命中位置更多，计时却未检出。M1 的机制在 aarch64 上不复现。代价是跨环境实验不能互相替代，见 4.10 节与 4.11 节。

四条工作合起来，回答的是同一个问题：同一份源码在不同产物上可以有不同结论，证据必须按产物、按层、按预算登记。每条结论都能回到一个具体的产物、一个明确的证据层和一组可复算的数字。这些结论不覆盖所有实现或所有环境。

## 1.5 关键结果与性能概览

五个模块在同一套测量与统计规程下形成了不同层次的结论：M1 的分支差异在修复后未检出，M2 虽在定位层命中但计时层未检出，M4 提供脆弱实现与常数时间实现的前后对照，M5 在既定预算内未误报。表 1.3 展示五个模块在定位层、端到端计时层与防护复测层的横向

结果。

五行共享同一计时入口、锁频工作点、预注册家族与功能门，因此测量规程可比；对象角色则各不相同，M1 与 M2 是历史缺陷，M3 与 M4 是人工夹具，M5 是对照。解读时应先比较各证据层，再结合对象角色理解每行结果，避免将规程上的可比性误作对象性质相同。

表 1.3　五模块分层结论

| 模块 | 对象与角色 | 定位层 | 完整解封装计时层 | 防护后 |
|---|---|---|---|---|
| M1 | liboqs 0.10.0 消息编码分支（FIPS 203 初始公开草案），真实历史案例 | 阳性：1 个秘密条件跳转 | 阳性：+32.311 ns [20.282, 44.339]，三次运行区间均不含 0 | 官方修复后分支消失，−8.150 ns [−21.916, 5.616]，未检出 |
| M2 | pq-crystals Kyber768 秘密除法（历史快照），真实历史案例 | 阳性：三处除法，1,280 次命中 / 3 处位置 | 未检出：+4.58 ns [−3.08, 12.25]，热点微基准 +0.044 ns [−0.430, 0.517] | 官方修复后除法计数归零，计时层前后均未检出 |
| M3 | 秘密下标查表，人工校准 | 阳性：1,280 次命中 / 2 处位置，互信息 1.922 bits | 未检出：−0.73 ns [−5.56, 4.10] | 消除查表版：0 报告，互信息 0.722 bits |
| M4 | 32 B 提前返回比较，人工校准 | 阳性：32 次命中 / 1 处位置，有界验证给出反例 | 阳性：脆弱实现 −4.089 ns [−4.215, −3.963] | 常数时间实现：+0.055 ns [−0.363, 0.473]，有界验证无秘密相关控制流 |
| M5 | mlkem-native，阴性对照 | 未命中：三种模式各 10 次，0 报告 | 未检出：−5.30 ns [−11.77, 1.16] | 无需修复，7 种工具误报 0 |

数字上的主线有四条。M1 未修复版合并差 $+32.311$ ns，区间 $[20.282, 44.339]$，三次运行区间均不含 0，官方修复后 $-8.150$ ns，区间 $[-21.916, 5.616]$ 含 0。M2 完整解封装 $+4.58$ ns，区间含 0，除法热点微基准 $+0.044$ ns，区间含 0，定位层三处除法在修复后计数归零。M4 脆弱实现 $-4.089$ ns，区间 $[-4.215, -3.963]$，常数时间实现 $+0.055$ ns，区间含 0。M5 合并差 $-5.30$ ns，区间 $[-11.77, 1.16]$ 含 0。

M2 按版本合并的均值差分别为 $+12.35$ ns 与 $-11.20$ ns，方向相反。衡量六次运行结果不一致程度的异质性指标 $I^2 = 58.3\%$，因此跨版本合并值只用于概括观测，正式判定仍采用采集前规定的逐次 Holm 检验，两版均未检出。M4 的脆弱实现区间不含 0，常数时间实现区间含 0，前后对照因此在同一张表上完成。每个数字背后是三次独立运行与每类 10,000 次调用。三次运行的区间列在附录 D，正文只给合并值与区间。

未检出的格子都附了预算：M2 的 MDE 为 24.7–67.1 ns，M3 为 17.7–33.7 ns，M5 为 13.5–28.3 ns。五个模块的结论按证据层分别解读：M1 与 M4 在定位层和计时层都发生变化，构成前后对照；M2 虽在定位层命中，计时层修复前后均未检出；M3 的开启查表版与消除查表版用于验证检测器能否识别已知机制；M5 是阴性对照。M4 是唯一在定位层与计时层均阳性、且常数时间实现均未检出的正对照，可据此检查检测链路对已知机制的响应。

M1 的三次运行区间均不含 0，Holm 家族中只有这一项被拒绝，这是本作品唯一在端到端层被判阳性的真实案例。

这些结论合起来说明一件事：修了与测不出是两件事。M1 的官方修复使分支消失，计时

层随之回到区间含 0。M2 的除法在定位层归零，计时层却从未给出可区分的基线。M5 没有任何阳性，也只能说明本预算下未检出。三句话分开，读者才能判断每一步证据的强度。

表 1.3 展示五个模块在定位层、端到端计时层与防护复测层的横向对比结果；各层分别记录机制证据、预注册统计判定和修复后变化。

性能开销按成对分块测。M1 修复使完整解封装 $+1.51\%$，M2 修复 $-2.92\%$：修复并不总意味着变慢，更快也不等于更安全。M3 的消除查表版相对开启查表版为 $+3.60\%$，区间 [0.36, 9.98] 较宽，按描述性结果报告。M4 是 32 B 比较调用，首字节与末字节的相对开销分别为 $+26.21\%$ 与 $+6.85\%$；M1、M2、M3 的开销来自完整解封装，M4 来自 32 B 比较调用，均不含键生成与封装。

本章介绍作品入口，方法见第 3 章，逐项结果见第 4 章。赛题 12 项要求均有对应实现和证据，并在附录 A 逐项登记为满足。

# 第二章　基础知识

后续章节会反复用到四组概念：解封装流程、时间侧信道、检测工具与统计检验。本章只讲这四组概念的约定与原理。

## 2.1 ML-KEM 与解封装流程

ML-KEM（模格密钥封装机制，Module-Lattice Key Encapsulation Mechanism）是 FIPS 203 标准化的格基密钥封装算法，由 CRYSTALS-Kyber 演化而来。密钥封装机制（KEM）解决的是两个通信方在不预先共享密钥的情况下协商出会话密钥的问题：一方用对方公钥把随机密钥封装进密文，另一方用私钥解封装得到同一个密钥。ML-KEM-768 是其中参数集为 768 的实例，FIPS 203 给出了它的密钥生成、封装与解封装三个算法。

本作品关心的是解封装。解封装在 Fujisaki-Okamoto（FO）变换下依次执行四步：用私钥解密得到消息，把消息重新加密，与收到的密文比较，根据比较结果返回真密钥或假密钥（隐式拒绝）。四步都接触私钥或解密消息，任何一步的实现若让耗时依赖这些秘密值，就打开了时间侧信道。隐式拒绝的设计目的，是让失败路径与成功路径在外部不可区分。比较与拒绝这一步一旦依赖秘密，攻击者不需要私钥，只凭每次解封装的耗时就能判断内部走到了哪一步。

解封装之所以是侧信道的重点，是因为它同时接触私钥、解密消息与比较结果，而且运行在每次密钥交换都会执行的热路径上。本作品的检测、定位、防护与复测都围绕这一步展开。整个密钥协商的结构见图 2.1：封装在发送方完成，解封装在接收方完成，信道上只传输密文。




图 2.1　KEM 密钥协商：封装与解封装

## 2.2 时间侧信道与常数时间

时间侧信道利用的是执行耗时对秘密值的依赖。程序的分支条件、访存地址、循环次数或可变时延指令（如除法）如果由秘密决定，处理不同秘密值的耗时就会不同，秘密信息就进入攻击者可以观测的时间通道。

常数时间实现的目标，是让执行路径与耗时独立于秘密：比较用掩码选择而不是分支，查

表用不依赖下标的访问方式，除法等可变时延指令避开秘密操作数。写代码时做到这些还不够。编译器优化可能改变产物形态，同一份源码在不同优化级别、不同架构上会得到不同的二进制，其中一些可能重新引入依赖秘密的分支。这就是源码级常数时间不等于产物级常数时间：判定必须对着具体的编译产物做，不能拿源码当承诺。

时间侧信道的观察需要把效应从噪声里分出来。本机计时的最小步进、调度、频率、缓存与后台进程都会叠加进单次耗时，几十纳秒的差异单次调用看不见，要靠重复采样与统计检验。锁频把工作点固定下来，减少噪声里可控制的一项，但压不住秘密相关差异本身。图 2.2 给出源码与产物分离的示意：源码写成了无分支，编译器可能在产物里重新引入依赖秘密的分支。




图 2.2　同一份源码经过编译器得到不同的产物形态

## 2.3 侧信道检测与定位工具

检测工具按回答的问题分三类。统计检测把程序当黑盒，比较不同输入类下的耗时分布：dudect 用 Welch 检验比较两类样本，R-Time-Leak-Finder（RTLF）用重采样把第一类错误控制在给定界内。这类工具回答两类样本能不能分开，不回答差异来自哪条指令。动态分析在程序内部找秘密依赖：Memcheck（Valgrind 的内存检查器）报告秘密进入分支条件或访存下标的时刻，Microwalk 用执行轨迹与互信息找与秘密关联最强的指令，Valgrind 的可变时延补丁 varlat 另行标记操作数含秘密的除法等指令。这类工具能给出函数级甚至指令级的落点，但只覆盖实际执行过的输入。形式化验证在给定模型内证明控制流与访存不依赖秘密：Binsec 在二进制层做有界检查，能给出反例路径，但通常不建模指令时延、缓存与流水线。

三类工具回答的问题不同，不能互相替代。定位阳性（工具在源码或产物里找到秘密依赖）与计时阳性（本机实测两类耗时可区分）是两件事，一个工具给出的单一读法不足以当判定，多个工具同向才有意义。三类工具的分工见图 2.3。

## 2.4 统计检验与检测预算

统计检验把重复采样的耗时变成可判定的结论。本作品的统计层用 Welch 检验比较两类耗时的均值，用 Holm 校正控制一组多重比较的总体误报率，用逆方差固定效应合并多次独立运行的区间，并以预注册在采集前冻结检验计划。异质性指数 $I^2$ 描述独立运行间结果的不

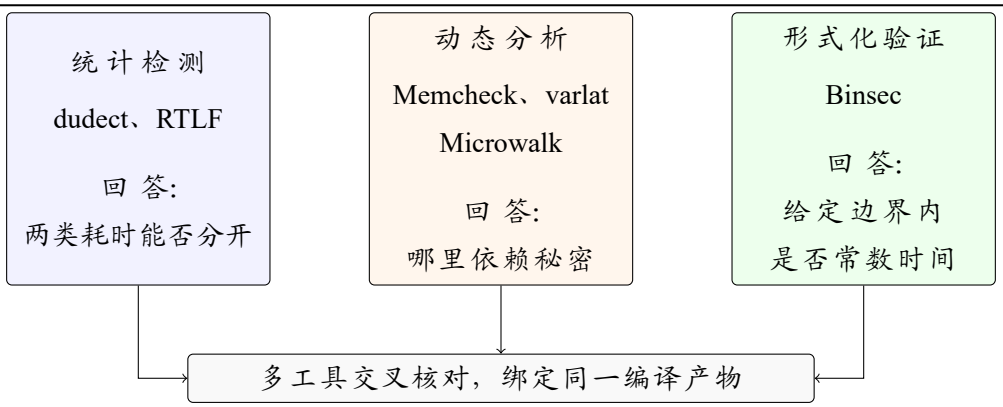



图 2.3　三类检测工具按回答的问题分工

一致程度；异质性较高时，合并值只作描述。

统计检验的结论有预算边界。最小可检测效应（MDE）可以类比为精密天平在当前环境噪声下的最小分辨量程：若当前分辨下限为 25 ns，而实际指令差异只有 5 ns，检出这种差异的概率可能较低，未检出不能说明效应为零。预注册与 Holm 校正像一把防作弊锁：采样前固定输入类别、样本量、停止条件与检验家族，既防止事后挑选数据，也控制多重检验偶然产生假阳性的风险。异质性指数 $I^2$ 则像多次称量间的环境基线漂移度；例如 M2 的 $I^2 = 58.3\%$，表示六次运行的估计结果一致性较弱，此时合并均值只作参考，判定严格依据单次预注册 Holm 结果。未检出必须附带 MDE 预算，才能交代当前“天平”的检出能力。图 2.4 画出统计检验面对的情形：两类耗时的分布部分重叠，检验要把均值差 $\delta$ 从噪声里分出来。

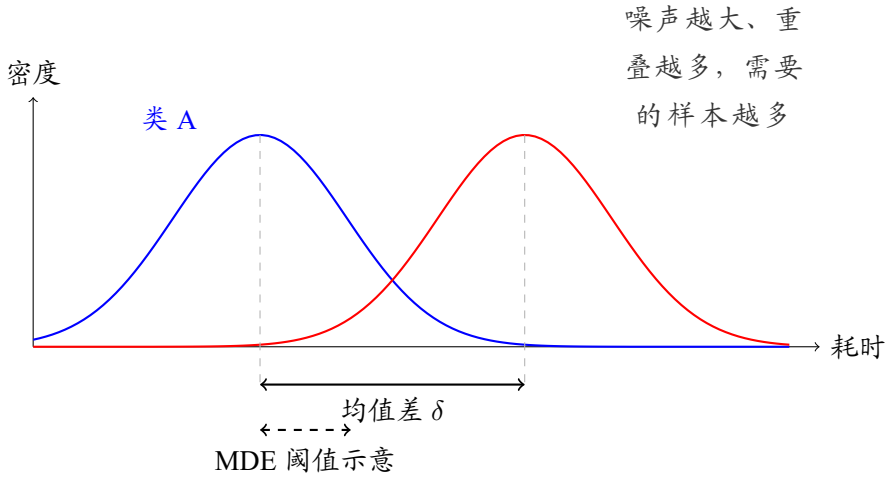



图 2.4　统计检验面对的两类耗时分布

本章的概念在后续各章会反复用到：第 3 章讲测量、判定、定位与防护的设计，第 4 章给系统测试与结果，第 5 章与现有工作对比，第 6 章收拢结论。

# 第三章　设计与实现方案

这一章讲检测平台怎么搭。起点不是工具，而是解封装流程本身：先弄清哪些步骤接触秘密，测量、判定、定位和防护才有依据。本作品要分辨的差异只有几十纳秒，单次测量看不出来，所以测量、统计和定位必须一起设计。下面从解封装流程与泄漏面讲起，按测量、判定、定位、防护的顺序说明每一步的做法与取舍。

## 3.1 解封装流程与泄漏面

ML-KEM 解封装在 Fujisaki–Okamoto 变换下完成解密、重加密、比较与隐式拒绝四步计算，每一步接触的秘密并不相同。解密接触私钥与解出的消息，重加密把消息重新送进编码、数论变换、压缩与序列化，比较同时接触真密钥与假密钥，隐式拒绝决定失败时输出什么。把这条链笼统称为“实现有侧信道”，后续的测量与定位就没有落点。泄漏面必须按步骤拆开，才可能落到函数与指令。[1]

**算法 1** ML-KEM 解封装与隐式拒绝

**输入：** 密文 $c$（1088 B），解封装密钥 $dk = (dk_{\mathrm{PKE}}, h, z)$

**输出：** 共享密钥 $K$（32 B）

1: $m' \leftarrow \text{K-PKE.Decrypt}(dk_{\mathrm{PKE}}, c)$ ▷ 解密，使用私钥

2: $(K', r') \leftarrow G(m' \,\|\, h)$ ▷ 派生候选密钥与重加密随机数

3: $c' \leftarrow \text{K-PKE.Encrypt}(dk_{\mathrm{PKE}}, m', r')$ ▷ 重加密

4: $\bar{K} \leftarrow J(z \,\|\, c)$ ▷ 隐式拒绝的假密钥

5: $K \leftarrow \text{Select}(c' = c,\ K',\ \bar{K})$ ▷ 常数时间比较与掩码选择

6: **return** $K$

算法 1 回答的是标准结构中哪些数据接触秘密，不表示任何具体实现编译后的样子。M1 的对象是 FIPS 203 初始公开草案（ipd），M2 是历史 Kyber768，二者都不等同于最终标准。标准伪代码也不记录实现选择: 消息编码可以逐系数写，也可以批量写。压缩可以原地进行，也可以先拷贝。实现选择决定一条分支落在哪个函数、哪条指令。后面排查表里的“步骤”因此只是第一层坐标，第二层坐标必须是函数与指令。

第 1 步解密的秘密是私钥与解出的消息。消息解码函数里的一次整数除法，就是 M2 三处秘密除法中的第一处。除法指令的时延随操作数变化时，私钥系数会进入耗时。[4] 第 3 步重加密把消息重新送进消息编码、数论变换、压缩与序列化: M1 的分支出现在消息编码，编译器把源码中的掩码选择还原成按位测试后条件跳转。M2 的另两处除法出现在多项式压缩与向量压缩。M3 的人工秘密下标也插在这条路径上。[5]

第 4 步比较对应 M4 的提前返回夹具。第 5 步的隐式拒绝必须由常数时间比较与掩码选择完成。只要实现让分支或提前返回依赖比较结果，成功与失败两条路径就可以从耗时上分

开，隐式拒绝的设计目的就落空。图 3.1 把这条对应关系画在同一张流程图上。M5 不挂在单独一步上，它作为整条链的阴性对照。图中“是/否”表示密钥的逻辑选择，实际实现同时计算拒绝密钥，再用掩码选择输出。

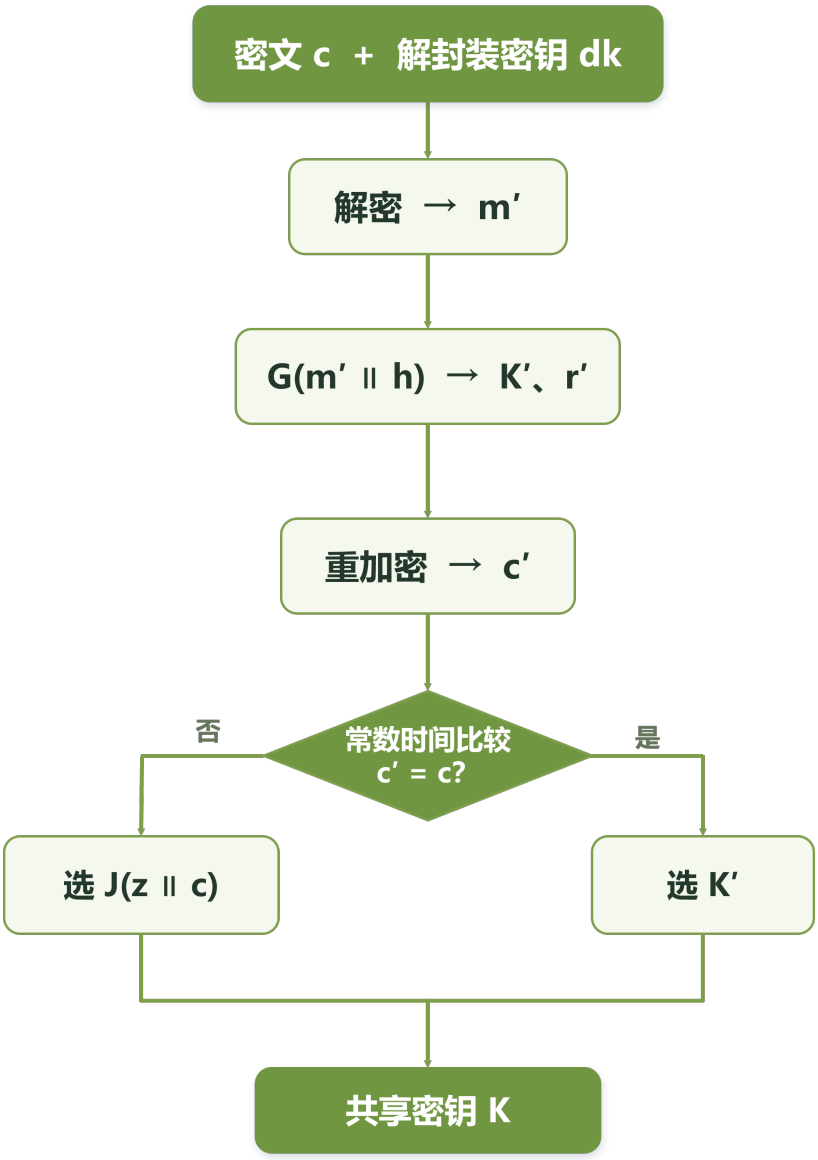



图 3.1　ML-KEM 解封装流程与常数时间选择

泄漏来源按题面第 5 条分为六类，每一类都在解封装链上有明确的作用点。

1. 敏感条件分支：消息位或比较结果决定执行路径，两条路径的指令数或访存不同。
2. 提前返回：比较在发现第一个不同字节时结束，函数返回时刻取决于失配位置。
3. 可变时间比较与可变时延指令：逐字节比较提前退出，或整数除法等指令的时延随操作数变化。
4. 秘密下标访存：查表下标含秘密位，不同下标落在不同缓存行。

5. 数据相关循环：循环次数或迭代上界随秘密变化。

6. 异常处理路径：成功与失败走不同的分支、日志与返回码。

六类来源的检查手段不同：前三类靠反汇编与秘密标记，第四类靠执行轨迹与秘密的互信息，第五类靠控制流核对，第六类靠泄露渠道审计。工具放在 3.7 节，这里只固定分类口径，避免同一现象被两个名字重复登记。

这六类不是互斥的标签，而是按“秘密以什么方式影响执行”划分的检查口径。同一条指令可能同时属于两类，例如逐字节比较既是可变时间比较，也可能带提前返回。排查时按“最早把秘密带入可观测差异的那一步”归类，避免同一现象在两格重复计数。归类规则写进排查表的使用说明，换人核对时按同一口径。题面列出的六类来源覆盖了从控制流、访存到错误处理的主要通道。功耗、电磁这类物理通道不在本作品范围内，因为平台只测时间。

把这六类来源分别放到三个实现、五个解封装步骤上，就得到 $3 \times 5 \times 6 = 90$ 格的全流程排查表。五个步骤是解密、哈希派生、重加密、密文比较与隐式拒绝。三个实现是 M1、M2 与 M5，人工夹具 M3、M4 不进入这张表。每一格只允许四种判定：存在、不存在、不适用、未检查。判定规则在排查前写定：判“存在”必须给出可指认的函数、指令或源码位置以及触发它的输入。判“不存在”必须写明用了哪种检查、覆盖了哪些输入。判“不适用”要求该机制在这一步骤上不可能出现，例如该步骤没有异常处理。判“未检查”单列，不得并入阴性。

逐格结论与证据编号见 4.5 节与附录 F，本节只定义方法与判定规则。

每一格的记录不只写判定，还要写证据。字段包括：实现与产物摘要、步骤、来源类别、判定、函数名或指令地址、触发输入、使用的检查手段、证据编号。判“存在”的格子必须有触发输入。没有触发输入的“存在”只能算“疑似”，在汇总时降级为“未检查”。判“不存在”的格子要写覆盖了哪些输入。只跑过一个输入的阴性不足以支撑“不存在”。这套字段让 90 格既是排查表，也是审计清单：换一个人按同样字段复核，结论应当一致。

五步的划分比算法 1 多出“哈希派生”一步。算法 1 把派生与重加密写在一起，排查时若合并，哈希路径上的问题会被记到重加密名下。拆开后，派生步骤只检查哈希调用本身，重加密步骤只检查编码、变换、压缩与序列化。代价是排查表多出一列，但定位结论可以直接落到“哪一步、哪个函数”。

如果不按步骤拆，M1 的消息编码分支与 M2 的压缩除法都会被归到“重加密有问题”这一个大类里。重加密路径覆盖编码、变换、压缩与序列化多段代码，定位只能停在这一层，修复也无法核对到底改了哪条指令。90 格把每一格绑定到“实现 × 步骤 × 来源”，任何一格判“存在”都附一个可复核的代码位置，修复后同一格可以被复查为“不存在”。修复后按同一张表复查，目标格从“存在”变为“不存在”，其余格保持不变，修复的副作用也能被看见。代价是排查前要为每个实现准备五步的证据口径，排查后要维护 90 条带证据编号的记录。这部分工作量换来的是定位结论可追溯。

## 3.2 泄漏机理与攻击可行性

输入类与重复调用遵循第 1.2 节的威胁模型：评估者选择密文类别与调用次数，观察完整解封装耗时。主检验比较两类合法等长密文的耗时分布，并报告样本量与检测预算。

一次解封装的耗时可以写成

$$T = T_0 + \delta(\text{secret}) + \text{noise},$$

其中 $T_0$ 是公开路径的平均耗时，$\delta$ 是秘密相关部分，noise 来自调度、频率、缓存与地址对齐。M1 未修版本在 57 µs 的均值上只多出 32.3 ns，$\delta/T_0$ 约为 0.06%。本地锁频压住的是噪声中的频率项，压不住 $\delta$ 本身。要在噪声里分出 $\delta$，只能靠大样本与预注册检验。每类 10,000 次、三次独立运行，是把可检出下界压到几十纳秒量级的直接原因（3.5、3.6 节）。图 3.2 把观测模型与泄漏方程画在一起：左侧是攻击者控制的输入，上方是固定密钥的解封装预言机，右侧保存耗时样本，下方给出统计判定与泄漏方程。

主类对在固定解封装密钥对 $(pk, dk)$ 下构造。由公开种子派生封装随机数，分别将内部 32 字节消息 $m'$ 固定为全 0（`0x00...00`）和全 1（`0xFF...FF`），再经合法封装流程生成长度均为 1088 字节的有效密文 $C_0$ 与 $C_1$。两类输入在解封装时均经过解密、重加密、密文比较和隐式拒绝选择代码；固定密钥、种子派生流程与长度一致，实验自变量是预设消息位，密文其他字段由标准算法随之派生。

**算法 2** 攻击者的计时观测与可区分性实验

**输入：** 解封装预言机 $\mathcal{O}$，两类合法等长密文 $C_0, C_1$，每类样本 $n$，显著性 $\alpha$

**输出：** 可区分 / 未检出（附效应量与区间）

**for** $b \in \{0, 1\}$ **do**
  **for** $i = 1$ **to** $n$ **do**
    $c \leftarrow \text{Sample}(C_b)$ ▷ 由公开种子派生，长度合法
    $t_{b,i} \leftarrow \text{Time}(\mathcal{O}(c))$ ▷ 只测一次完整解封装
  **end for**
**end for**
$\Delta \leftarrow \text{mean}(t_0) - \text{mean}(t_1)$
$CI \leftarrow \Delta$ 的 95% 置信区间
**if** $CI$ 不含 0 **then**
  **return** 可区分 $(\Delta, CI)$
**else**
  **return** 未检出 $(\Delta, CI, \text{MDE})$
**end if**

算法 2 回答的是给定观测模型与样本量时，两类密文的时间分布能否被分开。

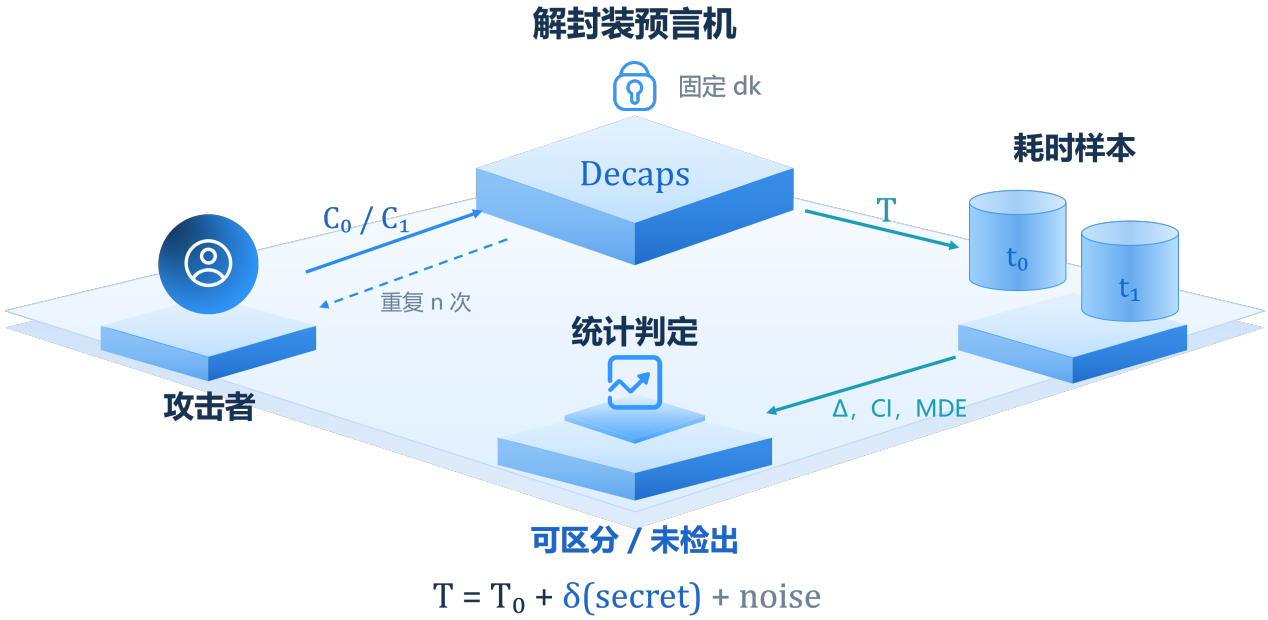



图 3.2　攻击者观测模型与泄漏方程

四类机理可以用最小形态写清楚。代码 1 到代码 4 只保留结构，不写完整实现，也不构成可直接复用的攻击代码。

代码 1　秘密相关分支的最小形态

```
/* 消息位决定走哪条路径；两条路径的指令数或访存不同 */
int16_t select_coeff(const uint8_t *msg, size_t i)
{
    if ((msg[i / 8] >> (i % 8)) & 1)
        return COEFF_ONE;
    return 0;
}
```

代码 1 对应 M1 的消息编码：源码本用掩码，编译器在特定选项下把它还原成按位测试后条件跳转。边界是：这段代码只说明机制，不代表上游源码的原样写法。分支是否出现取决于编译器与优化选项，必须看实际产物。

代码 2　秘密相关除法的最小形态

```
/* x 来自解密消息；整数除法的时延可能随操作数变化 */
int16_t compress_coeff(int16_t x)
{
    return (int16_t)((((int32_t)x << 4) + (Q / 2)) / Q);
}
```

代码 2 对应 M2 的三处除法，$Q$ 为公开模数。边界是：除法是否真的有可测时延取决于处理器与编译选项。本作品在特定产物上定位到三处整数除法，但端到端与热点微基准都未检出。

代码 3　秘密下标查表的最小形态

```
/* 下标来自消息位；不同下标落在不同缓存行 */
int16_t lookup_coeff(unsigned secret_index)
{
    static const int16_t table[16] = { /* 16 个公开常数 */ };
    return table[secret_index & 15];
}
```

代码 3 对应 M3 的人工校准夹具：查表插在常数时间基线的重加密路径上，开启查表版（on 版）与消除查表版（off 版）的输出逐字节相同，用于检验工具对秘密下标访存的识别能力。

代码 4　提前返回比较的最小形态

```
/* 发现第一个不同字节就返回；耗时取决于从哪里开始不同 */
int compare_bytes(const uint8_t *a, const uint8_t *b, size_t len)
{
    for (size_t i = 0; i < len; i++)
        if (a[i] != b[i])
            return 0;
    return 1;
}
```

代码 4 对应 M4 的比较夹具：脆弱实现（vulnerable 版）发现首个不同字节即返回，常数时间实现（ct 版）逐字节异或累积并在固定长度结束后统一判断。两版输出保持逐字节相同。

可区分性是侧信道分析链中的一环，机制是否存在、计时是否可区分、攻击是否可利用需要分别评估。M2 是典型例子：三处除法在定位层稳定命中，端到端完整解封装与除法热点微基准均未检出；同一份实现中，dudect 的最大 $t$ 还落在已修复版上。证据因此按机制、计时与可利用性分层登记。

主类对只让解密消息的位全 0 或全 1 不同，其他条件相同。这样两类的路径长度接近，差异只能来自消息位。随机密文、逐字节篡改、格式破损与截断属于族外对照：随机密文检验一般路径，篡改与破损检验隐式拒绝，截断检验公开长度检查。它们不进入主家族，因为输入路径长度或返回位置不同，混入会把路径差异误读为秘密泄漏。族外对照只作描述，不进入 Holm 家族，用于检查主类对之外的输入行为。

表 3.1　攻击可行性对照

| 机制 | 攻击者需要的观测 | 本作品做到哪 | 离密钥恢复还差什么 |
| --- | --- | --- | --- |
| 秘密分支（M1） | 同一密文类下，消息位不同导致耗时不同 | 本地锁频、完整解封装、三次运行可区分（+32.3 ns） | 选择密文构造、逐位消息恢复、纠错与多轮查询。本作品不做 |
| 秘密除法（M2） | 除法操作数不同导致耗时不同 | 定位层稳定命中三处除法。端到端与热点微基准未检出 | 更大样本量、针对除法的输入类、远程噪声下的可观测性。本作品不做 |
| 秘密下标（M3） | 不同下标落在不同缓存行导致耗时不同 | 人工夹具上动态检查与轨迹互信息命中。计时层未检出 | 共驻缓存攻击模型、缓存驱逐与观测。本作品不做 |
| 提前返回（M4） | 比较从第几个字节开始不同导致耗时不同 | 32 B 夹具上脆弱版可区分，常数时间版不可区分 | 完整 KEM 比较路径、远程噪声、逐字节恢复。本作品不做 |
| 阴性对照（M5） | 若存在机制，应能被工具或计时发现 | 三种动态模式 0 报告、有界验证无秘密相关控制流、计时未检出 | 不是“无泄漏”的证明。需要更大预算与其他工具 |

M2 的三处除法在定位层命中，端到端合并差 $+4.58$ ns、区间 $[-3.08, 12.25]$ 含 0，热点微基准合并差 $+0.044$ ns、区间含 0。表 3.1 将定位、计时与攻击所需条件并列，说明三类证据各自回答的问题。

## 3.3 总体架构与证据分层

平台的价值不在单个工具，而在把冻结计划、被测对象、测量、分析、登记串成一条可追溯的流水线。任何一个环节可以换工具，换不掉的是“结论必须能回到运行编号与证据编号”这条约束。

冻结计划层在采集前写定主类对、样本量、停止条件与 Holm 家族，并生成计划摘要。测量层每次运行校验摘要，防止采集开始后改规则。被测模块层固定五个对象，记录源摘要、产物摘要、编译器、选项与反汇编，判定与计时使用同一个二进制。测量层执行计时循环与锁频控制，每次调用逐次落盘，失败样本单列。分析层三路独立：统计判定回答“是否可区分”，定位工具回答“哪里依赖秘密”，族外检验回答“换个统计口径是否还成立”。结果登记层把每条结论写成四种状态之一，附运行编号与证据编号，未执行与不适用不并入阴性。图 3.3 给出五层的顺序与共用的规程。

五层的顺序不是排版顺序，而是防篡改顺序。计划在最前，后续每一层都只能按计划执行。被测模块在测量之前固定，避免测到一半换产物。分析在测量之后，三路互不覆盖。登记放在末端，任何一条结论都必须带运行编号与证据编号。探索性分析可以做，但不能进主家族，只能作为族外观察或后续工作登记。

登记层的每条记录都有唯一编号，编号由运行编号、证据编号与计划版本三段拼接。引用时只写编号，不再复述结论。这样第 4 章引用第 3 章的方法时，读者可以顺着编号回到原始记录，而不必在两章之间寻找同一句话。

计划摘要还有一个副作用：任何一次运行都能被独立复算。审计者拿到计划文件、运行

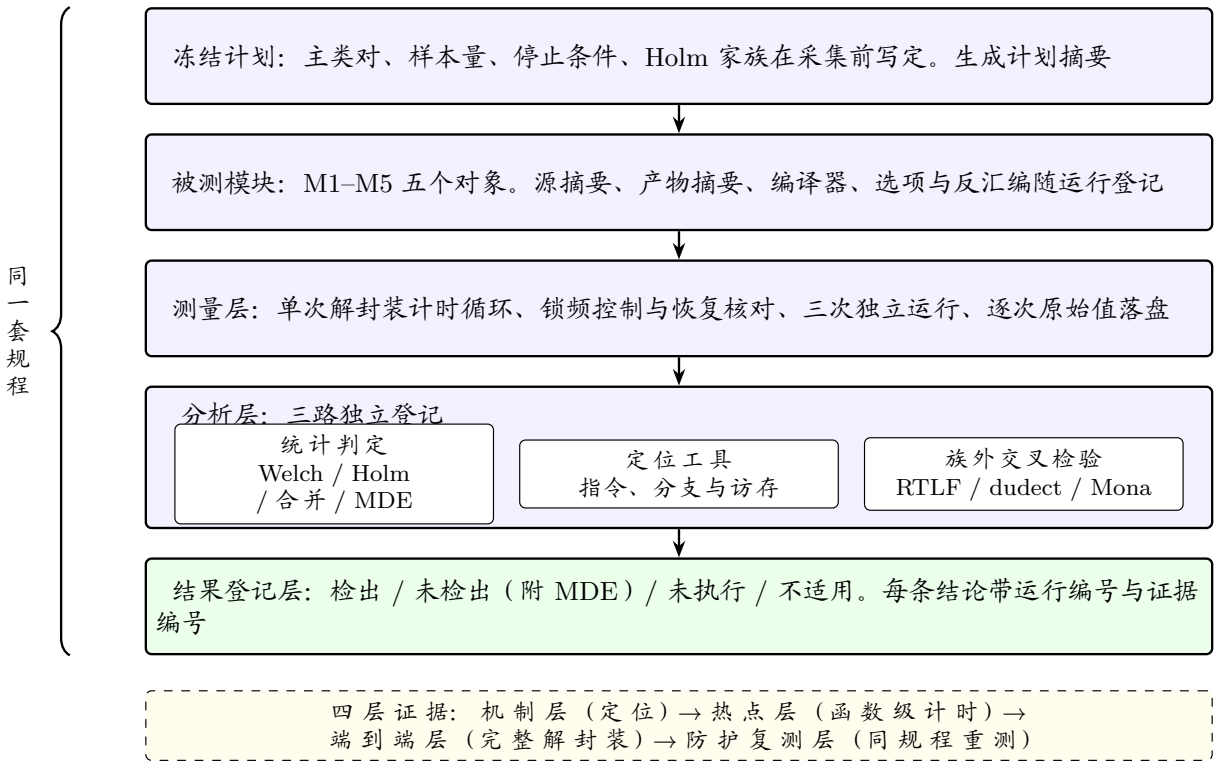



图 3.3　平台总体架构与证据分层

记录与原始值，可以重算摘要、重跑统计、核对恢复记录。不需要信任报告里的叙述。被拒绝的运行同样登记，编号连续，审计时能看到“尝试过但没通过”的记录，而不是只看到成功的样本。

四层证据是登记时的四个层次。机制层看源码、反汇编与动态标记能否指出秘密相关行为。热点层看函数级微基准或工具定位到的热点是否显示时间差。端到端层看完整解封装的类对检验。防护复测层看修复后在同一规程下重测的结果。

四层的判据不同：统计层的“检出”是预注册检验拒绝，“未检出”是预先规定的检验未拒绝零假设。定位层的“检出”是工具稳定命中且能归因到函数、指令或源码行，动态工具的“未检出”只覆盖执行过的输入。防护复测层的“检出”是修复后目标机制仍在或计时差仍可区分。四种状态互斥：检出、未检出、未执行、不适用。未执行是没做，不适用是机制与步骤的组合不可能出现。

四种状态里最容易混淆的是“未检出”和“未执行”。未检出有样本、有检验、有 MDE；未执行则记录缺少的条件与受影响条目。附录 A 与第 6.2 节列出这些记录，不计入阴性。

如果不分层，M2 的“定位阳性、计时阴性”会被写成自相矛盾的一句话。分成机制层与端到端层登记后，两句话分别属于不同层，可以同时为真，而且各自带各自的判据与预算。代价是同一模块要做多次登记，报告必须保留“未执行/不适用”行，篇幅增加。分析层三路还要各自维护输入清单，换工具就要重跑对应格。换来的是任何一条结论都不会被另一层的证据误读。

## 3.4 被测对象与模块设计

五个对象不是“五个例子”，而是三类角色：真实历史案例、人工校准与阴性对照。每个角色回答不同的问题：真实案例检验平台对真实编译产物能不能检出，人工校准检验工具能不能识别目标机制，阴性对照检验会不会冤枉好人。表 3.2 列出五个对象的固定版本、编译条件、泄漏机制与修复或对照，表中的提交号只用于定位历史版本，不表示对这些版本的推荐或背书。

表 3.2　被测对象与编译条件

| 模块 | 对象与固定版本 | 编译条件 | 泄漏机制 | 修复或对照 |
|---|---|---|---|---|
| M1 | liboqs 0.10.0，ML-KEM-768 初始公开草案的可移植 C 路径 | clang 18.1.3，面向体积优化，关闭向量化与 AVX2 分发 | 编译器把消息编码还原为按位测试后条件跳转 | 官方修复提交 982c762 |
| M2 | pq-crystals Kyber768 参考实现，未修快照 a621b8d | clang 18.1.3，最低优化级别（-Oz） | 三处秘密被除数除以公开模数 | 官方修复提交 272125f。另一提交 dda29cc 只修第一处 |
| M3 | 基于 mlkem-native，在消息编码路径插入秘密下标查表 | 与 M5 相同 | 秘密下标访存（人工埋入） | 开启查表版（on 版）/消除查表版（off 版），输出逐字节相同 |
| M4 | 独立 32 B 比较函数 | 独立编译，i386，-O1 | 发现首个不同字节即返回（人工埋入） | 脆弱实现（vulnerable 版）/常数时间实现（ct 版） |
| M5 | mlkem-native 固定版本 b3ba7b3 | 参考 C 路径 | 设计为常数时间 | 阴性对照 |

M1 与 M2 是真实历史案例。M1 的可移植 C 路径在特定编译器与选项下被还原出分支，M2 的历史 Kyber768 参考实现出现三处秘密除法 [2]。两者都有上游修复可比对，因此能同时回答“检出”与“修复后是否消失”。代价是真实案例不能任意控制变量：M1 的构建要额外处理运行时分发，M2 的除法只在特定优化级别出现。

M3 与 M4 是人工校准夹具：M3 在常数时间基线的消息编码路径上插入秘密下标查表，M4 用独立的 32 B 比较函数对比提前返回与常数时间两个版本。两版输出逐字节相同，确保校准的是检测器而不是算法语义。人工夹具回答“工具能不能识别目标机制”，不回答“真实库有没有这个缺陷”。M5 是阴性对照，设计为常数时间，用来估计工具误报。它在七种工具上的误报为 0，这个数字只说明这七种工具在这个对象、这些输入上没有报假阳性，不说明 M5 已达到常数时间。

人工夹具的设计要点是“只差目标机制”。M3 的开启查表版（on 版）与消除查表版（off

版）除查表外完全相同，M4 的两个版本除比较方式外完全相同。输出逐字节相同是先决条件，否则计时差异可能来自语义变化而不是泄漏。代价是夹具只能覆盖一种机制，不能代表完整解封装的交互。真实案例补上这一点，但真实案例又不能任意开关机制。三类角色合起来，才覆盖“能检出、能识别、不误报”三个问题。

选择历史 Kyber768 而不是最终标准作为 M2 的对象，是因为 KyberSlash 的两处修复提交落在历史代码上。最终标准改了消息编码与压缩的实现细节，同一机制在最终标准里可能已被间接消除。要检验平台能否识别这个机制，必须在机制存在的版本上做。把 M2 写成 ML-KEM-768 会把对象描述弄错。

判定用的二进制与计时用的二进制必须相同。每次运行记录源摘要、产物摘要、编译器版本、选项与反汇编摘要。采集前核对产物摘要，分析时再核对一次；M1 前期封存批次的来源在运行台账中单独标注。分析脚本拒绝没有摘要的输入，避免手工拷入的数据混入统计。

复现历史案例时，有两项编译配置直接决定待测机制是否真正执行。第一，liboqs 会根据 CPU 能力在运行时选择实现；只调整可移植构建选项，仍可能调用 AVX2 优化路径。因此需要同时关闭 AVX2 指令与对应实现开关，再用反汇编确认调用的是目标 C 函数。第二，M2 在本作品选定的 `-Oz` 体积优化构建中保留硬件 `div` 除法指令，换用其他优化配置可能被改写为乘法与移位。采集程序因此记录并核对二进制摘要，产物一变就拒绝沿用旧结论。

版本固定还意味着第三方代码只读：上游文件不改，修复以补丁形式叠加，补丁前后的源摘要与产物摘要都登记。这样一来，“官方修复”与“团队修改”不会混在同一次构建里，审计时可以分别核对。两次构建核对都进运行记录，缺一份就不能出具判定。

M1 的对象是 FIPS 203 初始公开草案，不是最终标准。草案与正式版的密钥生成采用不同的哈希输入，正式版增加了用于区分参数集的域分隔字节。KeyGen 测试专门核对这一标准演进差异，避免把版本不匹配误判为算法实现错误。对象版本必须与期望向量一致，功能结果和侧信道结论才能正确解释。

如果不绑定产物，就会出现“判定的是 A 二进制、计时的是 B 二进制”的情况：定位工具在带调试信息、未优化的构建上命中，计时却在发布构建上采集，两条结论无法拼到一起。产物绑定把每次判定压到一个摘要上，任何一次重跑都能回答“当时测的是哪个文件”。代价是每换一次编译器或选项都要重新构建、重新记录摘要，跨环境实验的构建次数因此成倍增加。

## 3.5 测量层设计

受控测量采用六项措施：将采集进程绑定到 CPU2 或 CPU6；关闭睿频并将频率上下限请求设为 4.30 GHz；将调速策略设为性能模式；在窗口外预热 100 次；按公开种子随机交错两类密文；将内存分配、日志和 I/O 全部移出计时窗口。各项配置及频率探针随运行保存，结束后恢复原设置。

测量层的目标不是“跑得快”，而是让每次调用的耗时都能被复算、被追溯，并且每一步

的边界都清楚。计时窗口里多放一次内存分配或一行日志，32 ns 量级的效应就会被淹没。窗口外少做一次预热，前几百次调用会把缓存与分支预测的冷启动写进均值。

**算法 3** 单次解封装的计时循环

**输入：** 被测二进制，类别集合 $\mathcal{C}$，每类样本 $n$，种子 $s$，CPU 核 $k$

**输出：** 逐次记录与元数据

生成输入流。按 $s$ 做可复现的类别乱序。记录输入流摘要
在计时窗口外预热 100 次
**for** 每个（类别，密文）按乱序 **do**
    $t_0 \leftarrow$ Clock()
    Decaps(密文)　　▷ 窗口只包住一次完整解封装
    $t_1 \leftarrow$ Clock()
    写入运行编号、实现、类别、序号、$t_1 - t_0$ 与单位
**end for**
记录元数据：源摘要、产物摘要、编译器、选项、时钟、频率、CPU、计划摘要
失败样本单列。不删普通长尾。不扣读钟开销

算法 3 回答的是一次调用的耗时怎样被测到、怎样被落盘。它不回答这个耗时是否可区分，也不回答测量环境是否受控，后者由算法 4 与独立运行设计负责。

计时循环还有一个容易忽略的约束：每次调用后返回值必须被用掉。编译器可能把没有用到的解封装调用整个删掉，或者把循环折叠成一次调用。把返回值写入一个随后被校验的累加器，才能保证每次调用真的发生。代价是多一次内存写，但它在窗口之外。

输入生成、内存分配、结果处理、日志与 I/O 全部在窗口外，窗口内只有一次完整解封装。类别乱序按种子生成，避免“先测完 A 再测 B”时环境漂移造成假差异。预热 100 次让缓存与分支预测进入稳定状态。原始值全量落盘，普通长尾保留，异常样本规则在计划中预先写定，不在看到数据后删点。代价是每次运行产生的逐次记录与样本量同阶，存储与整理成本随运行次数线性增长。收益是任何一次判定都能从原始值复算。

读钟开销不扣：扣掉一个常数不改变两类均值差，却会让原始值失去可复算性。读钟开销约 17.4 ns，已在分辨率说明里单列。窗口内的调用返回值必须被用掉，窗口外的日志与内存分配全部移走。这两条比“换一个更快的时钟”更能决定数据质量。预热次数也是计划里写定的。它只负责让缓存与分支预测进入稳定状态，继续增加预热不会改变分布，只会增加机时。

**算法 4** 锁频控制与恢复核对

**输入:** 目标频率 $f_0 = 4.30$ GHz，CPU 核 $k$，探针次数 $m = 20$

**输出:** 放行 / 拒绝。进入、采中、退出三段记录

保存睿频开关、调速策略、能效偏好与频率上下限的原始快照
关闭睿频。调速策略设为性能模式。能效偏好设为性能模式。频率下限与上限均设为目标频率
用绑核工具把被测进程绑定到核 $k$
等待 1.0 s
**for** $i = 1$ **to** $m$ **do**
    读当前频率。间隔 50 ms
**end for**
**if** 全部 $|f - f_0|/f_0 \le 2\%$ 且 $\max - \min \le 50000$ kHz **then**
    放行
**else**
    拒绝，恢复快照，记录原因
**end if**
采集结束后恢复快照。读取并比较恢复核对结果

算法 4 回答的是“这次采集的频率工作点是否可核对、可恢复”。它不回答物理频率是否在每次调用中恒定，也不回答后台服务是否影响了单次调用。

探针的判据有两个:全部采样落在目标频率 ±2% 内,且最大值与最小值之差不超过 50,000 kHz。前者防止工作点偏到另一个频率档，后者防止采集期间来回跳档。任一条不满足就拒绝本次采集，恢复快照并记录原因。拒绝的运行同样进台账并保留原因。20 次探针、50 ms 间隔覆盖约 1 s，这个窗口足以看到调速器是否稳定，但不足以保证整段采集不变频。采中的跨度因此单独记录。

放行判据作用于 20 次探针，不作用于采集中的每一次调用。采集中记录频率跨度，单独登记。恢复核对只证明设置恢复到进入前的状态，不证明物理恒频。不锁频与锁频的 A/B 对照里，变异系数比为 0.964、1.006、1.089，方向不一致。锁频提供统一、可核对、可复算的工作点。本机未配置 isolcpus，SMT 开启，后台约 39 个服务在运行。这些条件如实记录，量化尺度约 10 ns。

计时精度决定了什么样的效应可以被讨论。读钟开销约 17.4 ns，最小步进 10 ns。时间戳计数器的分辨率下界为 42–43 个计数、约 9.8–10.0 ns。计数器速率为 4,291,962 kHz。32 ns 的效应只有 2–3 个分辨率单位，单次调用不可分辨，必须靠每类 10,000 次、三次运行的大样本。1 ms 短区间的频率极差为 348 kHz，相对极差 $8.1 \times 10^{-5}$，相对标准差 $5.4 \times 10^{-6}$，说明短窗内频率稳定。长期稳定性未认证，只测了短区间与逐次运行中位频率。

M4 的比较调用改用不做频率调整的原始单调时钟,窗口只包住一次 32 B 比较。它与完整

解封装使用不同的时钟与不同的窗口，数值不可直接比较，所以表 3.4 不把 $-4.09$ ns 与 $+32.3$ ns 放在同一把尺子上。

每个模块做三次独立运行：r1 种子 31337、CPU2。r2 种子 424242、CPU2。r3 种子 31337、CPU6。换种子检查结论是否依赖特定输入，换核检查是否依赖特定物理核。每类 10,000 次、三次运行合计 30,000 条。两个版本合计 60,000 条，M5 单版本 30,000 条。

三个编号不是三次重复同一件事：r1 与 r2 同核不同种子，r3 同种子不同核。三者一致说明结论不依赖输入与物理核；三者不一致则提示环境漂移或输入依赖，按预注册规则登记，不选取对结论更有利的一次。CPU2 与 CPU6 共享内存控制器与电源域，核间一致只能说明结论不依赖某个核的局部状态。

主类对是同一份确定性密文的 10,000 次重复，单次运行的区间反映的是测量噪声，不是密文间的变异。跨种子复测部分弥补这一点，但不等价于在每类内部使用大量不同密文。如果不锁频、不核对，频率漂移可以伪造类间差异：按 57 µs 的均值换算，1% 的频率偏差约对应 570 ns 的均值移动，比 32 ns 的效应大一个数量级以上。锁频协议把工作点固定并留下可复核的记录。如果 I/O 留在窗口内，一次日志写入的微秒级开销同样会淹没信号。窗口设计把这类开销全部移到计时之外。

失败样本的来源分三类：调用返回错误、读钟异常、进程被调度打断。前两类单列。第三类不删，但按计划中写定的规则标记，敏感性分析时分别用截尾均值与中位数复核。规则在采集前写定，看到数据后不再改。这条约束与统计层的预注册是同一条纪律在测量层的体现。失败率与失败原因一并进台账，便于判断采集是否整体失效。

## 3.6 统计层设计

统计层的目标不是“找到一个 $p$ 值”，而是让“检出”和“未检出”都有可比较的含义。一个只报 $p$ 值的实验，看到不显著可以加样本，看到显著可以停。这条路径最终一定通向“显著”，却说明不了效应大小与可检测下界。

每个模块的检验在采集前归为一个家族，全族 $\alpha = 0.05$，做 Holm 校正：M1、M2、M3、M4 各 6 项（3 次运行 × 2 个版本），M5 3 项，共 21 项。主类对、样本量、停止条件与家族成员一次冻结，控制器每次运行校验计划摘要。停止条件固定为每类 10,000 次 × 3 次运行，不因中间结果延长或缩短。M1 的家族沿用前期封存计划，其余四份计划在采集前由同一提交冻结。

预注册的载体是一份计划文件：主类对的定义、每类样本量、三次运行的种子与核、家族成员、停止条件、异常样本规则都写在里面，采集前生成摘要。控制器每次启动校验摘要，摘要不符就拒绝运行。审计时独立复算摘要，确认计划没有被改过。计划文件与运行记录一起归档，任何一条判定都能回答“当时按哪份计划做的”。三次独立运行而不是一次，是为了估计运行间异质性：单次运行只能给测量噪声，三次才能算 $I^2$，并在合并时看出漂移。运行次数再增加会提高机时，按预注册的停止条件不再延长。

**算法 5** 统计判定与合并

**输入：** 两组样本 $t_0, t_1$。家族大小 $m$。$\alpha = 0.05$。功效 $1 - \beta = 0.80$

**输出：** $\Delta$、95% 区间、$p$、Holm 判定、合并 $\Delta$、$I^2$、MDE

$\Delta \leftarrow \text{mean}(t_0) - \text{mean}(t_1)$

$SE \leftarrow \sqrt{\text{var}(t_0)/n_0 + \text{var}(t_1)/n_1}$

$t \leftarrow \Delta/SE$。自由度按 Welch–Satterthwaite 计算

$p \leftarrow 2(1 - T_{df}(|t|))$。$CI \leftarrow \Delta \pm z_{1-\alpha/2}SE$

家族内 $m$ 个 $p$ 值升序排序。Holm 逐步比较 $\alpha/(m - i + 1)$

**for** 每次独立运行 $i$ **do**

    $SE_i \leftarrow (CI_{\text{high}} - CI_{\text{low}})/(2z)$

    $w_i \leftarrow 1/SE_i^2$

**end for**

$\Delta_{\text{pool}} \leftarrow \sum w_i \Delta_i / \sum w_i$。$SE_{\text{pool}} \leftarrow \sqrt{1/\sum w_i}$

$I^2 \leftarrow \max(0, (Q - (k-1))/Q)$，$Q$ 为 Cochran 异质性统计量

$\text{MDE} \leftarrow (z_{1-\alpha/2} + z_{1-\beta})\sqrt{s_0^2/n_0 + s_1^2/n_1}$

算法 5 回答的是给定样本如何得到均值差、区间、校正后的判定、合并估计与检测预算。它不回答输入类是否设计得对，也不把统计显著自动翻译成“可利用”。

选 Welch 而不是合并方差的学生 $t$，是因为两类的方差不一定相等：主类对只让消息位不同，路径长度接近，但缓存与分支预测的状态可能让两个版本的方差拉开。Welch–Satterthwaite 自由度不假设方差相等，代价是自由度略小、区间略宽。在每类 10,000 次下，这个代价可以接受。

统计量包括均值差 $\Delta$、Welch $t$ 与自由度、95% 正态近似区间、Holm 逐步校正、逆方差固定效应合并、$I^2$ 与 MDE。区间与 $p$ 值用大样本正态近似（$z = 1.9599639845$），不称为有限样本精确的 Student-$t$ 区间。三次运行按逆方差固定效应合并。M2 的 $I^2$ 为 58.3%，说明六次运行的结果一致性较弱，此时合并区间只作描述，判定以预注册的 Holm 结果为准。MDE 按 $(z_{1-\alpha/2} + z_{1-\beta})\sqrt{s_0^2/n_0 + s_1^2/n_1}$ 计算，Holm 口径取 $\alpha/m$。MDE 用于说明给定样本量下的检出能力，正式判定由预注册检验给出。

Holm 是逐步下降的校正：把家族内 $m$ 个 $p$ 值升序排序，第 $i$ 个与 $\alpha/(m-i+1)$ 比较，一旦某个不拒绝就停止，后面的全部不拒绝。它比 Bonferroni 少牺牲检验功效，同时仍然控制全族第一类错误。家族按模块划分，不把五个模块混成一个大家族：模块之间的效应机制不同，混在一起会让门槛过严，也会让一个模块的阳性掩盖另一个模块的问题。M1 的家族大小为 6，最小 $p$ 的门槛是 $0.05/6 \approx 0.00833$。门槛在计划里写定，不随观测到的 $p$ 值变化。

逆方差固定效应合并把三次运行看成同一真值的三次估计，权重取标准误平方的倒数。它假设运行之间只差抽样噪声。$I^2$ 衡量这个假设是否成立：M2 的 58.3% 说明运行间还有未受控漂移，此时合并值只作描述，不能拿来替代单次判定。合并的代价是它会把异质性平均掉，

所以每次合并都同时报告 $I^2$。

报告里同时给 Δ、区间与 $p$ 值，是因为三者回答不同问题：Δ 给效应大小，区间给估计的不确定度，$p$ 值只回答“在零假设下观察到这么大数据有多罕见”。只给 $p$ 值，读者无法判断效应是否值得修。只给 Δ，读者无法判断它是否超过噪声。

各家族的 MDE（Holm 口径）为：M1 31.39 ns，M2 25–67 ns，M3 18–34 ns，M5 14–28 ns，M4 最坏 1.47 ns。MDE 是事后功效描述，不改变已提交的判定。它回答的是“这个样本量下能看见多大的效应”，不是“真实效应有多大”。

MDE 与样本量的平方根成反比：要把 MDE 减半，样本量要翻四倍。每类 10,000 次、三次运行的选择，是在机时与可检测下界之间取的折中。M2 的 MDE 25–67 ns 大于除法可能带来的效应，这是“定位阳性、计时未检出”的直接解释，也是 6.2 节把 M2 列为局限的原因。

族外检验不并入 Holm 家族。RTLF 在 $\alpha = 0.01$、固定种子 2026101605、bootstrap 10,000 次的设置下对 M2–M5 的 42 个数据集给出 30 个未检出、12 个检出 [8]。dudect 报告在线最大 $t$[7]。Mona 用箱体分离比较两类样本的分布是否重叠。三者与 Welch 检验比较的对象不同：Welch 比较均值，RTLF 比较分布（含分位与尾部），dudect 与 Mona 是二值判据。它们的二值结论、最大 $t$ 或箱体结论都不转换成 $p$ 值参与校正，只作独立交叉印证。

族外检验的二值结论不转成 $p$ 值，原因有三：零假设不同（分布差异对均值差异）、错误控制方式不同（RTLF 用 bootstrap 控制第一类错误，dudect 用最大 $t$ 的经验阈值）、样本使用方式不同（在线检验与固定样本）。把三者并入同一个 Holm 家族，会得到一个既不是 0.05 也不是 0.01 的混合错误率。它们的位置是交叉印证：与主检验同向时增加信心，反向时提示输入类或环境有问题，而不是投票。

样本量不是先验拍定的：每类 10,000 次是在预实验中按观测标准差与目标 MDE 反推后冻结的，冻结之后不再调整。这样做的代价是方差估计本身也有不确定性，MDE 只能当作量级参考，不能当作精确的边界。正文提到 MDE 时都给区间或“最坏”值。

正态近似不是有限样本精确 $t$ 区间。MDE 是事后描述。$I^2$ 高时合并区间只作描述。这三条限定贯穿全文，4.7 节解释结果时仍按它们解读。如果不预注册，看到不显著就加样本、换分组，最终一定能“显著”。预注册把这条路径堵死，代价是失败尝试也要登记、机时成本上升，而且不能在看到更好结果时改计划。

另一条代价是探索性发现进不了主结论：90 格里后来注意到的现象只能作为族外观察登记，等下一轮计划再升级为正式检验。预注册的另一个效果是让阴性结果可以比较：未检出不是失败，而是带预算的结论。统计层的输出全部落成表格，任何读者都能用附录 C 的公式与原始值复算。

## 3.7 定位层设计

定位层把可疑行为落到函数、指令或源码行，并登记工具、输入、命中位置与上下文数量；机制命中和计时判定分别记录。

**算法 6** 秘密标记动态检查

**输入：** 被测二进制，私钥 $dk$，秘密片段 $S = dk[0{:}1152] \cup z[0{:}32]$，公开输入集合

**输出：** 错误数、上下文数、源码位置或指令

1: 在动态检查工具下启动被测程序
2: 把 $S$ 标记为未初始化（秘密）。公开输入保持已初始化
3: 运行完整解封装，覆盖接受与隐式拒绝两类路径
4: 收集“以秘密值为分支条件”与“以秘密值为访存下标”两类报告
5: 收集进入可变时延指令的秘密操作数报告
6: 归因到函数、源码行、调用栈与指令地址
7: 对未修与已修两个版本重复。比较错误数与上下文数

算法 6 检查秘密是否进入分支条件、访存下标或可变时延指令，并记录对应源码位置。秘密标记范围为私钥中参与解密的 1152 B 与隐式拒绝秘密的 32 B；公开输入保持已初始化。运行覆盖接受与隐式拒绝两类路径，报告按上下文归并：同一函数、同一源码行的多次命中记一个上下文，上下文数比错误数更能说明问题分布。

**算法 7** Binsec 有界常数时间检查

**输入：** 抽取内核，入口与终点，秘密数组 $s$，公开数组 $p$，公开长度范围 1–16 B

**输出：** 不安全（附反例）/ 有界安全 / 未知

1: 固定目标架构、优化级别与编译选项。记录二进制摘要
2: 声明 $s$ 为秘密。$p$ 与长度为公开
3: 做关系符号执行，检查控制流与访存是否只依赖公开输入
4: 若存在反例，输出反例地址与路径
5: 检查待定、截断、中止与求解器未知四类状态是否为 0
6: 负对照：同一配置下已知脆弱内核必须判为不安全

算法 7 回答的是给定入口、终点与长度范围内，控制流与访存是否只依赖公开输入。它不覆盖范围外的输入，也不检查运行时时序。有界验证的结论只在入口、终点与公开长度范围内成立。路径完整性要单独检查，搜索被截断时工具可能返回未知而进程仍然正常退出。负对照用同一配置下的已知脆弱内核，确保工具不会把脆弱代码判成安全。工具版本、补丁与调用参数在环境表里固定。换版本要重新跑受影响的格子，不能把旧结论直接搬过来。

五类工具各自回答不同的问题。反汇编回答“机器码里有没有目标指令形态”，不回答运行时是否真的触发。秘密标记的动态检查沿用 ctgrind 的思路，回答“秘密是否进入分支条件或访存下标”，不回答可变时延指令 [9]。可变时延检查回答“秘密是否进入除法等指令”，不回答缓存与分支。轨迹互信息回答“哪些指令与秘密的关联最强”，不回答时序可利用性 [10]。有界符号执行回答“给定范围内是否存在秘密相关路径”，不回答范围外的行为 [11]。工具的版本在环境表中固定，输出摘要进附录 E。

五类工具的输出先归一化成同一种证据记录：工具与版本、输入集合、命中位置、上下文数、处置结论。归一是为了交叉：同一位置被两类工具命中记两条证据，只有一类命中时保留但不升级为“确认”。处置结论只取四种状态，与 3.3 的登记口径一致。

归因要求可复现：同一位置至少在三次运行中的两次出现，且换种子后仍出现。只出现一次的命中记为“待复核”，不进入结论。反汇编命中的指令地址要与源码行对上，动态工具命中的源码行要与反汇编对上。对不上时保留两种坐标，不强行合并。

9 种工具 × 5 个模块共 45 格，31 格有真实运行产物，14 格未运行。未运行的格子写明原因，不填阴性。M5 上运行过的 7 种工具误报 0。这个数字支持“阴性对照没有被冤枉”，不支持“M5 已达到常数时间”。定位结论与计时结论分开登记：M2 的三处除法在定位层命中，端到端与微基准未检出。M1 的分支在定位层命中，端到端可区分。M4 的比较在定位层与计时层都命中。

如果只用一个工具，M2 会被读错两次。单看 dudect，未修版最大 $t$ 为 4.68、3.05、3.66，已修版反而为 36.27、30.94、61.91，阳性落在已修版上。单看 RTLF，两个版本都可能被判为可区分。只拿其中一个结论写报告，会得出“修复引入泄漏”或“两个版本都有泄漏”，与端到端 Welch 检验的未检出冲突。交叉后才知道 M2 是定位阳性、计时未检出，dudect 的阳性不能归因于除法。多工具交叉的代价是安装、版本固定与人工核对。14 格未运行也必须保留。另一条代价是工具之间的结论可能冲突，冲突本身要写进报告，不只保留对结论有利的一条。

工具覆盖的输入、入口与验证范围见附录 E；局限统一汇总于第 6.2 节。

## 3.8 防护设计与泄露渠道审计

防护不是“改源码”，而是“改完以后在机器码、功能、计时三个层面都能核验”。源码里把分支写成掩码，编译器仍可能还原成分支。功能测试全过，日志或退出码仍可能泄露验证结果。

**算法 8** 常数时间防护与产物核验

**输入：** 脆弱实现，目标机制（分支、除法、查表、提前返回或错误路径）

**输出：** 修复后二进制，定位层、功能层与计时层记录

1: 选择防护：掩码选择替换分支。乘法与移位替换秘密除法。去掉秘密下标。比较遍历全部字节。成功与失败走同一路径
2: 编译。记录编译器、选项与二进制摘要
3: 反汇编：检查目标分支、除法或查表是否消失
4: 动态检查：目标错误数应降为 0
5: 功能核验：修复前后输出逐字节一致。隐式拒绝仍然生效
6: 同规程复测：端到端可区分性、族外检验与性能开销

算法 8 回答的是“改了什么、在产物里是否生效、功能是否仍等价、同规程复测是否完

成”。它不回答修复是否在所有编译器与所有输入上都成立。表 3.3 把七类手段与对应的产物级核验并排列出。七类手段对应题面第 6 条列出的方向：消除数据相关分支、掩码选择、消除数据相关除法、消除数据相关访存、常数时间比较、固定循环上界、统一错误处理。每一类都给出实现方式与产物级核验，避免“源码改了就算防护”的写法。

表 3.3　防护手段与产物级核验

| 防护手段 | 对应模块 | 实现方式 | 产物级核验 |
|---|---|---|---|
| 消除数据相关分支 | M1 | 先清零系数，再用 16 位条件传送辅助函数按消息位做掩码选择 | 反汇编中按位测试后条件跳转消失。秘密标记报告 1→0 |
| 掩码选择 | M1、M5 | 以取负、异或、与、再异或完成选择。隐式拒绝对两个候选密钥做字节掩码 | 机器码为取负/异或/与序列。解封装函数体内无依赖比较结果的跳转 |
| 消除数据相关除法 | M2 | 用乘法与移位替代除以公开模数 | 三处整数除法指令各由 1→0（总计 3→0）。可变时延指令报告 1280/3→0/0 |
| 消除数据相关访存 | M3 | 消除查表版（off 版）去掉秘密下标查表 | 秘密标记 1280/2→0/0。互信息 1.922→0.722 bit |
| 常数时间比较 | M4 | 逐字节异或后按位或累积，循环结束统一判断 | 秘密标记 32/1→0。有界验证由不安全变为有界安全 |
| 固定循环上界 | M4、M5 | 比较循环始终遍历公开长度 | 反汇编与有界验证 |
| 统一错误处理 | M2、M5 | 比较失败时返回假密钥，返回路径与成功路径一致 | 1000 例额外输入拒绝 1000/1000。泄露渠道审计 |

M1 的防护用掩码选择替换分支：先清零系数，再由 16 位条件传送辅助函数按消息位在 0 与公开常数之间选择。辅助函数以取负、异或、与、再异或完成选择，不产生依赖秘密的跳转。M4 的常数时间比较把逐字节异或按位或累积，循环始终遍历公开长度，结束后统一判断。

**代码片段：掩码选择替换分支**

```
/* 先清零，再按消息位做掩码选择；不产生依赖秘密的分支 */
r->coeffs[8*i + j] = 0;
cmov_int16(r->coeffs + 8*i + j, (Q + 1) / 2, (msg[i] >> j) & 1);

void cmov_int16(int16_t *r, int16_t v, uint16_t b)
{
    b = -b;
    *r ^= b & ((*r) ^ v);
}
```

**代码片段：常数时间比较**

```
/* 逐字节异或后按位或累积；循环次数只取决于公开长度 */
unsigned int difference_bits = 0;
for (i = 0; i < len; i++)
    difference_bits |= (unsigned int)(secret_bytes[i] ^ candidate_bytes[i]);
return difference_bits == 0;
```

本作品的防护分为三层：对 M1、M2 真实历史案例引入上游官方补丁，核对反汇编与动态秘密标记结果，确认目标机制消失；对 M3、M4 人工夹具自主实现常数时间改造，并完成定位与计时复测，其中 M4 另有有界形式化验证；对平台自身开展输出与返回调用点审计，定位并修复退出码信息泄露。

常数时间改造还需要配合输出渠道检查。赛题第 6 条要求防护过程不能通过日志、异常或返回值泄露私钥和中间信息，因此本作品在代码改造后继续检查平台的可观察输出。对 128 个源文件中的 1,048 个输出或返回调用点做静态扫描，其中 122 个接触秘密相关数据，占 11.6%。秘密内容泄露 0 条。审计发现 1 条低危问题：功能自检分支使计时程序在不同构建下退出码不同，构成可观察信号。修复后该类问题由 1 条降为 0 条。动态差分对 7 种输入模式各运行 120 次，返回码、标准输出与标准错误逐字节一致。

那条低危问题的修复方式是把功能自检改成仅在显式参数下执行：计时模式下标准错误恒空，退出码只反映参数、I/O 或内存分配失败。修复后重新跑动态差分，7 种输入模式各 120 次的返回码、标准输出与标准错误逐字节一致。静态扫描负责找调用点，动态差分负责验证可观察行为。两者都过，才把该类问题记为 0。

审计不覆盖上游第三方代码、网络路径以及进程级元数据（打开的文件描述符、内存映射、资源用量），这些需要单独专项检查。M1 所用版本的掩码辅助函数未带值屏障，在本作品主构建的解封装函数体内未观测到由此产生的分支，但在其他编译器与选项下仍是未经验证的可移植性隐患。官方补丁的产物核验、人工夹具的自主改造与平台自身的输出审计共同构成上述三层防护。

如果只看源码，编译器可能把掩码还原成分支：M1 的源码本来用掩码，特定编译器选项下仍被还原成按位测试后条件跳转，只有反汇编与动态标记才能发现。如果只看功能测试，日志与退出码仍可能泄露验证结果：那条低危问题就是功能自检留下的可观察信号，功能测试全过并不能排除它。防护核验因此必须在机器码、功能与计时三层同时留下记录。

## 3.9 运行结果

运行结果只交代“跑了多少、频率是否受控、失败是否登记”，统计结论留给第 4 章。M1 的数据为前期封存：同一台机器、同一套锁频规程，但不是与 M2–M5 同一批采集。M2–M5 的正式采集共 54 次控制器运行，另有 6 次 M2 微基准。收尾阶段 117 次控制器运行（格式破损对照 18 次、性能开销 54 次、优化级别对照 36 次、锁频对照 9 次），三类合计 177 次。

“控制器运行”指一次经过计划校验、频率放行与退出恢复的采集单元。一次运行包含若干次调用，不等于一次解封装。M1 的封存数据不计入 177 次，统计口径仍按当时封存的家族。如果把拒绝的运行从台账里删掉，频率核对就只剩成功样本。本作品保留原编号与原因，批次按全部运行计。

以性能开销实验的 54 次运行为例，逐次实测中位频率的中位数为 4,309,722.5 kHz，相对名义 4.30 GHz 的偏差为 +0.005%。其中 1 次运行在采集期间的频率跨度为 50,310 kHz，略超 50,000 kHz 的跨度阈值，20 个采样值仍在 ±2% 带内，已如实登记。全部正式采集通过放行探针与恢复核对。失败尝试与异常样本单独登记，不并入阴性。频率偏差 +0.005% 是按逐次运行中位频率的中位数与名义值比较得到的，不是单次采样。频率核对的记录分进入、采中、退出三段，任何一段缺失都视为该次运行不合格。三次独立运行的原始值、失败样本与元数据一并归档。性能与优化级别的运行各自成批，不与主家族混算。

## 3.10 技术指标

表 3.4 用一张表汇总全平台的技术指标，便于对照题面第 2、8、9 条。指标均为本机实测值。未检出项附 MDE。如果只报已检出最小效应而不报 MDE，未检出会被读成没有效应。表中两者并列，未检出项都附预算。

表中各行分别汇总测量对象与样本规模、已检出最小效应与 MDE、定位覆盖、防护开销、功能正确性和复现测试，逐条要求与证据见附录 A。所有指标均绑定本次产物与环境，适用范围见第 1.2 节与第 6.2 节。

表 3.4　技术指标

| 指标 | 实测值 |
| --- | --- |
| 测量对象 | 一次完整 ML-KEM-768 解封装（M1 未修均值 57,010.7 ns） |
| 计时分辨率 | 读钟开销约 17.4 ns。最小步进 10 ns。时间戳计数器分辨率下界约 9.8–10.0 ns |
| 样本规模 | 每类 10,000 次 × 3 次独立运行。窗口外预热 100 次 |
| 已检出最小效应 | M1 +32.3 ns（约 0.06%）。M4 脆弱版 −4.09 ns |
| 未检出项的 MDE（Holm 口径） | M2 25–67 ns。M3 18–34 ns。M5 14–28 ns。M1 31.39 ns。M4 最坏 1.47 ns |
| 阴性对照误报 | M5 上运行过的 7 种工具误报 0 |
| 定位覆盖 | 90 格全流程排查，未检查 0 格。9 工具 × 5 模块矩阵中 31 格有真实产物 |
| 防护开销（解封装） | M1 +1.51% [1.36, 1.65]。M2 −2.92% [−3.06, −2.76] |
| 功能正确性 | M5、M3 各 80/80（100%）。M2 官方已知答案测试（KAT）100/100。M1 的 Encaps 25/25（100%）、Decaps 10/10（100%，含隐式拒绝）全部通过; KeyGen 25 例用于验证 FIPS 203 初始公开草案（IPD）与正式版在 K-PKE.KeyGen 哈希域分隔字节上的标准演进差异 |
| 复现测试 | 干净 clone 263 项通过。匿名包 CTest 5/5（软件流程验证，不产生计时结论） |

# 第四章　系统测试与结果

## 4.1 测试方案与环境

测试分六组，每组回答题面条目中的一个问题。环境是一台 x86_64 裸金属，所有时间结论只在这台机器上成立。六组是功能与语义等价、检测与定位、防护复测、功效与稳健性、性能、跨环境与多工具。分组按“回答哪一个问题”划分，不按工具划分：同一份原始数据可以在不同组里被引用，结论只在其所属组内成立。

六组与题面的对应关系：功能与语义等价对应第 1 条，检测与定位对应第 3–5 条，防护复测对应第 6、7 条，功效与稳健性对应第 4、8 条，性能对应第 8 条，跨环境与多工具对应第 9、11 条。第 2、10 条贯穿所有组。

实验采用绑核、关闭睿频、请求固定 4.30 GHz、性能调速策略、窗口外预热 100 次、输入随机交错，并将分配与 I/O 移出计时窗口。表 4.1 给出七项环境要素。处理器、操作系统、编译器、动态分析、形式化、计时源与频率控制各自固定。处理器行标注 SMT（同步多线程）开启、未配置 isolcpus（Linux 内核的 CPU 隔离参数）。换掉其中任何一项，时间结论都要重测。

表 4.1　测试环境

| 项目 | 配置 |
| --- | --- |
| 处理器 | AMD Ryzen 9 9950X（x86_64），SMT 开启，未配置 isolcpus |
| 操作系统 | Ubuntu 24.04.4 LTS，Linux 6.17，amd_pstate 频率驱动 |
| 编译器 | clang 18.1.3（主构建）。gcc 13.3.0 已安装但未用于对照实验 |
| 动态分析 | Valgrind 3.22.0（Memcheck）。Valgrind 3.25.1 + KyberSlash varlat 补丁。Intel Pin 3.28（Microwalk 的插桩引擎） |
| 形式化 | Binsec 0.11.3（关系符号执行）与 Z3 4.15.3（SMT 求解器） |
| 计时源 | 单调时钟（完整解封装）。原始单调时钟（M4 比较调用） |
| 频率控制 | 关闭睿频，性能策略，频率上下限均为 4,300,000 kHz，绑核 CPU2 / CPU6 |

表 4.1 汇总本次实验使用的处理器、编译器、工具版本与频率工作点。

操作系统与频率驱动最容易被忽略：锁频协议通过 sysfs 写频率上下限，换内核或换驱动，20 次探针的读数就可能落在不同工作点。编译器版本同样是背景信息：M1 的分支是 clang 18 在特定选项下的产物，换编译器等于换被测对象。环境表的作用是让“未检出”有明确的工具边界。

表 4.2 把五个模块、三类对象与七类实验环境排在同一张矩阵上。矩阵的行是实验类别，列是模块。空格不是遗漏，是该模块在该类实验中没有对象或不适用。

矩阵每格绑定产物摘要、种子、CPU 与判定层；矩阵之外的组合没有数据。

正式采集分三批：平台家族 54 次控制器运行、M2 微基准 6 次、收尾 117 次（格式破损

表 4.2　实验矩阵与运行批次

| 维度 | M1 | M2 | M3 | M4 | M5 |
|---|---|---|---|---|---|
| 对象类别 | 真实历史 | 真实历史 | 人工校准 | 人工校准 | 阴性对照 |
| 主类对家族 | 3 次 $\times 10^4$/类 | 3 次 $\times 10^4$/类 | 3 次 $\times 10^4$/类 | 3 次 $\times 10^4$/类 | 3 次 $\times 10^4$/类 |
| 定位层 | 反汇编、Memcheck | varlat、反汇编 | Memcheck、Microwalk | Memcheck、Binsec | Memcheck、varlat、Binsec |
| 90 格排查 | 30 格 | 30 格 | — | — | 30 格 |
| 优化级别 | 4 级 × 2 版 | — | — | — | 4 级 |
| aarch64 | 4 级 × 2 版 | — | — | — | 4 级 |
| WSL2 虚拟化 | 4 级 × 2 版 | — | — | — | 4 级 |
| 同机网络 | 2 版 × 2 次 | — | — | — | 2 次 |
| 性能开销 | 解封装 | 解封装 | 小内核 | 小内核 | 解封装 |
| 工具矩阵 | 6/9 种 | 6/9 种 | 6/9 种 | 6/9 种 | 7/9 种 |

运行批次：平台家族 54 次 + M2 微基准 6 次 + 收尾 117 次 = 177 次控制器运行。每格登记产物摘要、种子、CPU 与判定层。

18、性能 54、优化级别 36、锁频对照 9），合计 177 次。每次运行都经过计划校验、频率放行与退出恢复。M1 封存数据不计入 177 次，统计口径仍按当时冻结的家族。

被拒绝的运行与未执行项分开登记。优化级别实验中有 1 次运行因 20 次探针的频率跨度 54,629 kHz 超过 50,000 kHz 阈值被控制器拒绝，换名重跑后计入。性能批次有 1 次运行跨度 50,310 kHz 略超阈值，20 个采样值仍在 ±2% 带内，已如实登记。未执行项包括 GCC 对照、-O1/-Oz/-Og 三个优化级别、额外架构、isolcpus 与关闭 SMT、Microwalk 扩展到 M1/M2、Binsec 覆盖完整解封装、面向 CI 的回归门禁。它们在第 6.2 节按影响范围说明，不并进阴性。M1 数据来自前期封存批次，引用时单独标注，不与 M2–M5 合并。

## 4.2 功能与语义等价测试

功能测试先证明“被测对象仍是 ML-KEM”，再谈侧信道。修复版本必须与未修版本逐字节等价。被测对象的口径并不相同：M1 是 FIPS 203 初始公开草案的可移植 C 路径，M2 是历史 Kyber768 参考实现，M5 对应最终标准。三者的功能向量不能混用同一套期望值。M1 的密钥生成差异就是口径差异的直接结果。[1] 表 4.3 把五组功能结果列在一起。

表 4.3　功能正确性与语义等价

| 对象 | 测试向量 | 结果与说明 |
| --- | --- | --- |
| M5、M3 | NIST 独立向量 | 各 80/80（含隐式拒绝用例） |
| M1 两版 | NIST ACVP（自动密码算法验证程序）转换的 ML-KEM-768 向量 | Encaps 25/25（100%）、Decaps 10/10（100%，含隐式拒绝）全部通过；KeyGen 25 例用于验证 FIPS 203 初始公开草案（IPD）与正式版在 K-PKE.KeyGen 哈希域分隔字节上的标准演进差异。另有 20 例密钥校验因封装层无对应入口记为不适用 |
| M2 两版 | Kyber round3 官方 KAT | 100/100。普通构建与 ASan/UBSan（地址与未定义行为消毒器）构建测试各 7/7 |
| 修复等价性 | 同一驱动、同一公开输入 | 三对“未修 / 已修”实现的逐例输出摘要 diff 全空 |
| 额外输入 | 一半随机密文、一半逐字节篡改 | 隐式拒绝 1000/1000 按 FIPS 203 或 round3 规范生效 |

KeyGen 向量用于核验 FIPS 203 标准演进。liboqs 0.10.0 实现的是初始公开草案，草案的 K-PKE.KeyGen 哈希输入没有最终版新增的参数集域分隔字节，因此与最终版向量不同属于标准口径差异，不是实现错误。

60 例包括 Encaps 25 例、Decaps 10 例和用于标准演进差异核验的 KeyGen 25 例；其中 Encaps 与 Decaps 均全部通过。另有 20 例密钥校验因封装层没有对应入口，单列为不适用，不计入这 60 例。这 20 例不适用是因为封装层没有对应入口；该状态与向量执行结果分别登记，避免混淆接口覆盖与算法正确性。

M2 用 Kyber round3 官方 KAT，100/100。普通构建与 ASan/UBSan 构建各 7/7。M3 与 M5 用 NIST 独立向量，各 80/80，含隐式拒绝用例。三套对象的向量来源不同，是因为它们的标准版本不同：M2 是历史 Kyber768，只能用 round3 的 KAT。M5 对应最终标准，用 NIST 向量。M3 基于 M5 基线，向量与 M5 同源。把三套向量混在一起统计“总通过率”没有意义，表 4.3 因此按对象分行。

三对未修/已修实现的逐例输出摘要 diff 全空，说明修复没有改动算法语义。修复涉及三类改动：把分支改成掩码选择、把除法改成乘法与移位、把人工夹具的查表或提前返回改成固定路径。改动只落在执行路径上，不落在输出上。逐例摘要 diff 比“再跑一遍 KAT”更强：KAT 只比较最终共享密钥，摘要 diff 还把中间输出与返回码一起比较。任何一处字节变化都会留下痕迹。

1000 例额外输入（一半随机密文、一半逐字节篡改）中，隐式拒绝 1000/1000 按 FIPS 203 或 round3 规范生效。这条测试覆盖的是失败路径，不是接受路径。随机密文检验一般路径上的解析与拒绝，逐字节篡改检验比较逻辑在单个字节不同时的行为。两类输入都不接触私钥内容，只检验失败路径是否按规范返回假密钥。1000/1000 说明失败路径没有被修复破坏，不说明失败路径的耗时与成功路径不可区分。后者属于 3.3 的计时层。

ASan/UBSan 构建的 7/7 是另一条证据。普通构建只证明功能正确，消毒器构建还要证明没有越界、未定义行为与内存错误。7/7 说明修复没有引入这类问题。消毒器构建的性能不适合计时，功能测试只在普通构建上做计时，两者分开登记。

## 4.3 端到端可区分性

预注册 Holm 家族中，只有 M1 未修复版主类对与 M4 脆弱实现（vulnerable 版）被拒绝。M2、M3、M5 的家族均未拒绝任何检验。图 4.1 把七个类对的均值差与 95% 区间放在同一把尺子上：M1 未修复版与 M4 脆弱实现的区间不含 0，其余五个区间含 0。

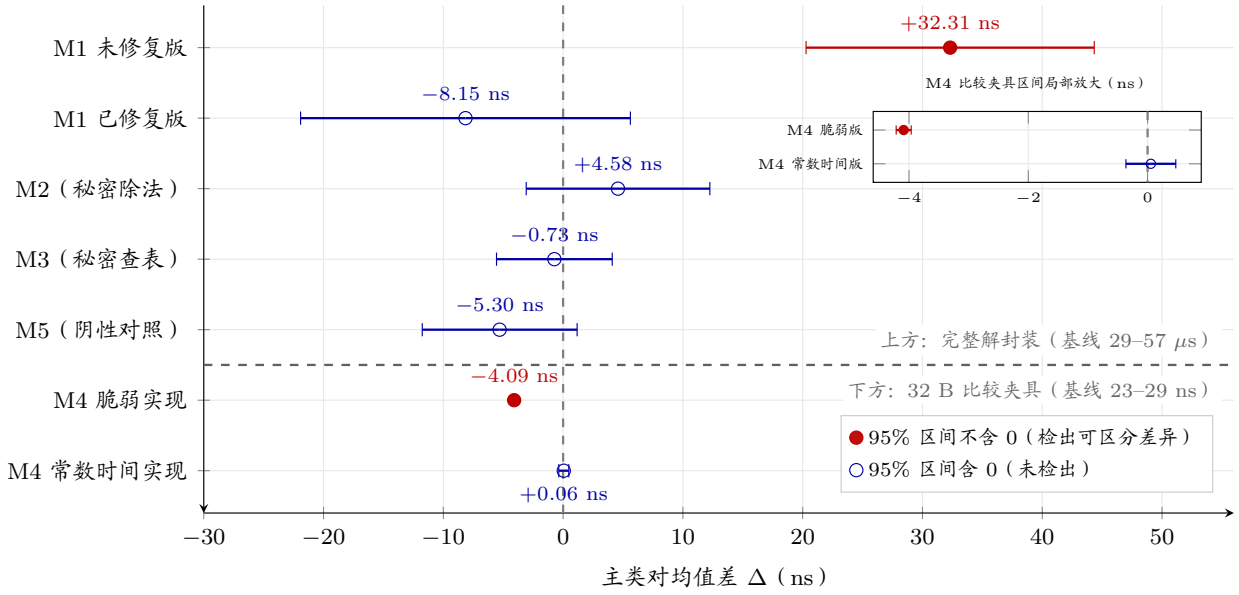



图 4.1　五模块主类对均值差与 95% 区间

家族的构成决定了门槛。M1、M2、M3、M4 各 6 项，M5 3 项。每个模块的 6 项来自 3 次独立运行 $\times$ 2 个版本或实现。Holm 是逐步下降校正：家族内 $p$ 值升序排列，第 $i$ 个与 $\alpha/(m-i+1)$ 比较，一旦某一步不拒绝就停止。M1 的 $m=6$，最小 $p$ 的门槛是 $0.05/6 \approx 0.00833$。M4 的 $m=6$，门槛相同。M5 的 $m=3$，门槛是 0.0167。门槛在采集前写进计划，不随观测到的 $p$ 值变化。

先看 M1。三次独立运行的均值差为 +39.337、+31.117、+22.337 ns，三个 95% 区间都不含 0。逆方差合并为 +32.311 ns，区间 $[20.282, 44.339]$。Holm 家族 $m=6$，最小 $p = 1.30\times10^{-5}$，低于阈值 0.00833，是家族中唯一被拒绝的检验。效应占完整解封装均值 57,010.7 ns 的 0.0567%，约 0.06%。32 ns 在 10 ns 的最小步进下只有 2–3 个分辨率单位，单次调用不可分辨，结论靠每类 10,000 次、三次运行。dudect 的最大 $t$ 为 26.41、40.64、14.92，RTLF 在未修版报告差异、在已修版未报告差异，方向与 Welch 检验一致。

三次运行的方向一致，比单次拒绝更重要。三次分别换种子、换物理核：同核不同种子两次，同种子不同核一次。三次都给出正号、区间都不含 0，说明结论不依赖某一组输入或某一

个核的局部状态。反过来说，如果三次方向不一致，即使合并后拒绝零假设，也要按预注册规则登记为不稳定，不能挑一次最有利的运行写进结论。M1 的三次结果没有触发这条规则。

效应大小与统计判定要分开读。32.311 ns 只占 57,010.7 ns 的 0.06%，在绝对值上很小。它的统计判定来自每类 10,000 次、三次运行与预注册家族，不来自效应大。把“拒绝零假设”当成“效应大”会误导读者：这个效应在单次调用上不可分辨，在本地锁频、大样本下才可分辨。

M4 的阳性更强。脆弱实现（vulnerable 版）三次运行的原始 $p$ 为 0、$9.23\times10^{-25}$、$3.14\times10^{-22}$，其中 0 为双精度下溢。Holm 家族 $m=6$，拒绝 3 项，均来自脆弱实现。合并差 $-4.089$ ns，区间 $[-4.215, -3.963]$；常数时间实现（ct 版）合并差 +0.055 ns，区间 $[-0.363, 0.473]$，三次原始 $p$ 均不拒绝。dudect 最大 $t$ 从 356.6、396.9、395.6 降至 2.06、1.90、2.27。

M4 的区间窄得几乎看不见，是因为比较调用的方差远小于完整解封装。32 B 比较的基线远小于完整解封装，单次调用没有 NTT、采样与序列化的开销，噪声源少，标准误小。同样的每类 10,000 次，M4 的 MDE 只有 0.19–1.19 ns，而完整解封装的 MDE 在几十纳秒。区间窄不等于效应大：$-4.089$ ns 在绝对值上只有 M1 的八分之一，但它的信噪比高，所以在图上反而更醒目。

效应绝对值与可检出性强弱并不同向：M4 只有 4 ns，MDE 低到 1.47 ns，信噪比高。M1 的 32 ns 更大，完整解封装的方差也大，MDE 高到 31.39 ns，信噪比反而低。判断可区分性要看区间与 MDE，不看点的位置。

M2 的家族没有拒绝项。完整解封装合并差 +4.58 ns，区间 $[-3.08, 12.25]$，含 0。按版本描述，未修版 +12.35 ns、已修版 $-11.20$ ns，$I^2 = 58.3\%$。两个版本方向相反且运行间一致性较弱，尚不能将差异归因于除法。按版本合并值不属于预注册的 6 个家族成员。家族成员是三次运行的逐次检验，按版本值只作描述。

主类对不是为除法操作数设计的：message-zero 与 message-ones 只让解密消息的位不同，除法的被除数是否随消息位系统变化没有控制。这一条在 4.7 的检测预算里再展开。M2 的 dudect 阳性落在已修版（4.68、3.05、3.66 对 36.27、30.94、61.91），不能归因于除法。4.12 把它作为单工具误读的例子。

M2 的未修版描述性合并区间为 [2.98, 21.71] ns，虽不含 0，却不属于采集前规定的六个逐次检验。六次检验经 Holm 校正后均未检出，不能用事后合并替代这一判定。$I^2 = 58.3\%$ 表示运行间差异较大，本身不识别漂移的具体来源。

M3 的合并差为 $-0.73$ ns，区间 $[-5.56, 4.10]$，Holm 家族未拒绝 0。M5 的合并差为 $-5.30$ ns，区间 $[-11.77, 1.16]$，家族 $m=3$，拒绝数为 0。M3 是人工夹具，开启查表版（on 版）与消除查表版（off 版）只差人工查表，计时层未能分辨该机制。M5 的 dudect 最大 $t$ 为 2.10、1.98、2.82，低于阈值 10。M5 的三个家族成员均未接近拒绝门槛，作为阴性对照与其余模块形成对比。

$\Delta = \bar{t}_{C_0} - \bar{t}_{C_1}$：正值表示全 0 消息类较慢，负值表示全 1 消息类较慢。

图 4.1 按模块排列主类对均值差与置信区间，M4 的两种实现相邻列示，便于比较其定位与计时结果。

族外检验给出另一层信息。RTLF 在 $\alpha = 0.01$、固定种子与 10,000 次 bootstrap 的设置下，对 M2–M5 的 42 个数据集给出 30 个未检出、12 个检出。检出的 12 个集中在 M2 主类对与 M4 脆弱实现（vulnerable 版）及其对照，M3 的 12 个与 M5 的 6 个数据集全部未检出。RTLF 与 Welch 检验比较的对象不同：前者比较分布与分位，后者比较均值。两者不互相替代，也不并入同一个 Holm 家族。[8]

族外检验不并入 Holm，是因为零假设与错误控制方式不同。RTLF 用 bootstrap 控制第一类错误，dudect 用最大 $t$ 的经验阈值，Mona 用箱体分离。把三者塞进同一个 Holm 家族，会得到一个既不是 0.05 也不是 0.01 的混合错误率。它们的位置是交叉印证：同向时增加信心，反向时提示输入类或环境有问题，而不是投票。

dudect 也属于族外。它用在线 $t$ 检验，阈值是上游默认的 10，不转换成 $p$ 值参与 Holm。M1 未修复版在三批内停止并报告泄漏，已修跑满批次仍为“可能常数时间”。M4 脆弱实现三次都报告泄漏，常数时间实现（ct 版）三次都未报告。M5 三次都未报告。[7]

主类对是同一份确定性密文的 10,000 次重复，单次运行的区间反映测量噪声，不是密文间的变异。跨种子复测部分弥补这一点，但不等价于在每类内部使用大量不同密文。10 ns 的量化尺度下，32 ns 的效应只有 2–3 个分辨率单位。这不是把 32 ns 说小，而是说明为什么必须用大样本与预注册家族。图 4.1 仅呈现均值差与区间；机制归因结合 4.4 节的函数定位证据。

## 4.4 函数级泄漏定位

定位层把每个阳性落到函数、指令或源码行。定位阳性与计时阳性是两件事，必须分开登记。同一个模块可以在定位层阳性、在计时层未检出，M2 就是这种组合。反过来，计时层阳性也必须能回到一条具体指令，否则无法判断修复是否生效。

动态检查的工作方式决定了它能回答什么。秘密标记把私钥中参与解密的 1152 B 与隐式拒绝秘密的 32 B 标成“未初始化”，公开输入保持已初始化。程序在标记下跑完整解封装，工具报告“以秘密值为分支条件”与“以秘密值为访存下标”两类事件。错误数是动态命中次数，上下文数是去重后的源码位置数。同一个循环执行 1280 次会报 1280 个错误，但只算 1 个上下文。报告里同时给两个数，是为了让读者看到“问题分布”而不是“执行次数”。

M1 未修复版的消息编码函数中，`bt` 测试秘密消息位，`jae` 根据该位决定是否跳过系数写入，因此执行路径直接依赖秘密。私钥标记下，Memcheck 报告 1 个秘密条件跳转。官方修复版用 `setb` 将测试结果转为寄存器中的 0 或 1，再调用 16 位条件传送函数 `cmov_int16`，通过取负、异或和按位与完成选择。反汇编里条件跳转消失，Memcheck 报告 0。两版的功能输出逐字节相同，说明消失的是执行路径，不是语义。[5]

反汇编上的差别可以逐条读。未修版本是“按位测试 + 条件跳转”：测试结果决定是否跳过写系数。修复版本是“按位测试 + 置位 + 调用条件传送辅助函数”：测试结果被转为 0 或 1，

辅助函数用取负、异或、与、再异或完成选择，不产生依赖秘密的跳转。修复前后的指令序列不同；修复后的选择不再通过秘密相关跳转完成。修复的代价是多了几条算术指令与一次调用，收益是分支消失。

M2 的三处整数除法分别落在消息解码、多项式压缩与向量压缩。未修版本在可变时延检查下报告 1280 errors / 3 contexts。修复版本用乘法与移位替代除法，除法计数从 3 降到 0，可变时延报告 0/0。关闭可变时延检查时，两版都报告 0，说明普通 Memcheck 看不到除法：它只检查秘密是否进入分支或访存下标，不检查指令时延是否随操作数变化。1280 次错误来自同一源码位置的多次执行，3 个上下文才是问题分布。[4]

三处除法各自对应一个上下文：消息解码一处、多项式压缩一处、向量压缩一处。三处的被除数都来自解密消息或重加密中间值，除数都是公开模数。修复用乘法与移位替代除法，三处的除法计数同时降到 0。varlat 的 1280 次错误分布在三个上下文上，说明三处都被执行到。如果只修一处，另外两处仍会报错。图 4.2 汇总了 M1 的消息位选择和 M2 三处除法在实际编译产物中的指令形态。

**M1：poly_frommsg 的消息位选择**

**x86_64 未修版**

```
0x16e: bt  %ecx,%r8d
0x172: jae 0x179
// 秘密比特决定是否跳转
```

**x86_64 已修版**

```
0x39e9: bt   %r13d,%eax
0x39ed: setb %dl
0x39f8: call cmov_int16
// 无秘密分支的掩码选择
```

**aarch64 未修版**

```
0x4b6c: sbfx w11,w11,#0,#1
0x4b70: and  w11,w11,w10
0x4be4: csel w11,wzr,w10,ge
// 条件选择，不改变控制流
```

**M2：三处秘密相关除法在修复前后**

| 函数 | 未修版（div） | 已修版（乘法与移位） |
|---|---|---|
| poly_tomsg | 0x1877: div %r9d | 0x185f: imul; 0x186d: shr |
| poly_compress | 0x16db: div %r8d | 0x16d5: imul; 0x16e1: shr |
| polyvec_compress | 0x1a7f: div %r8d | 0x1a74: imul; 0x1a7e: shr |

图 4.2　关键函数编译产物反汇编对比。注：图中 M2 地址为模块局部偏移，附录表 F2 中地址为完整驱动链接后的虚拟地址，二者对应同一机器码。

M1 未修复版片段来自锁频归档对象，已修片段来自主构建 x86_64 链接 ELF，aarch64 片段来自 aarch64 原生 ELF；M2 片段来自 KyberSlash Clang 18 -Oz 产物。地址均属于各自归档产物。

可变时延检查的边界也要写清楚。它标记的是“秘密操作数进入可变时延指令”，不是“这条指令一定产生可测差异”。除法在 x86_64 上的时延是否随操作数变化，取决于处理器微架构、操作数大小与流水线状态。工具能指出“这里有秘密操作数”，不能指出“这里一定泄漏多少纳秒”。M2 的定位阳性与计时未检出并不矛盾：前者是机制层，后者是端到端层，两层的判据不同。

M3 的 Memcheck 报告 1280 errors / 2 contexts，全部落在人工插入的秘密下标读取。消除查表版（off 版）去掉查表后为 0/0。Microwalk 在同一对象上给出互信息 1.922 bits，是全场最高。消除查表版（off 版）为 0.722 bits。Microwalk 只有 5 个公开 testcase、没有重复、没有统

计检验，互信息只作定位层的独立佐证，不升级为泄漏量。[10]

互信息 1.922 bits 的含义是“看过这条指令的访存地址后，对秘密的不确定性平均减少 1.922 bit”。它是关联强度，不是泄漏量。1.922 bits 不能换算成“恢复多少位密钥”。Microwalk 的 5 个 testcase 只覆盖 5 组公开输入，没有重复、没有统计检验。互信息高只说明这条指令与秘密关联强，不说明攻击者能利用它。

M4 的 Memcheck 报告 32 errors / 1 context，落在提前返回的比较行。Binsec 在同一对内核上判脆弱实现为 insecure，反例地址对应提前返回行; 常数时间实现（ct 版）判 bounded secure，路径探索完整。搜索深度限为 5 时，工具返回 unknown，而进程退出码仍是 0。M4 只开启控制流与访存检查，未开乘除法检查。[11]

Binsec 的反例地址对应提前返回行，说明工具找到了一条“从第几个字节开始不同”的秘密相关路径。常数时间实现（ct 版）的 bounded secure 不是“所有输入都安全”，而是“在入口、终点与公开长度 1–16 B 的范围内，没有找到秘密相关路径”。范围外的输入不在证明里。深度限 5 的 unknown 是另一个边界：工具没有证明不安全，也没有证明安全，退出码却是 0。

M5 在三种动态模式下各做 10 次完整解封装：普通 Memcheck、关闭可变时延、开启可变时延，三者都是 0 errors / 0 contexts。重加密路径的 Binsec 检查中，比较步骤 1088/1088 条路径完成，消息编码 1/1 条路径完成，没有秘密相关控制流。同一配置下 M4 的脆弱内核报告 insecure，说明工具在这套配置里不会把已知脆弱代码判成安全。Binsec 只覆盖重加密比较与消息编码，不覆盖完整解封装的所有性质。

三种动态模式的分工不同：普通 Memcheck 看分支与访存下标，关闭 varlat 看同一组输入下是否有普通标记错误，开启 varlat 看除法等可变时延指令。三者都是 0/0，说明在这组输入上没有这三类问题。换一组输入，未执行的路径不会被报告。Binsec 的 1088/1088 与 1/1 是路径计数，不是“所有路径都安全”的证明。它只覆盖重加密比较与消息编码两个内核。

定位层的输出要归一化成同一种证据记录：工具与版本、输入集合、命中位置、上下文数、处置结论。同一位置被两类工具命中记两条证据，只有一类命中时保留但不升级为“确认”。归因要求同一位置至少在三次运行中的两次出现，且换种子后仍出现，只出现一次的命中记为“待复核”。

五类工具的回答范围可以排成一条链：反汇编回答“机器码里有没有目标指令形态”，不回答运行时是否真的触发。秘密标记回答“秘密是否进入分支或访存下标”，不回答可变时延指令。可变时延检查回答“秘密是否进入除法等指令”，不回答缓存与分支。轨迹互信息回答“哪些指令与秘密的关联最强”，不回答时序可利用性。有界符号执行回答“给定范围内是否存在秘密相关路径”，不回答范围外的行为。链条上的每一环都只回答一个问题，合起来才覆盖机制层的几个侧面。

定位层结论限于各工具的输入与检查范围；M4 只覆盖 32 B 比较夹具，Binsec 只覆盖给定内核和公开长度。定位阳性与计时阳性分开登记：M1 两层都阳性，M2、M3 定位阳性而计时未检出，M4 两层都阳性，M5 两层都未检出。

## 4.5 泄漏源全流程排查

90 格排查把“六类来源 × 五个步骤 × 三个实现”逐格判定。5 个“存在”之外，还有一处更需要解释：官方修复后仍残留的同源分支。判定只有四种：存在、不存在、不适用、未检查。判“存在”必须给出可指认的函数、指令或源码位置以及触发它的输入。判“不存在”必须写明用了哪种检查、覆盖了哪些输入。判“不适用”要求该机制在这一步骤上不可能出现。判“未检查”单列，不并入阴性。

五个步骤是解密、哈希派生、重加密、比较与隐式拒绝。把哈希派生从重加密里拆出来，是因为哈希路径上的问题如果被记到重加密名下，定位只能停在“重加密有问题”这一层。拆开后，每一步都有自己的函数与指令，修复后可以逐格复查。六类来源是敏感分支、提前返回、可变时延指令、秘密下标、数据相关循环与异常路径。三个实现是 M1、M2 与 M5，人工夹具 M3、M4 不进入这张表，因为它们只覆盖单一机制，放进 90 格会把夹具结论与真实实现混在一起。

**算法 9** 90 格泄漏源排查

**输入：** 实现集合 {M1, M2, M5}。步骤集合 { 解密, 哈希派生, 重加密, 比较, 隐式拒绝 }。六类来源

**输出：** 90 格判定与证据

1: **for** 每个实现、每个步骤、每类来源 **do**
2: 检查源码与反汇编
3: 运行 Memcheck、varlat、Microwalk、Binsec（适用时）
4: 判定：存在 / 不存在 / 不适用 / 未检查
5: 记录函数、源码行或指令地址、证据编号
6: **end for**
7: 汇总：存在 5 / 不存在 75 / 不适用 10 / 未检查 0

算法 9 回答的是每一格在给定产物与检查手段下是否存在机制。它不回答该机制是否能被端到端计时分辨。表 4.4 给出汇总。分模块行按每模块 30 格减去存在与不适用导出，全表见附录 F。

表 4.4　全流程排查汇总

| 实现 | 存在 | 不存在 | 不适用 | 未检查 | 小计 |
|---|---|---|---|---|---|
| M1 liboqs 0.10.0（未修 / 982c762） | 1 | 26 | 3 | 0 | 30 |
| M2 Kyber768（a621b8d / 272125f） | 3 | 24 | 3 | 0 | 30 |
| M5 mlkem-native b3ba7b3 | 1* | 25 | 4 | 0 | 30 |
| 合计 | 5 | 75 | 10 | 0 | 90 |

*M5 此格为解封装密钥中公开密钥数据的哈希完整性检查，按公开信息例外登记，不计作秘密相关泄漏。

5 个“存在”逐格如下。M1 未修复版/重加密/敏感分支：消息编码函数中的按位测试后条件跳转，Memcheck 1 个上下文。M2 未修复版/解密/可变时延指令：消息解码中的整数除法。M2 未修复版/重加密/可变时延指令：多项式压缩与向量压缩中的整数除法。M2 未修复版与已修复版/重加密/敏感分支：消息编码函数中的同一分支在两版中都存在。M5/隐式拒绝/提前返回：上游设计行为，按有界例外登记。

这 5 格不是按“看起来可疑”选的，而是按“有可指认的函数、指令或源码位置，且有触发输入”选的。判“存在”的门槛比判“不存在”高：后者只要检查手段覆盖了输入且没有命中，前者必须给出触发路径。5 格里有 4 格在 M1/M2，1 格在 M5。M5 的那一格是上游设计行为，按有界例外登记，不算缺陷。

“不适用”的 10 格不是“没查到”，而是机制与步骤的组合不可能出现。例如某一步没有异常处理，异常路径一列就判不适用。某个函数不可能失败，提前返回一列就判不适用。判“不适用”要求给出“为什么不可能”的理由，而不是因为没时间查。10 格不适用集中在 M1 与 M2 的异常处理组合，M5 有 4 格，是因为它的错误路径设计与上游不同。

第 4 格呈现了全流程排查的超额发现：KyberSlash 官方修复提交消除了目标除法，但消息编码函数中的按位测试后条件跳转仍在。未修复版与已修复版的指令字节逐字节相同，秘密标记 Memcheck 在两版各报 256 errors / 1 context。该函数体与 M1 未修复版相同，即与 clangover 同源。主类对 message-zero 与 message-ones 使该分支在每个消息位上走系统性不同的路径，为 RTLF 在 M2 两版主类对上均报告分布差异、而格式破损对照未报告，提供了机制层候选解释。它说明只修复目标除法并不能据此认定产物中所有秘密相关机制均已消失；这一发现补充了排查覆盖，尚未确定该分支对端到端耗时的独立贡献。[1]

这一结果展示了全流程排查对单点补丁核验的补充作用：除法消失后，另一种秘密相关机制仍可能留在同一产物中。已修版三个种子的偏移分别为 +21.7、−17.1、−15.5 ns，Holm 拒绝数为 0；现有测量尚未分离代码布局和该分支的独立贡献，因此将其登记为解释 RTLF 分布差异的机器码层候选。90 格记录均包含实现与产物摘要、步骤、来源类别、函数或指令位置、触发输入和检查手段，完整结果见附录 F。

[1]官方修复提交 272125f。另一提交 dda29cc 只修第一处。提交号仅用于定位历史版本。

排查表的“不存在”也不是“永远不存在”。它只在给定产物、给定输入、给定工具下成立。换编译器、换优化级别、换输入类，结论可能变化。4.10 的 12 格就是例子：同一份源码在 -Os 有分支，在 -O0 没有。如果只查一个级别，就会把“这个级别没有”写成“这个实现没有”。90 格逐项结果与检查输入见附录 F，工具覆盖与未验证范围汇总于第 6.2 节。

## 4.6 防护前后复测

M1 与 M4 在定位层和计时层都实现了“修复前可区分、修复后不可区分”。M2 在定位层确认目标除法消失。表 4.5 把四个模块的防护前后放在同一张表上，证据层一列写明结论属于哪一层。

表 4.5　防护前后对比

| 模块 | 证据层 | 防护前 | 防护后 |
|---|---|---|---|
| M1 | 完整解封装（合并） | +32.311 ns [20.282, 44.339]，3/3 区间不含 0 | −8.150 ns [−21.916, 5.616]，3/3 区间含 0 |
| M1 | dudect 最大 $t$ / RTLF | 26.41、40.64、14.92 / 有差异 | 1.15、1.12、1.31 / 无差异 |
| M1 | 定位 | 按位测试后条件跳转。Memcheck 1 | 置位 + 条件传送。Memcheck 0 |
| M2 | 定位 | 三处整数除法。varlat 1280 / 3 | 除法计数 0。varlat 0/0 |
| M2 | 完整解封装（按版本描述） | +12.35 ns [2.98, 21.71] | −11.20 ns [−24.56, 2.15] |
| M3 | 定位 | Memcheck 1280 / 2。1.922 bits | Memcheck 0/0。0.722 bits |
| M3 | 完整解封装（按版本描述） | −0.62 ns [−7.78, 6.53]，未检出 | −0.83 ns [−7.38, 5.72]，未检出 |
| M4 | 定位 | Memcheck 32 次命中 / 1 处位置；Binsec 给出反例 | Memcheck 0；Binsec 在给定模型下有界安全 |
| M4 | 比较调用 | −4.089 ns [−4.215, −3.963]。dudect 356.6–396.9 | +0.055 ns [−0.363, 0.473]。dudect 1.90–2.27 |

复测的前提是同一规程。防护前后的计时使用同一个计时循环、同一个预注册家族、同一个功能门。修复后的二进制重新记录摘要，但输入类、样本量、种子与核不变。换一样东西，“修复后不可区分”就可能来自环境变化，而不是防护生效。M1 的修复版与未修版在同一台机器、同一锁频工作点上各跑三次。M4 的常数时间实现（ct 版）与脆弱实现（vulnerable 版）在同一驱动下各跑三次。M2 与 M3 同样成对。

四层前后对照的判据要分开写。定位层的完成判据是“目标机制在产物中消失且动态检查为 0”。计时层的完成判据是“修复后区间含 0 且族外检验不再报告”。功能层的完成判据是“输出逐字节相同”。性能层的完成判据是“开销有区间、可复算”。四层都过，才能说“这个机制在修复后不可区分”。只过一层，只能写那一层。M1 与 M4 四层都过，M2 只过定位层，

M3 过定位层与功能层、计时层未检出。

M1 的修复把消息编码分支换成掩码选择。计时层三次运行有三次区间含 0，合并 $-8.150$ ns、区间 $[-21.916, 5.616]$。dudect 最大 $t$ 从 26.41、40.64、14.92 降到 1.15、1.12、1.31。定位层从 1 个秘密条件跳转降到 0。功能层三对输出 diff 全空，性能开销 +1.51% [1.36, 1.65]。四层放在一起，才能说“这个机制在修复后既不在产物里，也不在计时上可区分”。M1 修复后的点估计为 $-8.150$ ns，95% 区间含 0；修复后的 dudect 最大 $t$ 为 1.15、1.12、1.31，均低于阈值 10。

M4 的常数时间实现（ct 版）把提前返回改成遍历全部字节、循环结束统一判断。计时层合并 +0.055 ns、区间 $[-0.363, 0.473]$。dudect 最大 $t$ 从 356.6、396.9、395.6 降到 2.06、1.90、2.27。定位层 Memcheck 从 32/1 降到 0/0，Binsec 从 insecure 变为 bounded secure；两个实现的输出逐字节相同。M4 为定位与计时双阳性、改造后双未检出的正对照。

M2 的防护效果体现在目标除法机制消失。除法计数从 3 降到 0，varlat 从 1280/3 降到 0/0。计时层修复前就是 +4.58 ns、区间含 0，修复后没有“降低”的基线可以比较。按版本描述 +12.35 与 $-11.20$ 的方向相反，$I^2 = 58.3\%$，不能直接归因于修复。M2 的修复前基线本来就是未检出，写“泄漏降低”没有可比的基线。

M2 的 dudect 阳性落在已修版，不能算作修复效果。已修版最大 $t$ 36.27、30.94、61.91，比未修版的 4.68、3.05、3.66 高得多。如果只看 dudect，会得出“修复引入泄漏”。把 dudect 与 Welch、varlat 交叉，才知道阳性落在已修版、且没有定位到除法。这不是修复效果，是另一种差异。

M3 两种实现的输出逐字节相同。消除查表版（off 版）的 Memcheck 为 0/0，互信息从 1.922 bits 降至 0.722 bits；计时层两版均未检出。该改造用于校准人工插入的秘密下标查表机制。

四条防护路径的代价不同。M1 的掩码选择增加少量算术指令与一次调用，完整解封装开销为 +1.51%；M2 以乘法与移位替代除法，开销为 $-2.92\%$；M3 去掉查表，开销为 +3.60% 且区间较宽；M4 遍历全部字节替代提前返回，32 B 比较窗口开销为 +26.21%。这些数值对应不同对象和运行窗口，按各自的成对结果解释。M2 修复提交中发现的消息编码分支另作为 90 格排查的超额发现记录（4.5）。

四条路径也说明“防护”不是一个动作。M1 改的是编译器还原出的分支，M2 改的是源码里的除法，M3 改的是人工插入的查表，M4 改的是比较函数的控制流。四条路径分别落在产物、源码与夹具层。把它们放在同一张表上，是为了看“同一套复测规程能不能覆盖不同层的修复”，而不是把四种修复当成同一种技术。

防护复测还包括泄露渠道审计。对 128 个源文件中的 1,048 个输出或返回调用点做静态扫描，其中 122 个接触秘密相关数据，占 11.6%。秘密内容泄露 0 条。审计发现 1 条低危问题：功能自检分支使计时程序在不同构建下退出码不同，构成可观察信号。修复后该类问题由 1 条降为 0 条。动态差分对 7 种输入模式各运行 120 次，返回码、标准输出与标准错误逐

字节一致。

防护手段与核验要一一对应。M1 用掩码选择替换分支，核验是反汇编里的条件跳转消失与 Memcheck 1→0。M2 用乘法与移位替换除法，核验是除法计数 3→0 与 varlat 1280/3→0/0。M3 去掉秘密查表，核验是 Memcheck 1280/2→0/0 与互信息 1.922→0.722。M4 用遍历全部字节替换提前返回，核验是 Memcheck 32/1→0/0 与 Binsec insecure→bounded secure。每条核验都落在产物或动态报告上，不落在“源码改了”这句话上。

泄露渠道审计与计时复测是两条独立的证据链。审计回答“日志、异常、返回码是否泄露秘密”，计时回答“耗时是否可区分”。一条过、一条不过，不能互相替代。那条低危问题由审计自己发现并修掉，说明防护过程也被审计。它不说明其他渠道没有遗漏，审计不覆盖上游第三方代码、网络路径与进程级元数据。

修复前后比较以逐字节功能等价为前提；团队自写的防护用于 M3、M4 人工夹具。M2 的除法修复与消息编码分支排查分别按目标机制和产物检查结果登记，见 4.5。

防护复测的结论只覆盖同一规程。换一台机器、换一个编译选项、换一组输入，修复后的可区分性可能不同。M1 的修复版在 -O0 到 -Os 四个级别都是 0 个秘密相关分支（4.10），这是产物层的跨级别证据，不是计时层的跨环境证据。把“四个级别都是 0”写成“所有环境都不可区分”会越过层级。

## 4.7 检测预算与稳健性

“未检出”表示预先规定的检验没有获得足够的差异证据。MDE 则说明在当前噪声和样本量下，检验预计对多大的真实均值差具有 80% 检出概率。稳健性分析用来检查结论是否依赖均值型检验。表 4.6 给出各家族的 MDE，按 $\alpha = 0.05$ 与 Holm 口径两列并排。

表 4.6　各家族最小可检测效应

| 家族 | MDE（$\alpha = 0.05$） | MDE（Holm 口径） | 判定 |
|---|---|---|---|
| M1 未修复版 | 25.27 ns | 31.39 ns | 单轮最大观测 +39.34 ns（合并 +32.31 ns），检出 |
| M1 已修复版 | 28.63 ns | 35.56 ns | 单轮观测 −13.21 ns（合并 −8.15 ns），未检出 |
| M2 端到端 | 19.9–54.0 ns | 24.7–67.1 ns | 6 项观测值均小于 MDE，未检出 |
| M3 端到端 | 14.2–27.1 ns | 17.7–33.7 ns | 两种实现观测值均小于 MDE，未检出 |
| M4 比较调用 | 0.19–1.19 ns | 最坏 1.47 ns | 脆弱实现（vulnerable 版）远超 MDE。常数时间实现（ct 版）0/3 超过 |
| M5 端到端 | 11.7–24.5 ns | 13.5–28.3 ns | 观测值小于 MDE，未检出 |

MDE 按 $(z_{1-\alpha/2}+z_{1-\beta})\sqrt{s_0^2/n_0 + s_1^2/n_1}$ 计算，Holm 口径把 $\alpha$ 取为 $\alpha/m$。每类 $n = 10{,}000$，

功效 $1-\beta=0.80$。MDE 与样本量的平方根成反比：要把 MDE 减半，样本量要翻四倍。M2 的端到端 MDE 为 24.7–67.1 ns，表明当前方案对更小差异的检出能力有限。主类对不是为除法操作数设计的，MDE 说明该预算对指定大小效应的检出能力，不能说明除法效应不存在。

公式里有三个假设：方差估计来自样本、均值差近似正态、功效取 0.80。每类 10,000 次下，正态近似可用。方差估计本身有不确定性，MDE 只能当量级参考，不能当精确边界。把功效从 0.80 提到 0.90，MDE 会变大。把 $\alpha$ 从 0.05 降到 0.01，MDE 也会变大。表 4.6 给的是 0.80 与 Holm 口径，换参数要重算。公式中的两项样本方差分别来自当前检验的两类输入。两类输入的方差不相等时，Welch 检验分别估计两项方差，这也是主检验用 Welch 的原因。

表中单轮观测用于说明测量灵敏度，三轮合并值用于概括总体观察，两者不能混为同一个数字。M4 比较调用的方差较小，MDE 也低于完整解封装，因此几纳秒的差异仍可能检出。正式判定由检验统计量和 Holm 校正给出，MDE 用于解释样本量是否足以发现指定大小的差异。

M2 的 MDE 为 24.7–67.1 ns，跨度来自六个检验成员各自不同的样本方差。逐项报告这一范围，比只给一个平均数更能说明各次运行的测量灵敏度。

稳健性分析对 64 个类对（其中 21 个为冻结家族成员）另用 p99.9 截尾均值差、中位数差与 Hodges–Lehmann 位置差三种方法复核。全样本方法与冻结家族的 $\Delta$ 最大绝对偏差为 0 ns。64 个类对中有 12 个在不同方法下发生判定翻转，家族成员 7 个、族外对照 5 个，但没有任何一个翻转改变 Holm 判定。

这里的“判定翻转”指未作多重校正时，个别类对随统计方法变化而跨过显著性门槛；冻结家族经 Holm 校正后的判定均保持不变。中位数与秩方法减少了长尾的影响，但本机耗时以 10 ns 为步进，大量样本数值相同，排序时出现并列秩，也会影响这些方法的检出能力。

敏感性分析的观测值取单次运行，合并值见第 4.3 节与第 4.6 节；辅助分析不改动预注册 Holm 判定。

MDE 是事后功效描述，不是事前承诺。采集前冻结的是样本量、家族与停止条件，MDE 是采集后按观测方差算出来的。它回答“这个样本量能看见多大的效应”，不回答“真实效应有多大”。采集后用观测方差估计 MDE，可以帮助规划下一轮样本量。

稳健性分析的 64 个类对不只是主类对。它们包括主类对、格式破损对照、截断对照与工具矩阵里的其他类对。21 个冻结家族成员是预注册的，43 个族外对照是描述性的。把 64 个都放进同一张敏感性表，是为了看“换一种统计量，结论会不会翻”。不是把 64 个都当成正式检验。族外对照的翻转不改变任何 Holm 判定，也不进入正文的主结论。

每类 10,000 次、三次运行兼顾了采集成本与检出能力。下一轮若要将 MDE 减半，在噪声水平相近时需将每类样本量增至四倍。

## 4.8 异常输入与接口边界

异常输入测试与内部消息测试分两级开展：本节先按题面检查合法密文与异常密文的行为，主实验再用等长合法密文深入检查内部消息位的影响。格式破损与截断是两类不同的输入：前者进入 KEM 走完整重加密，后者在封装层被公开长度检查拒绝。两组对照都在 M1 两版与 M5 上做，三个种子、每类 10,000 次、18 次控制器运行全部通过核对。

对照 A 比较合法密文与长度正确但内容破损的密文：u 字段前 960 B 置 0xFF，v 字段后 128 B 置 0x00。9 个 $\Delta$ 在 $-3.2$ 到 +9.8 ns 之间，9/9 区间含 0，原始 $p$ 均不小于 0.28。破损输入仍然通过长度检查，进入解封装后比较失败，按隐式拒绝走完整重加密。耗时与合法输入接近，符合设计预期。

这 9 组来自 3 个对象与 3 个种子的组合。格式破损只改变内容，输入长度仍为 1088 B，因此可用于检查完整隐式拒绝路径与合法路径的时间差，本轮未检出显著差异。

对照 B 比较合法密文与截断为 1087 B 的密文。截断输入的单次调用耗时 23.1–23.4 ns，与实现无关，说明它在进入 KEM 前就被封装层的公开长度检查拒绝。$\Delta$ 约为 29.3 $\mu$s（M5）与 55.1 $\mu$s（M1）。长度是公开信息，按固定路径拒绝是接口语义，不构成秘密相关泄漏，因此这组对照不进入主检验。

异常输入按上述两类分别登记：格式破损用于检查完整拒绝路径，截断用于检查公开长度边界。两组各按每类 10,000 次采集，与第 4.2 节的篡改、随机密文功能检查共同覆盖题面的异常输入要求。主类对则进一步控制内部消息位，检查等长合法密文之间更细微的时间差。

## 4.9 性能开销

防护开销必须用成对分块测量，不能用“前后各跑一次”的差值。题面第 4 条要求报告均值、方差或标准差、分位数与置信区间：表 4.7 汇总逐调用指标，第 4.7 节给出分块稳健性复核，附录 D 列出逐次运行区间。M4 为 32 B 比较窗口，其余为完整解封装。

表 4.7　防护实现与比较夹具运行时间

| 实现与用例 | n | 均值（ns） | SD（ns） | 均值 95% CI（ns） | p50（ns） | p90（ns） | p99（ns） |
|---|---|---|---|---|---|---|---|
| M1 未修复版 | 150000 | 57,010.7 | 564.1 | [57,007.9, 57,013.5] | 56,916 | 57,157 | 58,729 |
| M1 已修复版 | 150000 | 57,873.3 | 483.8 | [57,870.8, 57,875.7] | 57,798 | 57,959 | 59,522 |
| M2 未修复版 | 150000 | 57,878.5 | 595.7 | [57,875.5, 57,881.6] | 57,809 | 58,039 | 59,762 |
| M2 已修复版 | 150000 | 56,188.0 | 703.4 | [56,184.5, 56,191.6] | 55,995 | 56,656 | 58,088 |
| M3 开启查表版（on） | 150000 | 29,128.0 | 500.2 | [29,125.4, 29,130.5] | 29,074 | 29,125 | 30,718 |
| M3 消除查表版（off） | 150000 | 30,178.2 | 2,975.1 | [30,162.9, 30,193.3] | 29,195 | 30,947 | 40,216 |
| M4 脆弱实现（首字节失配） | 120000 | 23.0 | 10.1 | [22.9, 23.0] | 20 | 30 | 31 |
| M4 常数时间实现（首字节失配） | 120000 | 29.0 | 11.0 | [29.0, 29.1] | 30 | 30 | 31 |
| M4 脆弱实现（末字节失配） | 120000 | 27.1 | 26.5 | [27.0, 27.3] | 30 | 30 | 31 |
| M4 常数时间实现（末字节失配） | 120000 | 29.0 | 11.5 | [28.9, 29.1] | 30 | 30 | 31 |
| M5 | 150000 | 29,455.7 | 431.4 | [29,453.5, 29,458.0] | 29,404 | 29,455 | 31,077 |

**表注：** M1、M2、M3、M5 为完整解封装；M4 为人工构造的 32 B 单次比较窗口，使用 `CLOCK_MONOTONIC_RAW`。SD 按完整原始调用值计算；95% CI 为合并调用级 bootstrap 描述区间，未建模块内相关性，正式稳健性以第 4.7 节分块复核和附录 D 逐次区间为准。

完整解封装行的均值与中位数相差 52–983.2 ns，p99 高于中位数 1.64–11.02 $\mu$s；最大尾部来自保留异常块的 M3 消除查表版（p99 40,216 ns）。M1 与 M2 修复前后的均值差分别约为 862.6 ns 与 1,690.5 ns，对应图 4.3 的相对开销。

图 4.3 汇总成对相对开销：M1 修复 +1.51% [1.36, 1.65]，M2 修复 −2.92% [−3.06, −2.76]，M3 消除查表版（off 版）相对开启查表版（on 版）+3.60% [0.36, 9.98]，M4 常数时间实现（ct 版）相对脆弱实现（首字节）+26.21%、（末字节）+6.85%。

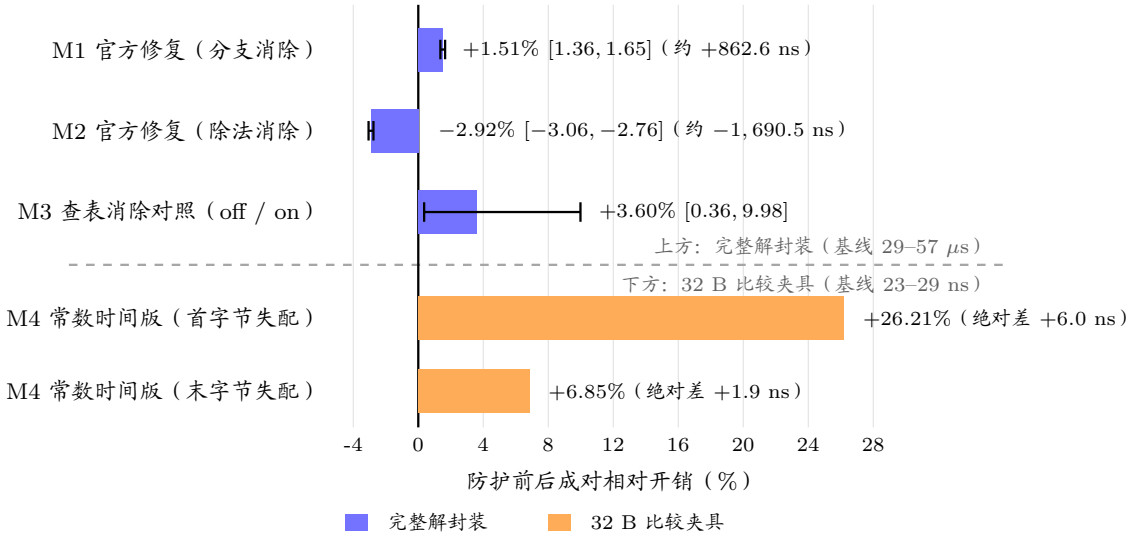



图 4.3　防护前后成对相对开销

开销测量不是“前后各跑一次”。decaps 每种实现 6 块 × 25,000 次，keygen 与 encaps 各 30,000 次，两种实现按 ABAB 交错，相对开销用 bootstrap 10,000 次给区间。交错是为了让频率漂移、缓存状态与后台服务的影响同时落在两种实现上。分块是为了让每块内部的顺序随

机化。一次“修复前跑一遍、修复后跑一遍”的差值，无法区分防护开销与环境漂移。

M1 的 +1.51% 可以拆成“多出的指令”与“多出的调用”。掩码选择把一条分支换成置位、取负、异或、与、再异或，指令数增加几条。16 位条件传送辅助函数在 -Os 下没有内联，多一次调用与返回。两部分加起来是 862.6 ns，占 57,010.7 ns 的 1.51%。这个拆法只作解释，不作为精确归因。指令级开销要看反汇编与微基准。

M1 的 +1.51% 是四次测量中最稳定的一个：区间窄，方向单一，与掩码选择多出的几条指令相符。M2 的修复反而更快，M3 的区间很宽，M4 的相对值很大但绝对差很小。三个对象的解释分别落在下面三段。

M2 的负开销来自编译器。除法被乘法与移位替代后，指令数减少，流水线停顿也减少。修复版反而比未修版快 2.92%。这不等于“修复更安全”，只说明“更安全”与“更快”可以同向，也可以反向。把负开销写成“优化”会偏离主题：本作品测的是防护开销，不是性能优化。

M3 的 +3.60% 区间 [0.36, 9.98] 较宽，主要来自消除查表版的方差较大、样本块较少。这个区间只作描述，不用它推断总体开销。M3 是人工夹具，开启查表版（on 版）与消除查表版（off 版）只差一张查表。查表的开销在完整解封装里占比很小，测量噪声容易淹没它。把 3.60% 写成“查表的开销”会高估精度。

M4 的 +26.21% 与 +6.85% 是相对值，绝对差只有 4 ns 量级。首字节与末字节的差异不同，是因为比较提前返回的位置不同：脆弱实现（vulnerable 版）在第一个字节不同时最快返回，在最后一个字节不同时最慢。常数时间实现（ct 版）始终遍历全部字节。两个百分比不能平均，也不能与完整解封装的百分比比较。

把相对开销换算成绝对时间：M1 每解封装多 862.6 ns，M2 少 1,690.5 ns。M3 的 +3.60% 按 29,455.7 ns 的参考均值换算约 1.06 $\mu$s，但区间跨到 9.98%，绝对量的不确定性也很大。M4 的 +26.21% 只在小内核的比较调用上成立。绝对量与相对量一起看，才能判断这个开销在实际使用中的位置。

M1 的 +1.51% 与 M2 的 −2.92% 方向相反，说明“防护一定变慢”不成立。掩码选择多出指令，除法替代减少指令。两个修复对指令数的影响不同，开销的方向也不同。把这两个数放在同一张图上，是为了让读者看到方向，而不是为了比较大小。不同对象的百分比不能直接排序。

M4 的 +26.21% 与 +6.85% 只用于说明“小内核的相对开销可以很大”。32 B 比较的基线远小于完整解封装，多几条指令就会带来 26% 的相对变化。换成 57 $\mu$s 的完整解封装，同样的指令数只占 0.0x%。相对开销必须绑定基线，脱离基线比较百分比没有意义。

开销的置信区间来自 bootstrap，不来自正态假设。M1 的区间 [1.36, 1.65] 窄，说明 6 块 × 25,000 次的成对测量稳定。M3 的区间 [0.36, 9.98] 宽，说明消除查表版（off 版）的方差大、块数少。区间宽时只作描述，不用它推断总体。区间窄时也不把点估计当成精确值，只说明这次测量的相对开销落在那个范围。

性能开销不是本作品的目标，防护效果才是。开销测量的作用是回答“防护的代价有多大”，不是“哪个实现更快”。把 M2 的 $-2.92\%$ 写成“性能优化”会偏离主题。报告给开销区间，是为了让读者判断这个代价是否可以接受，而不是为了给实现排名。

表中 M1、M2、M3、M5 为完整解封装；M4 是 32 B 比较夹具。M1、M2 的前后版本可比较防护开销，M3 的开启查表版（on 版）/消除查表版（off 版）描述查表夹具变化，M5 作为参考实现。跨对象均值差不作结论。

正文不单列周期数，只按名义 4.30 GHz 折算一次量级：57 $\mu$s 约合 $2.45 \times 10^5$ 个周期。实测频率中位数是 4,309,722.5 kHz，与名义值的偏差只有 +0.005%。用名义频率折算会带来同量级的系统误差，因此这个折算只用于建立量级概念，不与别的机器比较指令数，也不写进结论。

keygen 与 encaps 也各测量 30,000 次，但表 4.7 聚焦 decaps；M4 另列单次 32 B 比较夹具。不能把 decaps 的开销写成整个 KEM 的开销。

开销测量与功能测试使用同一份产物。换构建要重测，这条约束与产物绑定一致。成对分块与 ABAB 交错解决的是“环境漂移”与“顺序效应”。如果先跑未修版再跑已修版，频率漂移、缓存升温与后台服务变化都会落在已修版上。ABAB 让两种实现在时间上交错，漂移同时落在两种实现上。bootstrap 10,000 次给区间，不假设正态。区间窄说明相对开销稳定，不说明绝对耗时稳定。

表 4.7 的均值与图 4.3 的相对开销要一起读。均值给绝对量级，相对开销给可比的比例。只看均值，M1 与 M2 的差异只有 862.6 ns，容易被当成噪声。只看比例，+1.51% 又缺少绝对量级。两者并排，读者才能判断“这个开销在 57 $\mu$s 里占多少”。

## 4.10 编译选项与测量条件

同一份源码在不同优化级别与测量条件下会呈现不同的产物形态与可检出性。结论必须绑定编译条件。图 4.4 给出 12 格的合并均值差与 95% 区间：M1 两版与 M5 各四个优化级别。

12 格来自 3 个实现 $\times$ 4 个级别。3 个实现是 M1 未修复版、M1 已修复版与 M5。4 个级别是 -O0、-O2、-O3、-Os。每个格做产物判定（反汇编、除法计数、Memcheck）与锁频计时（每格 3 个种子），两列证据分开登记。

把产物判定与计时判定分开，是为了避免“分支存在但计时没测到”被读成“分支不存在”。-O2 与 -O3 正是这种情况。每格三个种子的二进制摘要逐字相同，判定用的产物与计时用的产物对齐。这一条是 12 格可比的前提。换优化级别或编译器都要重新登记产物，这条约束不随环境改变。

12 格是族外探索，不进任何 Holm 家族。它的作用是看“产物形态与可检出性是否随编译条件变化”，不是给每个优化级别下安全结论。每格 3 个种子，样本量不足以做家族检验，也不足以估计总体。结论止于“该级别下没有观察到可区分差异”。

M1 未修复版的 -Os 格合并 $-26.16$ ns，区间 $[-37.51, -14.81]$，是 12 格中唯一不含 0 的

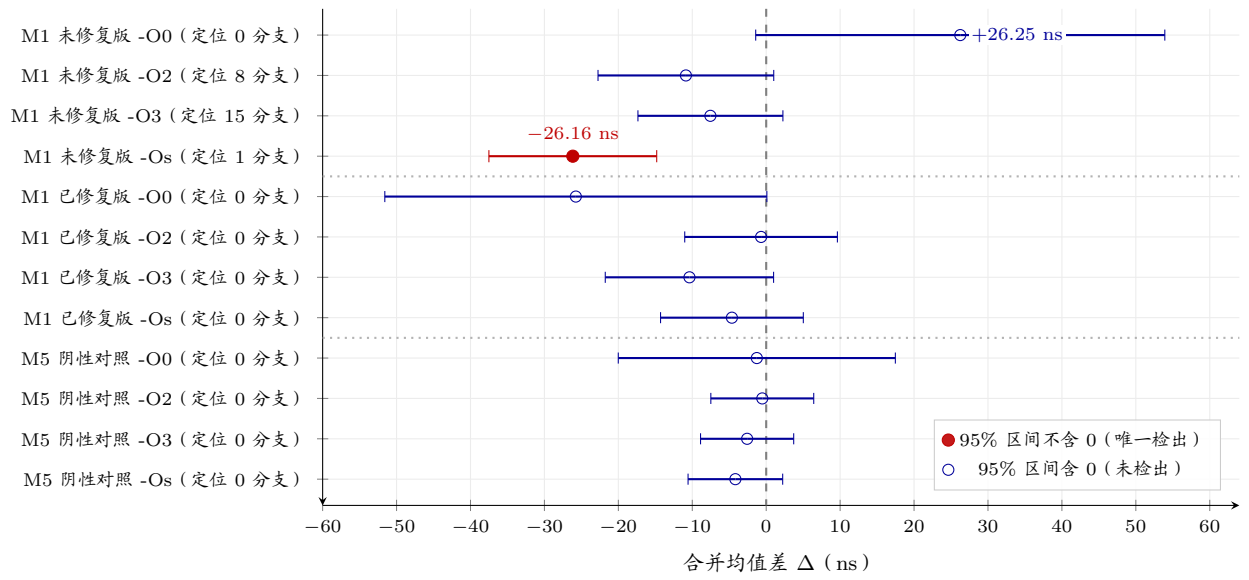



图 4.4　优化级别 12 格合并均值差与 95% 区间（$\Delta = \bar{t}_{\text{message}-\text{zero}} - \bar{t}_{\text{message}-\text{ones}}$）

区间。同一版本的 -O2 与 -O3 分别合并 $-10.86$ 与 $-7.55$ ns，区间都含 0。-O0 没有秘密相关分支，源码的算术掩码被原样保留。产物侧，-O2 与 -O3 的秘密相关分支位置数为 8 与 15，比 -Os 的 1 个更多。分支更多，计时反而未检出。修复版四个级别都是 0 个秘密相关分支，M5 四个级别都是 0，12 格都没有整数除法指令。

端到端 Δ 的符号不能单独按分支代价模型解释。主实验与优化级别实验均按 $C_0 - C_1$ 计算均值差；+32.311 ns 与 $-26.16$ ns 来自不同构建和采集批次，符号受二进制布局、对齐、指令调度与批次波动影响。可区分性按区间是否含 0 判定，不把符号单独作为机制证据。

其余 11 格的区间同样列出。M1 未修复版 -O0 合并 +26.25 ns，区间 $[-1.42, 53.92]$。-O2 $-10.86$ $[-22.75, 1.02]$。-O3 $-7.55$ $[-17.35, 2.26]$。M1 已修复版 -O0 合并 $-25.75$ $[-51.59, 0.09]$，上界紧贴 0。-O2 $-0.69$ $[-11.02, 9.63]$。-O3 $-10.38$ $[-21.77, 1.00]$。-Os $-4.64$ $[-14.29, 5.02]$。

M5 四格都在 $-5$ ns 以内，区间都含 0。只有 -Os 未修格的距离离 0 足够远。

“分支更多反而未检出”有两层解释。产物层：-O2/-O3 的每比特测试是“内存操作数加条件跳转”，形态与 -Os 的寄存器位测试不同。分支数量多，不等于单条分支的代价大。计时层：分支的代价被流水线、分支预测与乱序执行吸收，端到端均值差可能小于 MDE。-O2/-O3 的区间都含 0，说明这个预算下没有分辨出来。两层解释都指向同一个结论：不能从“分支数量”推断“可检出性”。

锁频与不锁频的对照给出另一个反直觉结果。三个对象的变异系数（CV，标准差除以均值）比为 0.964、1.006、1.089，SD 比为 0.741、0.757、0.956。不锁频的标准差反而更低。M1 未修复版在两种条件下都是 3/3 检出，M5 都是 0/3。判定一致，稳定性指标的方向却不一致。

锁频的核心价值是统一实验的频率工作点，使采集条件可核对、退出后可恢复。上述 CV 比值分布在 1 两侧，说明本机相对噪声水平接近；SD 比均小于 1，则与不锁频时频率较高、单次调用较短有关。控制器在采集前保存设置并进行 20 次频率探针核验，结束后核对恢复，从

而减少动态频率变化对前后比较的影响。

-O0 格没有秘密相关分支，是“该级别下这个机制没有出现”，不是“这份源码安全”。源码的算术掩码被原样保留，编译器没有把它还原成条件跳转。换一个编译器或换一个 -march，-O0 也可能出现分支。把 -O0 的 0/0 写成“最安全的级别”会越过产物边界。本作品只写“该级别下没有观察到该机制”。

修复版四个级别都是 0 个秘密相关分支，说明官方修复在不同优化级别上都生效。它不说明修复版在计时层也不可区分。M1 已修复版的四个级别合并值分别为 $-25.75$、$-0.69$、$-10.38$、$-4.64$ ns，区间都含 0。这是产物层与计时层分开登记的结果。产物层 0 个分支、计时层区间含 0，两者同向，但不能互相替代。

三台 WSL2 虚拟化环境的复现把“机制”和“可检出性”分开。确定性证据与主平台一致：M1 未修复版 -Os 1 个、-O2 8 个、-O3 15 个、-O0 0 个 Memcheck 上下文，修复版与 M5 全为 0，反汇编形态逐项相同。时间证据不同：三台机器上 M1 未修复版 -Os 的合并区间都含 0，M4 脆弱实现的合并区间都不含 0（$-7.13$、$-11.97$、$-22.37$ ns）。机制存在与否可以跨环境复现，数十纳秒效应的可检出性依赖测量环境。WSL2 数据不进任何 Holm 家族，也不与裸金属数据合并。

WSL2 的三台机器不是三台测量机。它们是虚拟化环境，不锁频，时间数据的标准误在数百纳秒。三台机器的一致之处是确定性证据（反汇编、Memcheck），不是时间证据。M4 脆弱实现在三台机器上都给出不含 0 的区间，说明管线在效应足够大时仍能检出。M1 未修复版 -Os 的区间都含 0，说明 32 ns 效应在虚拟化噪声下不可检出。

WSL2 的结果还说明“效应足够大时管线仍能检出”。M4 的 4 ns 效应在虚拟化噪声下仍能给出不含 0 的区间，是因为比较调用的方差小、信噪比高。M1 的 32 ns 效应在 WSL2 上反而测不到，是因为完整解封装的方差大、虚拟化噪声高。可检出性不是只由效应绝对值决定，还由噪声与样本量决定。同一台机器上，M4 的 MDE 比 M1 小两个数量级，原因就在这里。

计时精度决定了什么样的效应可以被讨论。读钟开销约 17.4 ns，最小步进 10 ns。时间戳计数器（TSC）的速率为 4,291,962 kHz，单次读数的分辨率下界为 42–43 个计数、约 9.8–10.0 ns。1 ms 短区间的频率极差为 348 kHz，相对标准差 $5.4 \times 10^{-6}$。逐调用物理恒频不可主张：锁频协议固定的是工作点，不是每一次调用的频率。

时间戳计数器的读数不是物理时间的直接测量。TSC 速率 4,291,962 kHz 是按名义频率校准的。单次读数 42–43 个计数对应 9.8–10.0 ns，是分辨率下界，不是精度。1 ms 短区间的频率极差 348 kHz、相对标准差 $5.4 \times 10^{-6}$，说明短窗内频率稳定。长期稳定性没有认证，逐调用物理恒频不可主张。

锁频/不锁频对照每组只有 3 次运行，只作描述。3 次运行不足以估计总体方差，也不足以推断其他处理器或调速策略。CV 比 0.964–1.089 只能说明“这台机器上方向不一致”，不能说明“锁频在所有机器上都不降噪”。把这条对照写成普遍结论，会超出样本边界。

## 4.11 架构与网络边界

M1 的泄漏机制与 x86_64 编译产物相关。同机网络各档在本预算下未检出。图 4.5 把架构对照与网络检测预算放在一起：左侧是同一份源码在两个架构上的产物形态，右侧是五档网络分辨同一量级效应所需的样本数。环境适用范围见第 1.2 节与第 6.2 节。

aarch64 的实验条件是“真指令集、原生二进制”。它在 Apple Silicon 的容器内运行，运行时报告的机器类型是 aarch64，不是 QEMU 翻译，也不是交叉编译后在 x86_64 上跑。同一份 liboqs 0.10.0 源码、同一个 clang 18、同一组优化级别，只换架构。这个条件让“产物形态不同”可以归因到架构，而不是源码或编译器版本。

容器环境的限制要写明：没有锁频，没有裸金属，CPU 与其他容器共享。这些条件不影响确定性证据（反汇编、Memcheck），但影响时间证据。aarch64 的 12 格只做产物判定与 Memcheck，不做锁频计时。把它的时间数据与 x86_64 合并会引入不可控的环境差异。

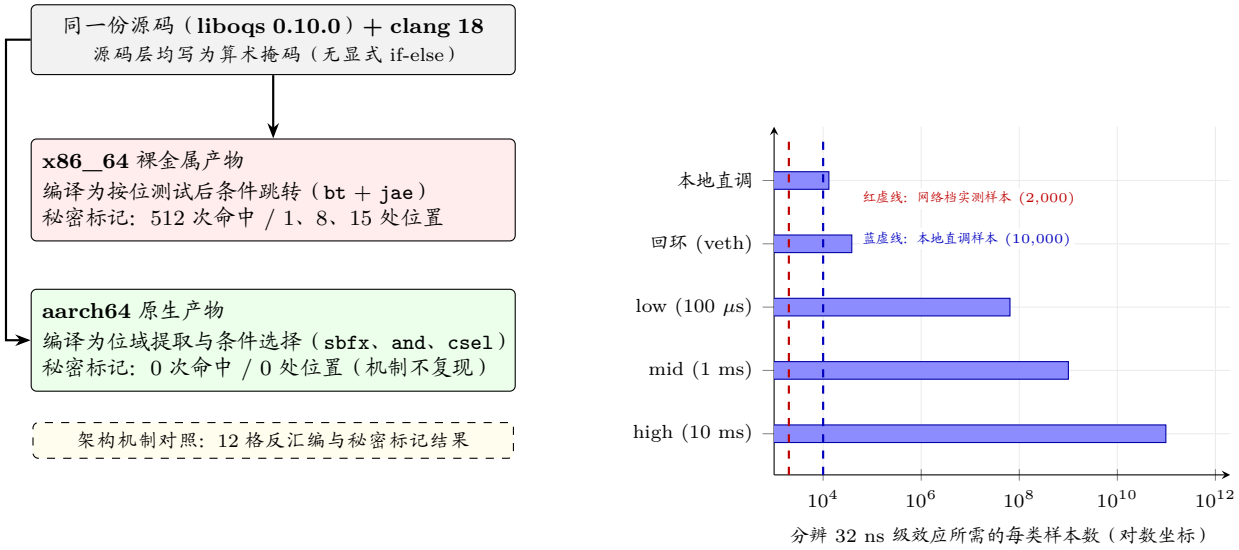



图 4.5　架构与网络边界

aarch64 的 12 格全部是 0 errors / 0 contexts。x86_64 同格的 M1 未修复版为 512 errors，其中 -O2、-O3、-Os 分别对应 8、15、1 个上下文；-O0 为 0/0。反汇编中，x86_64 使用按位测试后条件跳转（bt、jae），aarch64 使用位域提取、与掩码和条件选择（sbfx、and、csel）。12 格中的 9 格一致，3 格差异均出现在 x86_64 M1 未修复版，消息位选择的机制随实际产物而异。M5 的探针实现不完全相同：x86_64 四格与 aarch64 使用不同探针，两侧均为 0 只能作为弱支撑，支持“两套配置均未检出”，不能升级为跨架构常数时间结论。

同机网络场景用裸金属、匿名网络命名空间、自有 veth（虚拟以太网设备）与内核 netem（网络模拟器）构造两端 TCP，分五档：本地直调、回环、low、mid、high。本地直调档 M1 未修复版两次运行一次可分辨（+37.1 ns，区间 [20.4, 53.8]）、一次未检出（−7.9 ns，区间 [−25.0, 9.2]）。网络各档全部未检出。分辨同一量级效应所需的每类样本数为 13,098、38,417、64,636,555、1,003,153,317、97,586,175,893。网络各档实际每类 2,000，比所需少 1.3–7.7 个数

量级。

网络拓扑是同机两端：服务端在匿名 netns 内，绑到锁频 CPU2。客户端在宿主 netns，绑到 CPU0/1/3，不占 CPU2/CPU6。veth 两端用一对私有 /30 地址。netem 挂在 veth 两端，单向延迟与抖动分别是 100 $\mu$s/20 $\mu$s、1 ms/200 $\mu$s、10 ms/2 ms。这个拓扑能加入协议栈、排队与链路抖动，不能加入真实广域网的拥塞与路由变化。

网络五档的设计是为了分离“协议栈”“排队”与“链路抖动”三项开销。本地直调不走 socket，回环走 veth 与 TCP 但不加 netem，low/mid/high 逐级加入延迟与抖动。五档的检测预算逐级上升，说明每加入一层开销，分辨同一效应所需的样本数就增加。这个阶梯是定性的，不是对真实网络的拟合。

本地直调档的两次运行给出了两种结果：一次 +37.1 ns、区间 [20.4, 53.8]，一次 $-7.9$ ns、区间 $[-25.0, 9.2]$。两次的噪声水平接近，差别来自效应本身的运行间变动。把两次合并得到的参照效应区间含 0，只能当量级参照。这段结果说明：即使在同一台机器、同一锁频工作点，32 ns 级效应的可分辨性也不是每次都稳定。网络档每类只有 2,000，更不可能稳定分辨。

网络档的 netem 参数是单向 100 µs/20 µs、1 ms/200 µs、10 ms/2 ms，三档对应 low、mid、high。回环档不走 netem，只有 veth 与 TCP 的开销。本地直调档不走 socket，直接调用解封装。五档的设计是为了让“协议栈与排队”逐级加入，而不是一步跳到真实远程。即使最轻的回环档，所需样本数也比实际多 1.3 个数量级。加上 netem 后，所需样本数上升到 $10^7$–$10^{11}$，实际每类 2,000 远远不够。

检测预算按 $n = 2(1.96\,\mathrm{SD}/|\Delta|)^2$ 计算，与 MDE 同口径。本地直调 13,098、回环 38,417、low 64,636,555、mid 1,003,153,317、high 97,586,175,893。网络各档实际每类 2,000，回环差 1.3 个数量级、low 差 4.5 个、mid 差 5.7 个、high 差 7.7 个。预算数字不是“需要这么多才能攻击”，是“需要这么多才能分辨这个量级的差异”。两者不能混。

网络档每类 2,000 的样本量是预算与机时的折中。按本地直调的 13,098 所需样本，2,000 只有 0.15 倍。按 high 档的 97,586,175,893，2,000 差 7.7 个数量级。样本量不足时，未检出是预期结果，不是“网络没有泄漏”的证据。报告把预算与差距一起写出，是为了让读者自己判断这个阴性有多强。

netem 的三档不是“轻度、中度、重度网络”的通用定义，是本作品选的三组参数。low 档 100 $\mu$s/20 $\mu$s 接近同城链路，mid 档 1 ms/200 $\mu$s 接近跨城，high 档 10 ms/2 ms 接近跨洲。这些类比只用于读者建立量级感，不是真实测量。

本次网络实验为同机模拟，各网络档实际每类 2,000 个样本，低于预算所需 1.3–7.7 个数量级；“未检出”限于本次参数与预算。实验范围及外推边界见第 1.2 节和第 6.2 节。

## 4.12 多工具交叉验证

单工具会给出方向相反的读法。图 4.6 汇总 9 工具 $\times$ 5 模块的 45 格；当前 31 格有真实运行产物，14 格未运行。

| 检测 / 定位工具（分层归类） | M1 | M2 | M3 | M4 | M5 |
|---|---|---|---|---|---|
| [统计] Welch + Holm（主检验） | ● | ○ | ○ | ● | ○ |
| [统计] dudect（在线 $t$ 检验） | ● | △ | ○ | ● | ○ |
| [统计] RTLF（分位数重采样） | ● | △ | ○ | ● | ○ |
| [统计] Mona（箱体分离） | △ | △ | — | — | — |
| [定位] 反汇编（机器码指令形态） | ● | ● | ● | ● | ○ |
| [定位] Memcheck（秘密分支/访存） | ● | — | ● | ● | ○ |
| [定位] KyberSlash varlat（除法） | — | ● | — | — | ○ |
| [定位] Microwalk（轨迹互信息） | — | — | ● | — | — |
| [定位] Binsec（有界符号执行） | — | — | — | ● | ○ |

● 阳性（定位命中或计时可区分）　○ 阴性（未命中或未检出）
△ 已运行但与主检验或修复方向不一致　— 未运行（不作阴性计）
合计 45 格：31 格有真实运行产物，14 格未运行。M5 阴性对照上已运行的 7 种工具误报为 0。

图 4.6　多工具交叉矩阵

矩阵分为族内统计（Welch+Holm）、族外统计（dudect、RTLF、Mona）和定位工具（反汇编、Memcheck、varlat、Microwalk、Binsec）。14 个未运行格的原因分别是：Memcheck 缺 M2 完整解封装探针，varlat 缺 M1/M3/M4，Binsec 缺 M1/M2/M3 抽取内核，Microwalk 缺 M1/M2/M4/M5 接入，Mona 缺 M3/M4/M5 接入。矩阵以 2026-10-16 冻结产物为底，随后补入 2026-10-21 的 M5 Binsec 阴性结果，现为 31/45。

三类工具的回答范围不同。族内统计回答“均值差是否可区分”，族外统计回答“分布是否不同”，定位工具回答“哪里依赖秘密”。同一格在族内统计是阴性、在族外统计是阳性，不是矛盾：它们比较的对象不同。矩阵把三类放在一起，是为了让读者看到“同一对象在不同判据下可以给出不同读法”，而不是让它们投票。

M1 是五路同向的例子：反汇编看到按位测试后条件跳转，Memcheck 看到 1 个秘密条件跳转，Welch+Holm 拒绝主类对，RTLF 在未修版报告差异、在已修版未报告，dudect 最大 $t$ 在未修版为 26.41、40.64、14.92、在已修版为 1.15、1.12、1.31。五类工具回答的问题不同，结论同向。五路同向增加信心，但不等于“五倍证据”：它们共享同一份产物与同一组输入，不是五个独立实验。

M4 是人工夹具中定位层与计时层都阳性、常数时间实现（ct 版）全阴性的正对照。M5 上运行过的 7 种工具全部阴性，误报 0；该结果只覆盖本对象与本次输入。M4 的正对照同时检查定位工具能否命中已知机制、计时工具能否区分两种比较实现；M5 则检查本次工具链是否误报。误报 0 的分母是 7 个工具 × 1 个对象；每种工具只跑一组输入或一个探针，不能据此估计总体误报率。

单工具误读在 M2 上最清楚。只看 RTLF，会把 M2 读成“两个实现都可区分”：主类对 6/6

检出，未修与已修各 3 次。只看 dudect，会因阳性落在已修版（36.27、30.94、61.91 对 4.68、3.05、3.66）而得出“修复引入泄漏”。两个读法都与端到端 Welch 检验的未检出冲突。[7]

Mona 的不一致原样保留。M1 未修复版全部无分离、已修版 2/6 次分离，与 Welch、RTLF、dudect 的方向都不一致。它的“置信”字段是批次成功百分比，不是 $p$ 值，不能参与 Holm，也不能用来推翻既有结论。把 Mona 的不一致写成“工具有问题”或“结论有问题”都越界。它只说明这个工具的判据与另外三类不同。

M2 的两个误读来自不同的判据。RTLF 比较分布与分位，主类对使消息位走系统性不同的路径，分布差异可能来自分支而不是除法。dudect 用在线 $t$ 检验，阳性落在已修版，说明它测到的是修复引入的另一种差异，不是除法。两个工具都没有定位到除法热点。把它们与 Memcheck、varlat 的定位结论交叉，才能得到“定位阳性、计时未检出”的完整读法。

14 个未运行格分布在 5 个工具上：Memcheck 缺 M2，varlat 缺 M1/M3/M4，Binsec 缺 M1/M2/M3，Microwalk 缺 M1/M2/M4/M5，Mona 缺 M3/M4/M5。每一格均登记为未运行。

交叉矩阵不产生新的统计判定。它把已有工具的结论放在同一张表上，让读者看到“同一对象在不同判据下可以给出不同读法”。它不做投票，也不把多数工具的意见当成结论。主结论仍由预注册 Holm 家族给出，矩阵只负责解释与边界。

## 4.13 可复现性

可复现不是“代码在仓库”，而是“干净 clone 能跑、包内能复算、每个数字能回到原始行”。一键入口分 quick 与 full 两档：quick 不写 sysfs、不绑核，用固定输入跑功能与统计复算。full 需要锁频控制器与裸金属硬件，按冻结计划重采时间数据。

quick 档的设计目标是“在任何一台能编译的机器上跑通软件流程”。它不写 sysfs，不需要 root，不绑核。它跑功能测试、统计重算与报告生成，不产生新的时间结论。full 档才需要锁频控制器、裸金属与固定的 CPU。它按冻结计划重采，产出原始长表与运行台账。两档分开，是为了让“软件流程可复现”与“时间结论可复现”不混为一谈：前者任何机器都能做，后者需要硬件。

quick 档的输入是固定的，输出是功能测试结果与统计重算结果。它不产生新的时间数据，因此不需要锁频。full 档的输入是冻结计划，输出是原始长表与运行台账。它需要锁频控制器、绑核与恢复核对。两档的入口是同一个脚本，参数不同。把两档分开，是为了让读者知道“跑通”与“复现时间结论”是两件事。

干净 clone 的四个测试集合计 263 项通过、0 失败：platform 88、ordinary 57、sanitize 57、historical-fixed 61。匿名包解出后 CTest（CMake 测试驱动）5/5。随包提供 42 份原始长表、逐次运行台账、冻结计划、产物摘要与统计脚本。每个正文数字可以按附录 B 的运行编号回到原始行，统计判定可以按附录 C 的公式与脚本复算。

42 份原始长表是逐次运行的原始值，不是汇总表。每份长表带运行编号、模块、实现、种子、CPU 与产物摘要。逐次运行台账记录每次控制器运行的进入、采中、退出三段。冻结计

划记录主类对、样本量、停止条件与 Holm 家族。统计脚本按附录 C 的公式重算。四类材料合起来，第三方可以从原始值走到正文数字。

263 项测试是软件流程验证，不是计时复现。platform 88 项覆盖平台自身的构建与接口，ordinary 57 项覆盖普通构建，sanitize 57 项覆盖 ASan/UBSan 构建，historical-fixed 61 项覆盖历史修复版。四类合起来说明代码在各构建配置下能跑、功能测试能过，不说明时间结论能在别的机器上复现。263 项证明的是流程，不是时间。

完整锁频采集需要硬件；匿名包提供已有数据的复算流程。外部 RTLF、Mona、dudect、Binsec 保留输出摘要与调用参数，未在包内重跑。环境适用范围见第 1.2 节与第 6.2 节。

匿名包解出后 CTest 5/5，覆盖包内构建、功能测试与统计脚本；外部工具与时间数据按附录材料复核。

# 第五章　与现有工作对比及应用前景

## 5.1 泄漏检测与评估方法对比

软件时间侧信道的评估分成三条路线：统计检测、动态分析与形式化验证。本作品没有发明新的检测方法，贡献在把三条路线组织进同一个流程：所有结论都挂在编译产物上，每一步都能审计。第 3 章讲这套流程怎么搭，本章讲三条路线各自回答什么问题、答案能用在哪一步。

统计检测把程序当黑盒。TVLA[12] 比较固定类与随机类的轨迹分布，dudect[7] 把同一套判据用在在线计时上，用最大 $t$ 统计量与固定阈值给出“有/无”两种结论，RTLF[8] 比较分布与分位，并用 bootstrap 控制第一类错误。三者都只回答“两类样本能不能分开”，不回答差异来自哪条指令。类怎么配对、样本按什么顺序跑、两次运行之间有没有漂移，都会改变结论。第 4.12 节的 M2 展示了不同判据的区别：RTLF 在未修复版与已修复版均检出分布差异，dudect 的阳性落在已修复版，预注册均值检验却对两个版本都未拒绝。

动态分析进到程序内部找秘密依赖。ctgrind 沿用 Valgrind 的秘密标记，报告秘密在什么时候进入分支条件或访存下标 [9]。Microwalk 用执行轨迹与互信息，找出与秘密关联最强的指令 [10]。varlat 检查除法这类可变时延指令的操作数里有没有秘密 [4]。这类工具能指出问题在哪个函数、甚至哪条指令，但只覆盖实际执行过的输入，也不回答“这台机器上能不能测到”。定位阳性与计时阳性是两件事，M2 的三处除法正是两者分离的例子（第 4.4 节、第 4.7 节）。

形式化验证在给定模型与边界内给出另一种结论：证明控制流与访存不依赖秘密。ct-verif 把“常数时间”翻译成程序安全性的命题，在中间表示层完成证明 [13]。Binsec/Rel 用关系符号执行，在二进制层做有界检查，并给出反例路径 [11]。这类方法成本高，一般也不建模指令时延、缓存与流水线的行为，所以能回答“是否依赖秘密”，不能回答“本机计时是否可区分”。第 4.4 节 M4 的不安全判定与常数时间实现的有界安全、M5 的 1088 条路径全部通过（1088/1088），都属于有界结论。

表 5.1 把三条路线与本作品的定位放在同一组维度上。每一行的最后一列写出这个差异导致的实践后果，而不只是一个勾。

表 5.1　泄漏检测与评估方法对比

| 方法 | 本机可测性 | 函数级定位 | 产物级证明 | 未检出的量化 | 多机制与复测 |
|---|---|---|---|---|---|
| TVLA[12]、dudect[7] | 是（黑盒计时） | 否 | 否 | 一般不报，阴性无预算 | 需另行组织。后果：换产物要整体重判 |
| RTLF[8] | 是（第一类错误有界） | 否 | 否 | 随样本量提高功效，不报 MDE | 需另行组织。后果：分布差异无法归因到指令 |
| ctgrind[9]、Microwalk[10] | 否（不回答本机可测性） | 是（分支、访存、轨迹） | 否 | 不适用（只报依赖） | 单一机制类。后果：定位阳性易被读成可利用 |
| varlat[4] | 否 | 是（除法等可变时延指令） | 否 | 不适用 | 针对单一指令类。后果：只覆盖已知机制，漏掉新形态 |
| ct-verif[13]、Binsec/Rel[11] | 否 | 是（函数或内核） | 是（给定边界内。ct-verif 在中间表示层） | 不适用（二值结论） | 单一函数或内核。后果：不建模指令时延，答不了本机计时 |
| 本作品 | 是（预注册 + Holm + MDE） | 是（多工具交叉） | 部分（M4、M5 有界） | 报告 MDE 与样本量 | 5 模块统一规程、防护前后同规程复测。结论绑定产物并按证据层追溯 |

五列维度各有用途。本机可测性决定结论能不能用来做发布判断，函数级定位决定阳性结论能不能落到具体代码，产物级证明决定结论是否覆盖所有输入，未检出的量化决定阴性结论有多少说服力，多机制与防护复测决定一次评估能不能复用。五列合起来，读者才能判断一条结论该信到哪一步。

本作品把三条路线当作组件来用。同一个二进制既进定位工具，也进计时窗口。定位结果、计时判定与防护复测按证据层分开登记。每个阴性附最小可检测效应，每个未执行单列。再用 90 格排查与 9×5 工具矩阵做交叉（第 4.5 节、第 4.12 节）。代价是采集和跑工具要多花时间，收益是每个结论都能说清“在哪一层、对哪个产物、在什么预算下成立”。

这套组织的收益在证据归档，不在检测灵敏度。定位工具还是那些工具，统计检验还是那套检验。改变的是证据怎么归档：同一次运行的对象、产物摘要、编译条件与判定结果绑在一起，别人也能按同样的顺序复查。

与单机制研究的分界在问题范围。KyberSlash 研究秘密除法，clangover 追踪编译器引入的分支，Azouaoui 等量化 Kyber 解密与重加密各步骤的泄漏 [4, 5, 14]；本作品侧重检测方法的可审计性与多机制覆盖。功耗、电磁与故障注入采用不同的观测信号，属于互补方向。

**机制定位与计时结果分别报告。**M2 的三处除法在定位层稳定出现，varlat 报了 1280 处错误，集中在 3 个调用上下文；端到端与除法热点微基准均未检出，最小可检测效应为 24.7–67.1

ns。

**锁频提供统一且可核对的工作点。**不锁频与锁频的变异系数（CV）比为 0.964、1.006、1.089，三个比值没有统一的走向。锁频提供的是可核对、可恢复的工作点，让“这次采集在什么频率下完成”可被审计。

**同一机制在不同优化级别与架构上形态不同。**M1 在 -Os 下被编译成按位测试后条件跳转，-O2/-O3 下定位工具的命中更多（8 个、15 个）却计时未检出。aarch64 上位域提取与条件选择取代了分支，机制不复现。分支更多不一定更容易检出，结论必须绑定产物与环境。

表 5.1 展示三条路线各自的问题域，也呈现本作品如何将统计检测、机制定位与有界验证的结果放在同一证据框架中。“部分”指 M4、M5 的有界验证范围，不影响赛题 12 条要求的覆盖状态；附录 A 对照表中的 12 项均为满足。各路线的测量条件与方法覆盖不同，表中按证据范围比较，不据此排列总体优劣。

## 5.2 应用场景与推广边界

本作品可以作为密码库发布前检查、构建配置回归、防护方案评估与教学评测的本地工具。它不替代独立审计或形式化认证。四个场景的共同前提是：使用者手里有源码或二进制，能在一台受控机器上重复运行，并愿意记录编译条件与样本量。对 44 名密码库开发者的调查显示，开发者认为时间侧信道的防护并不难实施，真正的差距在工具有没有被采用、工程投入够不够 [15]。

后量子密码库发布前检查由库开发者使用。时机是版本冻结、发布候选版编译完成之后。输入是待发布的二进制、编译条件与公开向量（官方给出的固定测试输入）。输出是定位层的机制清单，以及固定机器上的计时判定。定位层用反汇编、Memcheck 与 varlat，成本低，可以在每次候选版上跑。计时层需要锁频与裸金属，按版本或按周在固定机器上跑。

小团队可以只跑定位层，把计时层留给有裸金属的机器。两层的结果分开登记，分别回答目标机制是否存在、本机计时能否区分。第 4.12 节的 9×5 矩阵说明，单工具既可能漏掉机制，也可能给出相反的读法，发布判定至少要两类工具结论一致。

编译器与构建配置变更回归由平台维护者使用。时机是升级编译器、改优化级别或换目标架构之前。clangover 表明这些变更会静默改变产物中的分支形态 [5]。本作品按“二进制摘要 + 优化级别”组织判定，先比对新旧产物的反汇编与 Memcheck，再在同一锁频工作点复测计时。得到的是新旧产物的差异，得不到“新版本安全”的结论：新产物可能引入新机制，也可能只是把旧机制挪了位置。面向持续集成（CI）的自动回归闸门还没有实现，列为后续工作。

防护方案的工程评估由实现者与评审者使用。时机是候选补丁进入主线之前。输入是同一份源码的防护前后两版、同一组公开向量与同一锁频规程。输出是定位结果、计时判定与成对开销。M1 的掩码选择使解封装 +1.51%，M2 的除法替代使 −2.92%，前者变慢、后者变快，说明防护的代价可快可慢，不能一概而论。得到的是“这个方案在这个产物上的代价与

效果”，得不到“所有方案中的最优选择”。

教学与评测把 M3、M4 的人工校准夹具与 M5 阴性对照当作工具测试集。时机是工具版本冻结或课程实验开始之前。教师或工具开发者用已知机制的夹具检查工具能否识别，用阴性对照检查会否误报。输入是夹具源码与固定输入类，输出是识别、漏报与误报的清单。M5 上运行过的 7 种工具误报为 0。

各场景的测量环境、攻击面与未验证项集中见第 1.2 节和第 6.2 节。本作品围绕 12 条赛题要求完成三层防护闭环：真实历史缺陷 M1、M2 接入标准补丁，并进行二进制产物级消融核验；人工校准夹具 M3、M4 实施自主常数时间改造并复测，其中 M4 另完成有界验证；平台自身的 1,048 个调用点接受静态污点审计，发现并修复 1 处退出码侧信道。对应状态与证据见附录 A，12 条要求全部覆盖并满足。KyberSlash 产物中同源消息编码分支是 90 格全流程排查得到的超额发现，说明单修除法不能替代完整产物核验，不是未完成的防护项。

最终交付是一组按产物、证据层与适用边界组织的可追溯本地记录。

# 第六章　结论

## 6.1 主要结论

本作品在 ML-KEM 解封装上建立了一条“检测—定位—防护—复测”的分层证据链，并在真实历史案例、人工校准与阴性对照上把四个环节各走了一遍。五条判断各自绑定证据层与适用边界。数字回指第 4 章与附录 D。

最直接的一条判断是：几十纳秒量级的真实泄漏可以被检出，也可以对着编译产物核验修复。M1 未修复版在主类对上的合并均值差为 $+32.311$ ns，95% 区间 $[20.282, 44.339]$，预注册 Holm 家族中 $p = 1.30 \times 10^{-5}$，低于阈值 0.00833。族外的 dudect[7] 与 RTLF[8] 给出一致结论。官方修复后合并差为 $-8.150$ ns，区间 $[-21.916, 5.616]$，三次独立运行的区间都含 0。解封装开销增加 1.51%。这条判断限定在本机：32.311 ns 只有两三个分辨率单位，依赖每类 10,000 次、三次运行与预注册家族，换产物或换机器要重新判定。

第二条结论：定位层与计时层必须分开登记。M2 的三处秘密除法在定位层稳定出现，可变时延检查记录了 1,280 次命中，集中在 3 处位置，官方修复后归零。同一产物的端到端合并差为 $+4.58$ ns，区间 $[-3.08, 12.25]$，除法热点微基准为 $+0.044$ ns，区间 $[-0.430, 0.517]$，两层都未检出。M2 的检测预算为 24.7–67.1 ns，对小于该量级的除法效应检出能力有限。-O2 与 -O3 的产物里定位工具的命中更多，计时同样未检出。定位阳性回答“机制在不在”，计时阳性回答“本机分不分得开”，两者不能互相代替。

90 格排查得到 5 格存在、75 格不存在、10 格不适用、0 格未检查，覆盖三个实现、五个解封装步骤与六类泄漏来源。KyberSlash[4] 官方修复提交消除了目标除法，但消息编码函数中与 clangover[5] 同源的分支仍在；两版对应指令字节逐字节相同，秘密标记各记录 256 次命中、集中于 1 处位置。这是全流程方法发现的历史产物机制，不构成新漏洞主张，也未作逐指令时序归因。该结果说明除法补丁不能替代对完整产物的检查。

阴性对照没有出现误报。M5 在本机、本编译产物、本类对与每类 10,000 次的预算下，合并差为 $-5.30$ ns，区间 $[-11.77, 1.16]$ 含 0，预注册家族没有被拒绝。在 M5 上运行过的 7 种工具误报为 0；结论限于本次输入、产物与矩阵。

测量条件决定可检出性。锁频提供统一的频率工作点：不锁频与锁频的变异系数比为 0.964、1.006、1.089，三个比值没有统一的走向，锁频提供的是可核对、可恢复的工作点。M1 的泄漏机制在 aarch64 上不复现，同一份源码被编译成位域提取与条件选择。同机网络的五个档位在本预算下全部未检出，要在网络上分辨同量级的效应，所需样本比本机实际高出 1.3–7.7 个数量级。可检出性由产物、架构与测量条件共同决定，时间证据不跨环境合并。

本作品围绕编译产物组织统计检测、机制定位与防护复测证据，并给出可审计判定。题面 12 条要求均有对应实现与证据，状态见附录 A。

## 6.2 局限与未验证项

本作品的结论受测量对象、测量环境、方法覆盖与证据完整度四类限制。这些限制决定结论能用到哪。

第一类是测量对象。M1 是 FIPS 203[1] 初始公开草案（ipd，initial public draft）的实现，M2 是历史 Kyber768[2] 快照，两者都不等同于最终标准。M5 才对应最终标准。M1 的计时数据是同一机器、同一锁频规程下的前期封存数据，不是与 M2–M5 同一批采集。M4 是 32 B 比较夹具，它的前后对照只说明检测链路能跑通，不外推到完整解封装。所以结论要按对象分别读，不能从一个对象推到“ML-KEM 实现”这个整体。

第二类是测量环境。全部时间结论来自一台 x86_64 裸金属与 clang 18.1.3。本机未配置 isolcpus（Linux 内核的 CPU 隔离参数），SMT（同步多线程）开启，后台约 39 个服务在运行。最小步进 10 ns，32.311 ns 的效应只有两三个分辨率单位。网络只有同机 veth 与 netem，没有真实远程链路。aarch64 只构成第二架构，不是第二台测量机，其容器未锁频、非裸金属。三台 WSL2 虚拟化复现只在机制层一致，时间数据没有进入任何 Holm 家族。所以结论不能外推到其他 CPU、其他编译器、隔离更好的机器或真实远程。

第三类是方法覆盖。M2 的主类对不是为除法操作数设计的，message-zero 与 message-ones 只让解密消息的位不同，没有控制除法里的被除数。检测预算 24.7–67.1 ns 对小于该量级的除法效应检出能力有限。90 格排查定位到修复产物中的同源消息编码分支，但尚未进行逐指令时序归因；统计判定与机制发现分别回答不同问题。

工具一侧的覆盖同样有限。Binsec[11] 是有界验证，只覆盖给定入口与范围。9×5 工具矩阵有 14 格未运行，Microwalk 只跑过 M3 一个对象。优化级别对照未覆盖 GCC 与 -O1/-Oz/-Og，M2 主实验单独采用 -Oz。

第四类是工程化覆盖。M1/M2 已核验上游标准补丁产物，M3/M4 完成人工夹具的自主常数时间改造；平台对 1,048 个输出与返回调用点完成泄露审计，并修复退出码侧信道。各项按同一规程复测，三对修复版本的输出摘要相同，支持语义保持。面向持续集成（CI）的自动回归闸门尚未接入。

四类限制不互相抵消。一条结论同时受几类限制时，按限制最严的那一类来读。四类分开写，读者才能判断这条结论该信到哪一步。

## 6.3 后续工作

后续工作按“先深化机制时序归因、再扩展功效预算、后延伸跨环境与工程化”排序。每一项都写明针对哪个局限、怎么做、做到什么算成功、要付什么代价。

第一项是进一步研究 KyberSlash 官方修复版本中已定位的消息编码分支，单独评估其时序影响。可构造屏蔽该分支的对照产物，比较反汇编、秘密标记与预注册计时结果；若用于新版本，另行核对功能向量和产物摘要。该研究用于补充机制因果分析，不代表当前防护工

作尚未完成。

第二项针对方法覆盖里主类对的缺口：M2 的输入类与样本量。做法是按除法操作数设计输入类，每类内部变化密文，并按 MDE 扩大样本。判据是能回答“残留分支存在却未检出”的原因。代价是机时增加，预注册家族需要重开。

第三项针对测量环境的量化与隔离缺口。做法是改用 rdtscp（带序列化语义的时间戳读取指令）、配置 isolcpus、关闭 SMT。判据是单次分辨率与 32 ns 效应的比值改善。代价是需要独占机器。

第四项针对方法覆盖的编译器与级别缺口。做法是补 gcc 13.3.0 与 -O1/-Oz/-Og。判据是每一格都给出产物判定与计时判定，与现有 12 格同口径。代价是每格重新构建与采集。

第五项针对测量环境的远程缺口。做法是从同机 veth 走到真实两端，测出本地与远程的检测预算比。判据是给出可复算的预算比与置信区间。代价是网络环境不可控，需要新的预注册。

第六项针对证据完整度的 CI 缺口。做法是把反汇编、Memcheck 与可变时延检查接入持续集成。判据是每次提交自动跑定位层并给出与基线的差异。代价是维护工具版本与基线，工具升级本身要重新登记。

六项的排序依据是研究依赖关系：先把 M2 的输入设计与残留机制的时序影响分开，再扩展测量环境和自动回归覆盖。

# 附录 A　题面要求对照

本附录逐条对照题面的 12 条要求，并依据仓库内可打开的证据登记状态。第 6、7 条的防护与同规程复测证据见各自条目。第 9 条的可选项分列为优化级别、架构与同机网络三组。

| 序号 | 要求 | 本作品的做法 | 章节 | 状态 | 证据 |
|---|---|---|---|---|---|
| 1 | 实现或接入符合规范的 ML-KEM 密钥生成、封装与解封装，用公开或主办方向量验证。不得改安全语义换常数时间。 | 三条实现路径接入统一接口。公开向量、修复等价与隐式拒绝测试。M1 标注为 FIPS 203 初始公开草案（ipd），M2 标注为历史 Kyber768。 | 3.1、3.4、4.2 | 满足 | A-33 至 A-35 |
| 2 | 建立可重复的高精度计时框架，说明处理器、操作系统、编译器、优化级别、频率管理、测试次数与异常样本处理。 | 单调时钟计时。锁频控制器 20 次探针与退出恢复核对。28 项环境表。原始长尾全留，失败样本单列。 | 3.5、3.9、4.1 | 满足 | A-29 至 A-31 |
| 3 | 至少两类有明确区别的解封装输入，重复次数足够。输入设计服务于泄漏分析。 | 主类对为 message-zero 与 message-ones 等长合法密文。辅以位翻转、随机、格式破损与截断。每类 10000 次、三次运行。 | 3.1、4.1、4.8 | 满足 | A-01、A-03、A-28 |
| 4 | 用统计方法判断不同类别的执行时间是否显著可区分。均值、方差、分位数、置信区间与检验并用，不得凭少量样本。 | Welch 检验、Holm 校正、逆方差固定效应合并、$I^2$ 与 MDE。RTLF、dudect、Mona 作族外交叉。 | 3.6、4.3、4.7 | 满足 | A-09 至 A-16 |
| 5 | 定位泄漏来源，说明其与解封装流程的对应关系。 | 反汇编、Memcheck、varlat、Microwalk、Binsec。3 实现 × 5 步骤 × 6 类 = 90 格逐格判定，5 格存在、75 格不存在、10 格不适用、0 格未检查。 | 3.7、4.4、4.5 | 满足 | A-17 至 A-23 |
| 6 | 对已发现问题采用常数时间比较、掩码选择、固定循环、统一错误处理或消除敏感分支等方式修改。不得通过日志、异常、返回码泄露私钥、中间值或验证结果。 | 三层防护闭环：M1/M2 接入上游标准补丁并做二进制产物级消融核验；M3/M4 人工夹具自主改造，其中 M4 完成有界形式化验证，M3/M4 均有统计复测；平台 1,048 个输出与返回调用点完成污点审计并修复 1 处退出码侧信道。122 个调用点接触秘密，未发现其他秘密泄露；同规程复测见第 7 条。审计不含 vendor 树。 | 3.8、4.6 | 满足 | A-18、A-23、A-36 |

表 A1　题面 12 条对照（一）。状态以仓库内可打开的证据文件为判据，不以文稿措辞替代实验完成。

| 序号 | 要求 | 本作品的做法 | 章节 | 状态 | 证据 |
|---|---|---|---|---|---|
| 7 | 防护后在与防护前相同或等价的测试环境下重采数据，对比不同输入类别的时间分布与统计结果，说明泄露是否明显降低。同时验证功能正确性。 | M1/M2 的上游补丁前后按同一规程复测；M3/M4 人工夹具自主改造前后在定位与计时层作同规程对照。平台调用点审计与退出码修复后复核，公开向量与语义测试通过。 | 4.6、4.9 | 满足 | A-01、A-02、A-06、A-07、A-35 |
| 8 | 给出防护前后解封装平均时间、典型分位数与性能开销。不得牺牲安全性或正确性。 | 25 行按实现版本和操作汇总、11 行成对开销；M1 修复 +1.51%，M2 修复 −2.92%，M4 比较夹具 +26.21% 与 +6.85%。 | 4.9 | 满足 | A-25 |
| 9 | 至少在一种软件环境完成完整实验。高水平作品可增加优化级别、CPU 架构或本地与网络场景。 | 12 格优化级别、12 格 aarch64、同机网络 5 档 × 3 种实现 × 2 次。aarch64 为机制对照，网络为同机模拟。 | 4.10、4.11 | 满足 | A-26 至 A-28 |
| 10 | 明确攻击者可控输入、可观测信息、本地权限与攻击条件。区分本地高精度计时与远程网络时间攻击的边界。 | 本地白盒模型、可控输入与观测量见第 1.2 节；适用范围与未验证项见第 6.2 节。 | 1.2、3.2、6.2 | 满足 | A-28、C-05、C-06 |
| 11 | 提供测试程序、数据生成方法、实验参数、编译选项与统计过程，使评测人员可复现。 | 一键入口分 quick 与 full 两档。干净 clone 263 项测试通过。vendor 取件两层 SHA-256。逐次运行台账与原始长表随附。 | 4.13、附录 G | 满足 | A-37 |
| 12 | 至少在自动化检测、敏感路径定位、常数时间实现、统计方法、低开销防护或跨平台验证之一取得改进，并用可复现实验说明效果。 | 分层证据与 MDE。预注册锁频。90 格与 9 × 5 工具矩阵。产物与架构依赖对照。 | 1.4、4.12、5.1、6.1 | 满足 | A-22、A-24、A-26、A-27 |

表 A1　题面 12 条对照（二）。状态按仓库内可打开的证据文件判定；第 6、7 条的防护和复测证据均列于同行。

题面 12 条要求均有对应实现与证据，状态全部为“满足”。防护工作形成三层完整闭环：对 M1、M2 接入标准补丁并核验二进制产物；对 M3、M4 人工夹具实施常数时间改造，以统计复测覆盖两个夹具，并对 M4 比较内核作有界形式化验证；平台对 1,048 个调用点作静态污点审计，修复 1 处退出码侧信道。90 格排查发现的 M2 同源消息编码分支作为方法超额发现记录，不改变第 6、7 条的满足状态。第 9 条的优化级别、架构与同机网络实验均已执行，各自的测量范围按表中说明。

状态判定依据是可打开的原始数据、运行记录与产物摘要。第 9 条的可选项按覆盖范围列示：优化级别有完整 12 格，aarch64 提供机制层 12 格，网络实验覆盖同机模拟 5 档；环境范围见第 1.2 节与第 6.2 节。

12 条分为基础、防护与差异三层：第 1 至 5 条覆盖功能、测量、输入、统计与定位；第

6 至 8 条覆盖防护修改、同规程复测与性能开销；第 9 至 12 条覆盖实验环境、威胁模型、复现与改进。每项结论均回指相应证据，适用范围在对应条目中说明。

证据列用的是台账编号，不是文件路径。A 类编号可在附录 B 至 F 或正文对应节找到原始记录。C 类编号是未执行项，只在边界段引用。编号与章节号并列，是为了让读者既能按章读，也能按台账核。台账编号不替代证据文件，只负责把正文的一句话指回一条可打开的记录。如果编号与文件对不上，以文件为准，并回改正文。

## 附录 B　实验与运行总表

本附录按运行家族给出逐次记录与失败清单。表 B1 覆盖平台四模块的 54 次锁频控制器运行。表 B2 覆盖 M2 除法热点微基准的 6 次运行。表 B3 覆盖收尾的 117 次锁频运行、同机网络的 30 次运行、aarch64 的 12 格与 M1 封存批次，并把失败尝试单列。控制器的“0/完成/是”表示 controller_exit = 0、reason = worker_finished、restore_match = yes。每次进入的 20 个频率探针均在 ±2% 带内且跨度不超过 50000 kHz。

| 序 | 模块 | 运行 | 实现版本 | 类别 | 每类样本 | 日期 | 控制器 | 产物 / 异常 |
|---|---|---|---|---|---|---|---|---|
| 1 | M5 | s1 31337 CPU2 | 参考 | measure 主类对 | 10000 | 10-16 | 0/完成/是 | CSV+meta / — |
| 2 | M5 | s1 31337 CPU2 | 参考 | online dudect | 10000/批 | 10-16 | 0/完成/是 | max t 日志 / — |
| 3 | M5 | s2 424242 CPU2 | 参考 | measure 主类对 | 10000 | 10-16 | 0/完成/是 | CSV+meta / — |
| 4 | M5 | s2 424242 CPU2 | 参考 | online dudect | 10000/批 | 10-16 | 0/完成/是 | max t 日志 / — |
| 5 | M5 | s3 31337 CPU6 | 参考 | measure 主类对 | 10000 | 10-16 | 0/完成/是 | CSV+meta / — |
| 6 | M5 | s3 31337 CPU6 | 参考 | online dudect | 10000/批 | 10-16 | 0/完成/是 | max t 日志 / — |
| 7 | M3 | s1 31337 CPU2 | on | measure 主类对 | 10000 | 10-16 | 0/完成/是 | CSV+meta / — |
| 8 | M3 | s1 31337 CPU2 | off | measure 主类对 | 10000 | 10-16 | 0/完成/是 | CSV+meta / — |
| 9 | M3 | s2 424242 CPU2 | on | measure 主类对 | 10000 | 10-16 | 0/完成/是 | CSV+meta / — |
| 10 | M3 | s2 424242 CPU2 | off | measure 主类对 | 10000 | 10-16 | 0/完成/是 | CSV+meta / — |
| 11 | M3 | s3 31337 CPU6 | on | measure 主类对 | 10000 | 10-16 | 0/完成/是 | CSV+meta / — |
| 12 | M3 | s3 31337 CPU6 | off | measure 主类对 | 10000 | 10-16 | 0/完成/是 | CSV+meta / — |
| 13 | M3 | s1 31337 CPU2 | on | online dudect | 10000/批 | 10-16 | 0/完成/是 | max t 日志 / — |
| 14 | M3 | s1 31337 CPU2 | off | online dudect | 10000/批 | 10-16 | 0/完成/是 | max t 日志 / — |
| 15 | M3 | s2 424242 CPU2 | on | online dudect | 10000/批 | 10-16 | 0/完成/是 | max t 日志 / — |
| 16 | M3 | s2 424242 CPU2 | off | online dudect | 10000/批 | 10-16 | 0/完成/是 | max t 日志 / — |
| 17 | M3 | s3 31337 CPU6 | on | online dudect | 10000/批 | 10-16 | 0/完成/是 | max t 日志 / — |
| 18 | M3 | s3 31337 CPU6 | off | online dudect | 10000/批 | 10-16 | 0/完成/是 | max t 日志 / — |

表 B1　平台四模块运行总表（一）。运行编号与正文 4.1、4.3 至 4.7 节一一对应。失败尝试单列于表 B3，不计入正式结果。

| 序 | 模块 | 运行 | 实现版本 | 类别 | 每类样本 | 日期 | 控制器 | 产物 / 异常 |
|---|---|---|---|---|---|---|---|---|
| 19 | M4 | s1 31337 CPU2 | 脆弱实现 | measure 主类对 | 10000 | 10-16 | 0/完成/是 | CSV+meta / — |
| 20 | M4 | s1 31337 CPU2 | 脆弱实现 | 截断对照 | 10000 | 10-16 | 0/完成/是 | CSV+meta / — |
| 21 | M4 | s1 31337 CPU2 | 脆弱实现 | online dudect | 10000/批 | 10-16 | 0/完成/是 | max t 日志 / — |
| 22 | M4 | s1 31337 CPU2 | 常数时间实现 | measure 主类对 | 10000 | 10-16 | 0/完成/是 | CSV+meta / — |
| 23 | M4 | s1 31337 CPU2 | 常数时间实现 | 截断对照 | 10000 | 10-16 | 0/完成/是 | CSV+meta / — |
| 24 | M4 | s1 31337 CPU2 | 常数时间实现 | online dudect | 10000/批 | 10-16 | 0/完成/是 | max t 日志 / — |
| 25 | M4 | s2 424242 CPU2 | 脆弱实现 | measure 主类对 | 10000 | 10-16 | 0/完成/是 | CSV+meta / — |
| 26 | M4 | s2 424242 CPU2 | 脆弱实现 | 截断对照 | 10000 | 10-16 | 0/完成/是 | CSV+meta / — |
| 27 | M4 | s2 424242 CPU2 | 脆弱实现 | online dudect | 10000/批 | 10-16 | 0/完成/是 | max t 日志 / — |
| 28 | M4 | s2 424242 CPU2 | 常数时间实现 | measure 主类对 | 10000 | 10-16 | 0/完成/是 | CSV+meta / — |
| 29 | M4 | s2 424242 CPU2 | 常数时间实现 | 截断对照 | 10000 | 10-16 | 0/完成/是 | CSV+meta / — |
| 30 | M4 | s2 424242 CPU2 | 常数时间实现 | online dudect | 10000/批 | 10-16 | 0/完成/是 | max t 日志 / — |
| 31 | M4 | s3 31337 CPU6 | 脆弱实现 | measure 主类对 | 10000 | 10-16 | 0/完成/是 | CSV+meta / — |
| 32 | M4 | s3 31337 CPU6 | 脆弱实现 | 截断对照 | 10000 | 10-16 | 0/完成/是 | CSV+meta / — |
| 33 | M4 | s3 31337 CPU6 | 脆弱实现 | online dudect | 10000/批 | 10-16 | 0/完成/是 | max t 日志 / — |
| 34 | M4 | s3 31337 CPU6 | 常数时间实现 | measure 主类对 | 10000 | 10-16 | 0/完成/是 | CSV+meta / — |
| 35 | M4 | s3 31337 CPU6 | 常数时间实现 | 截断对照 | 10000 | 10-16 | 0/完成/是 | CSV+meta / — |
| 36 | M4 | s3 31337 CPU6 | 常数时间实现 | online dudect | 10000/批 | 10-16 | 0/完成/是 | max t 日志 / — |
| 37 | M2 | s1 31337 CPU2 | 未修复版 | measure 主类对 | 10000 | 10-16 | 0/完成/是 | CSV+meta / — |
| 38 | M2 | s1 31337 CPU2 | 未修复版 | measure 对照类对 | 10000 | 10-16 | 0/完成/是 | CSV+meta / — |
| 39 | M2 | s1 31337 CPU2 | 已修复版 | measure 主类对 | 10000 | 10-16 | 0/完成/是 | CSV+meta / — |
| 40 | M2 | s1 31337 CPU2 | 已修复版 | measure 对照类对 | 10000 | 10-16 | 0/完成/是 | CSV+meta / — |
| 41 | M2 | s1 31337 CPU2 | 未修复版 | online dudect | 10000/批 | 10-16 | 0/完成/是 | max t 日志 / — |
| 42 | M2 | s1 31337 CPU2 | 已修复版 | online dudect | 10000/批 | 10-16 | 0/完成/是 | max t 日志 / — |
| 43 | M2 | s2 424242 CPU2 | 未修复版 | measure 主类对 | 10000 | 10-16 | 0/完成/是 | CSV+meta / — |
| 44 | M2 | s2 424242 CPU2 | 未修复版 | measure 对照类对 | 10000 | 10-16 | 0/完成/是 | CSV+meta / — |
| 45 | M2 | s2 424242 CPU2 | 已修复版 | measure 主类对 | 10000 | 10-16 | 0/完成/是 | CSV+meta / — |
| 46 | M2 | s2 424242 CPU2 | 已修复版 | measure 对照类对 | 10000 | 10-16 | 0/完成/是 | CSV+meta / — |
| 47 | M2 | s2 424242 CPU2 | 未修复版 | online dudect | 10000/批 | 10-16 | 0/完成/是 | max t 日志 / — |
| 48 | M2 | s2 424242 CPU2 | 已修复版 | online dudect | 10000/批 | 10-16 | 0/完成/是 | max t 日志 / — |
| 49 | M2 | s3 31337 CPU6 | 未修复版 | measure 主类对 | 10000 | 10-16 | 0/完成/是 | CSV+meta / — |
| 50 | M2 | s3 31337 CPU6 | 未修复版 | measure 对照类对 | 10000 | 10-16 | 0/完成/是 | CSV+meta / — |
| 51 | M2 | s3 31337 CPU6 | 已修复版 | measure 主类对 | 10000 | 10-16 | 0/完成/是 | CSV+meta / — |
| 52 | M2 | s3 31337 CPU6 | 已修复版 | measure 对照类对 | 10000 | 10-16 | 0/完成/是 | CSV+meta / — |
| 53 | M2 | s3 31337 CPU6 | 未修复版 | online dudect | 10000/批 | 10-16 | 0/完成/是 | max t 日志 / — |
| 54 | M2 | s3 31337 CPU6 | 已修复版 | online dudect | 10000/批 | 10-16 | 0/完成/是 | max t 日志 / — |

表 B1　平台四模块运行总表（二）。运行编号与正文 4.1、4.3 至 4.7 节一一对应。失败尝试单列于表 B3，不计入正式结果。

| 序 | 运行 | 实现版本 | 种子 / 核 | 每类样本 | 控制器 | 产物 / 异常 |
|---|---|---|---|---|---|---|
| 1 | m2-micro-s1-unfixed | 未修复版 | 31337 / CPU2 | 10000 | 0/完成/是 | CSV+meta / — |
| 2 | m2-micro-s1-fixed | 已修复版 | 31337 / CPU2 | 10000 | 0/完成/是 | CSV+meta / — |
| 3 | m2-micro-s2-fixed | 已修复版 | 424242 / CPU2 | 10000 | 0/完成/是 | CSV+meta / — |
| 4 | m2-micro-s2-unfixed | 未修复版 | 424242 / CPU2 | 10000 | 0/完成/是 | CSV+meta / — |
| 5 | m2-micro-s3-unfixed | 未修复版 | 31337 / CPU6 | 10000 | 0/完成/是 | CSV+meta / — |
| 6 | m2-micro-s3-fixed | 已修复版 | 31337 / CPU6 | 10000 | 0/完成/是 | CSV+meta / — |

表 B2　M2 除法热点微基准（6 次）。窗口只包一次消息编码调用，与完整解封装不可比。6 次运行均通过控制器四判据。

| 批次 | 运行构成 | 次数 | 日期 | 样本 / 类 | 控制器 | 产物 / 失败尝试 |
|---|---|---|---|---|---|---|
| 异常输入对照（格式破损） | 3 种实现 × 3 种子 ×（measure + dudect） | 18 | 10-16 | 10000 | 18/18 | CSV 与 max t 日志。无失败 |
| 性能块 | 9 个实现版本 × 6 个交错块 | 54 | 10-16 | decaps 150000。keygen、encaps 30000。M4 120000 | 54/54 | CSV+meta。18 次无效运行（块号参数错误，退出码 2，无计时值，不计入） |
| 优化级别 | 12 格 × 3 种子 | 36 | 10-16 | 10000 | 36/36 | CSV+meta。1 次被拒（频率探针跨度 54629 kHz，超过 50000 kHz，无计时值） |
| 锁频对照 | 3 种实现 × 3 种子（锁频组） | 9 | 10-16 | 10000 | 9/9 | CSV+meta。不锁频组 9 次另列，不计入 117 |
| 同机网络 | 3 种实现 × 2 次 × 5 档 | 30 | 10-20 | 本地直调 10000。网络各档 2000 | 30/30 | 客户端 RTT 原始值 + 服务端摘要。5 次频率探针拒绝、2 次客户端缺失，均无计时值 |
| aarch64 格 | 3 种实现 × 4 个优化级别 | 12 | 10-21 | 不采时间 | 无控制器 | Memcheck 日志与反汇编。无失败 |
| M1 封存批次 | 两个实现版本 × 3 次运行 | 6 | 10-15/16 | 10000 | 6/6 | replication.csv。同一机器与锁频规程，非与 M2 至 M5 同批采集 |

表 B3　收尾运行、网络、架构与封存批次。117 = 18 + 54 + 36 + 9，均为锁频控制器运行。不锁频 9 次与失败尝试不并入。

同机网络场景的逐格结果见表 B4。五档为本地直调、回环、low、mid、high。网络各档每类 2000 个样本，三次运行中的每一次都单独记录。检测预算按 $n = 2(1.96\,\mathrm{SD}/|\Delta|)^2$ 计算，参照效应取本地直调档两次运行的合并值 14.6 ns，只作量级参照。

| 实现版本 | 次 | 本地直调 | 回环 | low | mid | high |
|---|---|---|---|---|---|---|
| M1 未修复版 | r1 | +37.1 / 可分辨 | +63.1 / 未检出 | −801.3 / 未检出 | +6515.5 / 未检出 | +39569.7 / 未检出 |
| M1 未修复版 | r2 | −7.9 / 未检出 | −12.4 / 未检出 | +263.0 / 未检出 | −1863.5 / 未检出 | −51509.7 / 未检出 |
| M1 已修复版 | r1 | +11.9 / 未检出 | −52.3 / 未检出 | +736.4 / 未检出 | −3151.9 / 未检出 | +72863.2 / 未检出 |
| M1 已修复版 | r2 | +7.0 / 未检出 | +5.9 / 未检出 | −942.0 / 未检出 | −6885.7 / 未检出 | +39109.8 / 未检出 |
| M5 参考 | r1 | −11.3 / 未检出 | −52.8 / 未检出 | −665.1 / 未检出 | +2861.4 / 未检出 | −30649.4 / 未检出 |
| M5 参考 | r2 | +2.7 / 未检出 | −57.7 / 未检出 | −455.4 / 未检出 | +654.6 / 未检出 | −61629.6 / 未检出 |
| 分辨 14.6 ns 所需样本 / 类 | | 13098 | 38417 | 64636555 | 1003153317 | 97586175893 |
| 本轮实际样本 / 类 | | 10000 | 2000 | 2000 | 2000 | 2000 |

表 B4　同机网络五档的均值差（ns）与判定。网络各档在本预算下未检出。这是同机模拟、非真实远程，所需样本比实际高 1.3 至 7.7 个数量级。

| 实现版本 | -O0 | -O2 | -O3 | -Os | 反汇编形态 | 判定 |
|---|---|---|---|---|---|---|
| M1 未修复版 | 0 / 0 | 0 / 0 | 0 / 0 | 0 / 0 | `sbfx`、`and`、`csel` | 四格均无秘密相关分支 |
| M1 已修复版 | 0 / 0 | 0 / 0 | 0 / 0 | 0 / 0 | 掩码选择 | 四格均无命中 |
| M5 参考 | 0 / 0 | 0 / 0 | 0 / 0 | 0 / 0 | 掩码选择 + 值屏障 | 四格均无命中 |
| x86_64 M1 未修复版（对照） | 0 / 0 | 512 / 8 | 512 / 15 | 512 / 1 | `bt`/`jae` 或逐位测试分支 | x86_64 三格有命中 |

表 B5　aarch64 12 格与 x86_64 同格对照（错误数 / 上下文数）。

表 B1 至表 B5 的共同口径是：一条运行记录只对应一个模块、一个实现版本、一个种子与一个控制器终态。同一模块的 measure 与 dudect 分开记，不合并成一次。54 次平台家族、6 次微基准与 117 次收尾运行合计 177 次控制器运行。M1 的 6 次主类对属于封存批次，不并入 177。失败尝试单列，不删、不改名、不重算。它们没有计时值，因此既不构成阳性也不构成阴性。表 B4 的网络与表 B5 的 aarch64 不进任何 Holm 家族，时间证据不与主平台合并。

样本预算按主类对设计：每类 10000 次、三次独立运行，换种子与换物理核。微基准只包一个热点函数，与完整解封装不可比。网络档每类 2000 次，检测预算按同一式子反推所需样本。预算不同的表不能横向比较 Δ 的大小，只能各自回答“在本预算下是否可分辨”。这也是表 B3 把 54、6、117 分开列、不合成一个总数的原因。

日期与批次也要一起读。平台四模块、微基准、性能、优化与锁频对照集中在 10 月 16 日。异常输入对照同日。同机网络在 10 月 20 日。aarch64 在 10 月 21 日。M1 的主类对是 10 月 15 日至 16 日的封存批次。同一台机器、同一锁频规程，不等于同一批采集。跨日数据只做同口径对照，不把日期差当成处理效应。表 B1 的 54 次运行全部有控制器终态与原始 CSV，表 B3 的 117 次运行另有逐份 terminal 记录。两类记录都保留失败尝试，避免“只留成功”造成的工作量虚高。

失败尝试单列还有一层作用：它们能区分“设备没跑起来”与“跑了但没有差异”。控制器拒绝、参数错误、客户端缺失都属于前者，没有计时值。它们既不支持阳性，也不支持阴性。把它们写成“失败”而不是删掉，读者才能判断 54、6、117 这些数字是不是从全部尝试里挑出来的。本附录的所有计数都以“成功且四判据通过”为分子，以“成功 + 失败尝试”为分母。分母写在表里，分子写在表注里。

# 附录 C　统计公式与复算说明

本附录给出统计层用到的七个公式、本次参数与手算例。公式与统计电池、固定效应合并工具的实现一致。dudect 与 RTLF 的工具行为按上游实现 [7, 8]。手算例只用合成数核对公式，不替代正式数据。正式结果见附录 D 与正文 4.3 至 4.7 节。

**（1）Welch 检验与自由度。**

$$t = \frac{\Delta}{\sqrt{s_0^2/n_0 + s_1^2/n_1}}, \qquad \mathrm{df} = \frac{\left(s_0^2/n_0 + s_1^2/n_1\right)^2}{\dfrac{(s_0^2/n_0)^2}{n_0 - 1} + \dfrac{(s_1^2/n_1)^2}{n_1 - 1}}. \tag{C1}$$

其中 $\Delta = \bar{x}_0 - \bar{x}_1$，$s_0^2$、$s_1^2$ 为样本方差，$n_0 = n_1 = 10000$。本次不称其为有限样本精确 $t$ 区间。$p$ 用 $\mathrm{erfc}(|t|/\sqrt{2})$ 的正态近似。

**（2）95% 置信区间。**

$$[\Delta - z_{0.975}\,\mathrm{SE},\ \Delta + z_{0.975}\,\mathrm{SE}], \qquad \mathrm{SE} = \sqrt{s_0^2/n_0 + s_1^2/n_1}, \quad z_{0.975} = 1.9599639845. \tag{C2}$$

区间含 0 记“未检出”。区间不含 0 记“可区分”。未检出必须同时给出 MDE。

**（3）Holm 逐步校正。**

$$p_{(1)} \le p_{(2)} \le \cdots \le p_{(m)}, \qquad \text{第} i \text{ 项阈值} = \frac{\alpha}{m - i + 1}. \tag{C3}$$

从最小 $p$ 开始逐项比较。第一个不满足即停止，其后各项均不拒绝。全族 $\alpha = 0.05$。M1、M2、M3、M4 的 $m = 6$，M5 的 $m = 3$。

**（4）逆方差固定效应合并。**

$$w_i = \frac{1}{\mathrm{SE}_i^2}, \qquad \Delta_{\text{pool}} = \frac{\sum_i w_i \Delta_i}{\sum_i w_i}, \qquad \mathrm{SE}_{\text{pool}} = \frac{1}{\sqrt{\sum_i w_i}}. \tag{C4}$$

每次运行的 $\mathrm{SE}_i$ 由该次 95% 区间反推：$\mathrm{SE}_i = (\text{上界} - \text{下界})/(2z_{0.975})$。

**（5）异质性 $I^2$。**

$$Q = \sum_i w_i \left(\Delta_i - \Delta_{\text{pool}}\right)^2, \qquad I^2 = \max\left(0, \frac{Q - (k-1)}{Q}\right). \tag{C5}$$

$k$ 为成员数。$I^2$ 高时，合并区间只作描述，判定仍以预注册的 Holm 结果为准。

**（6）最小可检测效应。**

$$\mathrm{MDE} = \left(z_{1-\alpha/2} + z_{1-\beta}\right)\sqrt{\frac{s_0^2}{n_0} + \frac{s_1^2}{n_1}}, \tag{C6}$$

取 $\alpha = 0.05$、$1-\beta = 0.80$，$z_{0.975} = 1.959963985$、$z_{0.80} = 0.841621234$。Holm 口径把 $\alpha$ 换成 $\alpha_{\mathrm{eff}} = \alpha/m$。M1、M2、M3、M4 的 $z_{1-\alpha_{\mathrm{eff}}/2} = 2.638257273$，M5 为 2.393979800。MDE 是事后功效描述，不改变已提交的判定。

**（7）RTLF 秩分位数 bootstrap。** RTLF 对两类样本的分位数差做 bootstrap 重采样，$\alpha = 0.01$，固定种子 2026101605，重采样 10000 次，输出二值结论：退出码 10 为未检出、11 为检出。它不产生 $p$ 值，不进入 Holm 家族，也不与均值检验互相替代。

| 公式 | 符号 | 本次取值 | 用途 |
|---|---|---|---|
| C1 | $t$、df、$s^2$、$n$ | $n_0 = n_1 = 10000$。$\alpha = 0.05$ | 单次运行的均值差检验与自由度 |
| C2 | SE、$z_{0.975}$ | $z_{0.975} = 1.9599639845$ | 每次运行的 95% 区间 |
| C3 | $p_{(i)}$、$m$、$\alpha$ | $m = 6$（M5 为 3）。$\alpha = 0.05$ | 家族内多重比较 |
| C4 | $w_i$、$\Delta_{\mathrm{pool}}$ | 三次运行按区间反推权重 | 跨运行合并 |
| C5 | $Q$、$k$ | $k = 6$（M5 为 3） | 异质性描述 |
| C6 | MDE、$z_{1-\beta}$ | $z_{0.80} = 0.841621234$。Holm 口径 $\alpha_{\mathrm{eff}} = \alpha/m$ | 量化未检出 |
| RTLF | 分位数、bootstrap | $\alpha = 0.01$。种子 2026101605。10000 次 | 族外分布交叉 |

表 C1　统计公式与参数。公式与统计电池、固定效应合并工具的实现一致。手算例只用于核对，不替代正式数据。

**手算例一（Welch 与区间）。** 取两组各 3 个数：$x_0 = \{10, 12, 11\}$，$x_1 = \{7, 9, 8\}$。均值 $\bar{x}_0 = 11$、$\bar{x}_1 = 8$，$\Delta = 3$。样本方差 $s_0^2 = s_1^2 = 1$。$\mathrm{SE} = \sqrt{1/3 + 1/3} = 0.8165$，$t = 3.674$，$\mathrm{df} = 4$，双侧 $p \approx 0.021$。95% 区间为 $3 \pm 1.96 \times 0.8165 = [1.40, 4.60]$，不含 0。这个例子只说明式 C1、C2 的算法，不构成任何实验结论。

**手算例二（Holm）。** 取 $m = 4$ 与 $p = \{0.001, 0.020, 0.030, 0.040\}$。阈值依次为 0.0125、0.0167、0.025、0.05。第一项 $0.001 < 0.0125$，拒绝。第二项 $0.020 > 0.0167$，停止。其后两项均不拒绝。Holm 的停止规则使“前面不拒绝”自动覆盖后面，不需要逐项再比。

**手算例三（合并与 $I^2$）。** 三次运行的 $\Delta = \{10, 14, 8\}$，$\mathrm{SE} = \{2, 2, 4\}$。

权重 $w = \{0.25, 0.25, 0.0625\}$，$\sum w = 0.5625$。

$\Delta_{\mathrm{pool}} = (2.5 + 3.5 + 0.5)/0.5625 = 11.556$。$\mathrm{SE}_{\mathrm{pool}} = 1/\sqrt{0.5625} = 1.333$。95% 区间 $11.556 \pm 2.613 = [8.94, 14.17]$。$Q = 0.25(1.556)^2 + 0.25(2.444)^2 + 0.0625(3.556)^2 = 2.89$，$k - 1 = 2$，$I^2 = (2.89 - 2)/2.89 = 30.8\%$。

**手算例四（MDE）。** 取 $s_0 = 700$ ns、$s_1 = 600$ ns、$n_0 = n_1 = 10000$、$\alpha = 0.05$、功效 0.80。$\mathrm{MDE} = 2.801585 \times \sqrt{700^2/10000 + 600^2/10000} = 2.801585 \times 9.2195 = 25.83$ ns。若改用 $m = 6$ 的 Holm 口径，$z_{1-\alpha_{\mathrm{eff}}/2} = 2.638257$，MDE 变为 $(2.638257 + 0.841621) \times 9.2195 = 32.08$ ns。

**口径说明。** M1 族在采集后发生过一次成员调整：dudect 的多批最大 $t$ 不是有效 $p$ 值，因此从参数族移出，原始记录另存。两种口径下的家族级判定一致，正文与附录 D 的判定均未改变。这里不展开成员数，避免把一次口径调整写成新的实验事实。

区间与 $p$ 值采用大样本正态近似；MDE 是基于实测标准差的事后功效描述，不改变预注册判定。异质性较高时，合并区间只作描述；M4 的家族合并混合了脆弱实现与常数时间实现，需结合分版本结果解释。RTLF 只给二值结论，不转成 $p$ 值，也不与均值检验合并。

Holm 与 MDE 是两件事：Holm 控制家族错误率，MDE 描述可检测性。一个阴性结论要同时给两者，读者才能判断“没拒绝”是因为效应小，还是因为样本不够。手算例只用来核对公式，不替代正式数据。正式数据在附录 D，逐次值与家族判定并列。

## 附录 D　模块级完整结果

本附录给出 M1 至 M5 的逐次运行结果。Δ 为 class0 减 class1 的均值差。95% 区间为正态近似。$p$ 为双侧原始值。Holm 列为该成员在预注册家族中的判定。dudect 与 RTLF 为族外交叉，不进入 Holm。M1 的逐次 $p$ 由该次 Δ 与区间反推 SE 后按式 C1、C2 复算，属 B 类。其余 $p$ 取自成员表原件。M4 的 $p = 0$ 是双精度下溢，不写“精确为零”。

| 实现版本 | 运行 | Δ / ns | 95% CI / ns | $p$ | Holm | dudect max $\lvert t\rvert$ | RTLF | MDE / ns |
|---|---|---|---|---|---|---|---|---|
| 未修复版 | s1 31337 CPU2 | +39.337 | [21.658, 57.017] | $1.30 \times 10^{-5}$ | 族内拒绝 | 26.41 | 检出 | 25.27 / 31.39 |
| 未修复版 | s2 424242 CPU2 | +31.117 | [6.549, 55.684] | 0.0131 | 族内拒绝 | 40.64 | 检出 | 25.27 / 31.39 |
| 未修复版 | s3 31337 CPU6 | +22.337 | [0.282, 44.392] | 0.0471 | 族内拒绝 | 14.92 | 检出 | 25.27 / 31.39 |
| 已修复版 | s1 31337 CPU2 | −13.209 | [−33.238, 6.820] | 0.1962 | 不拒绝 | 1.15 | 未检出 | 28.63 / 35.56 |
| 已修复版 | s2 424242 CPU2 | +8.354 | [−52.988, 69.696] | 0.7895 | 不拒绝 | 1.12 | 未检出 | 28.63 / 35.56 |
| 已修复版 | s3 31337 CPU6 | −4.885 | [−24.811, 15.042] | 0.6309 | 不拒绝 | 1.31 | 未检出 | 28.63 / 35.56 |
| 合并（未修复版） | | +32.311 | [20.282, 44.339] | $1.30 \times 10^{-5}$ | 拒绝 | — | — | 25.27 / 31.39 |
| 合并（已修复版） | | −8.150 | [−21.916, 5.616] | 0.196 | 不拒绝 | — | — | 28.63 / 35.56 |

表 D1　M1 逐次结果。逐次值取自封存原始数据。“未检出”附 MDE。M1 的家族主检验 $p = 1.30 \times 10^{-5}$、阈值 0.00833。已修复版主检验 $p = 0.196$、阈值 0.05（末位成员）后不拒绝。

| 实现版本 | 运行 | Δ / ns | 95% CI / ns | $p$ | Holm | dudect max $\lvert t\rvert$ | RTLF | MDE / ns |
|---|---|---|---|---|---|---|---|---|
| 未修复版 | r1 31337 CPU2 | +10.530 | [−6.396, 27.455] | 0.223 | 不拒绝 | 4.68 | 检出 | 19.9 至 54.0 / 24.7 至 67.1 |
| 未修复版 | r2 424242 CPU2 | +19.126 | [0.028, 38.224] | 0.0497 | 不拒绝 | 3.05 | 检出 | 19.9 至 54.0 / 24.7 至 67.1 |
| 未修复版 | r3 31337 CPU6 | +9.983 | [−3.915, 23.881] | 0.159 | 不拒绝 | 3.66 | 检出 | 19.9 至 54.0 / 24.7 至 67.1 |
| 已修复版 | r1 31337 CPU2 | +21.657 | [−16.123, 59.436] | 0.261 | 不拒绝 | 36.27 | 检出 | 19.9 至 54.0 / 24.7 至 67.1 |
| 已修复版 | r2 424242 CPU2 | −17.080 | [−45.991, 11.830] | 0.247 | 不拒绝 | 30.94 | 检出 | 19.9 至 54.0 / 24.7 至 67.1 |
| 已修复版 | r3 31337 CPU6 | −15.512 | [−31.925, 0.900] | 0.0640 | 不拒绝 | 61.91 | 检出 | 19.9 至 54.0 / 24.7 至 67.1 |
| 合并 | | +4.58 | [−3.08, 12.25] | — | 拒绝数 0 | — | — | 24.7 至 67.1 |

表 D2　M2 逐次结果。逐次值由封存原始数据重列。“未检出”附 MDE。dudect 阳性落在已修复版、RTLF 两个版本均检出，均不可归因于除法变体，只作族外登记。

| 实现版本 | 运行 | Δ / ns | 95% CI / ns | $p$ | Holm | dudect max $\lvert t\rvert$ | RTLF | MDE / ns |
|---|---|---|---|---|---|---|---|---|
| on | r1 31337 CPU2 | −10.723 | [−29.703, 8.258] | 0.268 | 不拒绝 | 1.93 | 未检出 | 14.2 至 27.1 / 17.7 至 33.7 |
| on | r2 424242 CPU2 | −5.763 | [−17.016, 5.489] | 0.315 | 不拒绝 | 3.41 | 未检出 | 14.2 至 27.1 / 17.7 至 33.7 |
| on | r3 31337 CPU6 | +7.126 | [−3.497, 17.749] | 0.189 | 不拒绝 | 2.81 | 未检出 | 14.2 至 27.1 / 17.7 至 33.7 |
| off | r1 31337 CPU2 | −5.464 | [−20.612, 9.683] | 0.480 | 不拒绝 | 2.24 | 未检出 | 14.2 至 27.1 / 17.7 至 33.7 |
| off | r2 424242 CPU2 | +4.800 | [−5.816, 15.417] | 0.375 | 不拒绝 | 3.37 | 未检出 | 14.2 至 27.1 / 17.7 至 33.7 |
| off | r3 31337 CPU6 | −3.775 | [−13.732, 6.183] | 0.457 | 不拒绝 | 3.03 | 未检出 | 14.2 至 27.1 / 17.7 至 33.7 |
| 合并 | | −0.73 | [−5.56, 4.10] | — | 拒绝数 0 | — | — | 17.7 至 33.7 |

表 D3　M3 逐次结果。逐次值来自封存记录。“未检出”附 MDE。M3 为人工构造校准，不是上游缺陷。

| 实现版本 | 运行 | Δ / ns | 95% CI / ns | $p$ | Holm | dudect max $\lvert t\rvert$ | RTLF | MDE / ns |
|---|---|---|---|---|---|---|---|---|
| 脆弱实现 | r1 31337 CPU2 | −4.102 | [−4.232, −3.973] | 下溢 0 | 拒绝 | 356.6 | 检出 | 0.19 至 1.19 / 1.47 |
| 脆弱实现 | r2 424242 CPU2 | −4.351 | [−5.181, −3.521] | $9.23 \times 10^{-25}$ | 拒绝 | 396.9 | 检出 | 0.19 至 1.19 / 1.47 |
| 脆弱实现 | r3 31337 CPU6 | −3.515 | [−4.226, −2.805] | $3.14 \times 10^{-22}$ | 拒绝 | 395.6 | 检出 | 0.19 至 1.19 / 1.47 |
| 常数时间实现 | r1 31337 CPU2 | +0.415 | [−0.523, 1.353] | 0.386 | 不拒绝 | 2.06 | 未检出 | 0.19 至 1.19 / 1.47 |
| 常数时间实现 | r2 424242 CPU2 | −0.227 | [−0.800, 0.346] | 0.438 | 不拒绝 | 1.90 | 未检出 | 0.19 至 1.19 / 1.47 |
| 常数时间实现 | r3 31337 CPU6 | +0.347 | [−0.460, 1.155] | 0.399 | 不拒绝 | 2.27 | 未检出 | 0.19 至 1.19 / 1.47 |
| 合并（脆弱实现） | | −4.089 | [−4.215, −3.963] | — | 拒绝 3 项 | — | — | 1.47（最坏） |
| 合并（常数时间实现） | | +0.055 | [−0.363, 0.473] | — | 不拒绝 | — | — | 1.47（最坏） |

表 D4　M4 分版本逐次结果。逐次值由原始成员表复列。“未检出”附 MDE。$p = 0$ 是双精度下溢，不写“精确为零”。

| 实现版本 | 运行 | Δ / ns | 95% CI / ns | $p$ | Holm | dudect max $\lvert t\rvert$ | RTLF | MDE / ns |
|---|---|---|---|---|---|---|---|---|
| 参考 | r1 31337 CPU2 | −5.596 | [−18.912, 7.720] | 0.410 | 不拒绝 | 2.10 | 未检出 | 11.7 至 24.5 / 13.5 至 28.3 |
| 参考 | r2 424242 CPU2 | −6.946 | [−24.103, 10.211] | 0.427 | 不拒绝 | 1.98 | 未检出 | 11.7 至 24.5 / 13.5 至 28.3 |
| 参考 | r3 31337 CPU6 | −4.819 | [−13.019, 3.382] | 0.249 | 不拒绝 | 2.82 | 未检出 | 11.7 至 24.5 / 13.5 至 28.3 |
| 合并 | | −5.30 | [−11.77, 1.16] | — | 拒绝数 0 | — | — | 13.5 至 28.3 |

表 D5　M5 逐次结果。逐次值取自封存记录；“未检出”附 MDE。

逐次表的读法与正文不同：正文给分层结论，附录给每次运行的原始量。Δ 的符号由 class0 减 class1 决定，同一模块内方向不一致时，不取绝对值平均。先看每次区间是否含 0，再看 Holm 家族是否拒绝。M2 的未修复版描述性合并为正、已修复版为负，这是实现版本间的漂移，不是泄漏降低。M4 的家族合并被两个版本混合拉宽，按实现版本分开后脆弱实现与常数时间实现的区间才可读。M1 的逐次 p 是 B 类复算值，版本合并 p 才是族内判定。两者并列，不互相替代。

五个模块呈现不同的证据组合：M1 定位与计时均阳性，官方修复后计时未检出；M2 定位阳性而端到端与微基准未检出；M3 为人工校准；M4 为定位和计时均阳性的正对照，常数时间实现未检出；M5 为阴性对照，七种工具均未报阳性。逐次值与模块结论分别见表 D1–D5。

MDE 也要逐模块读。M1 未修复版的检测预算约 25 至 31 ns，已修复版约 29 至 36 ns。M2 为 25 至 67 ns。M3 为 18 至 34 ns。M4 最坏 1.47 ns。M5 为 14 至 28 ns。观测效应小于本模块 MDE 时写“未检出”，不写“零效应”。M4 的 MDE 小，是因为比较窗口只有几十纳秒、方差也小。这不表示 M4 的结论可以外推到完整解封装。反过来，M2 的 MDE 大，除法热点即使有几十纳秒的效应也可能落在预算之外，所以定位阳性与计时阴性可以同时成立。

逐次 p 的来源分两类：M2 至 M5 直接取自成员表原件。M1 的逐次 p 由该次 Δ 与区间反推 SE 后按正态近似复算。复算值只用于附录对照，族内判定仍用版本合并 p。把两类值并列，

是为了让读者看到“逐次方向一致”与“家族拒绝”不是同一个层次。前者描述三次运行，后者描述预注册检验。

# 附录 E　定位工具输出摘要

本附录只摘录各工具的关键输出行，不贴完整日志。表 E1 覆盖统计与动态标记工具，表 E2 覆盖形式化、访存与反汇编工具。“未运行”按原因登记，不填阴性。工具版本写在各行括号内。同一工具在不同模块上的结论不可互相替代。

| 工具（版本） | 对象 | 关键输出 | 上下文 / 样本 | 位置 / 地址 | 结论 |
|---|---|---|---|---|---|
| dudect（上游头文件快照） | M1 | 未修复版 max $\|t\|$ 26.41 / 40.64 / 14.92。已修复版 1.15 / 1.12 / 1.31 | 3 次运行，21 批 | 完整解封装 | 检出（未修复版） |
| dudect | M2 | 未修复版 4.68 / 3.05 / 3.66。已修复版 36.27 / 30.94 / 61.91 | 3 次运行 | 完整解封装 | 未检出。阳性落在已修复版，不可归因 |
| dudect | M3 | on 1.93 / 3.41 / 2.81。off 2.24 / 3.37 / 3.03 | 3 次运行 | 完整解封装 | 未检出 |
| dudect | M4 | 脆弱实现 356.6 / 396.9 / 395.6。常数时间实现 2.06 / 1.90 / 2.27 | 3 次运行 | 32 B 比较调用 | 检出（脆弱实现） |
| dudect | M5 | 2.10 / 1.98 / 2.82 | 3 次运行 | 完整解封装 | 未检出 |
| RTLF（R 4.3.3，$\alpha = 0.01$） | M1 | 未修复版 2/2 exit 11。已修复版 2/2 exit 10 | 2 次运行 | 完整解封装 | 检出（未修复版） |
| RTLF | M2 | 主类对 6/6 exit 11（两个版本各 3）。对照 6/6 exit 10 | 6 个数据集 | 完整解封装 | 检出。两个版本均检出，不可归因 |
| RTLF | M3 | 主类对与对照共 12 个数据集全部 exit 10 | 12 个数据集 | 完整解封装 | 未检出 |
| RTLF | M4 | 脆弱实现 6/6 exit 11（含截断）。常数时间实现 6/6 exit 10 | 12 个数据集 | 32 B 比较调用 | 检出（脆弱实现） |
| RTLF | M5 | 主类对与对照共 6/6 exit 10 | 6 个数据集 | 完整解封装 | 未检出 |
| Welch + Holm（统计电池） | M1 | 主检验 $p = 1.30 \times 10^{-5}$，阈值 0.00833。已修复版 $p = 0.196$ | 每类 10000 | 完整解封装 | 检出（未修复版） |
| Welch + Holm | M2 | 合并 $\Delta = +4.58$ ns，CI $[-3.08, 12.25]$。拒绝 0 | 6 项 | 完整解封装 | 未检出 |
| Welch + Holm | M3 | 合并 $\Delta = -0.73$ ns，CI $[-5.56, 4.10]$。拒绝 0 | 6 项 | 完整解封装 | 未检出 |
| Welch + Holm | M4 | 脆弱实现 3 项拒绝。常数时间实现 0 项 | 6 项 | 32 B 比较调用 | 检出（脆弱实现） |
| Welch + Holm | M5 | 合并 $\Delta = -5.30$ ns，CI $[-11.77, 1.16]$。拒绝 0 | 3 项 | 完整解封装 | 未检出 |
| Memcheck（Valgrind 3.22.0） | M1 | 未修复版 1 error / 1 context。已修复版 0 / 0 | 完整解封装 | 消息编码函数第 179 行 | 检出（未修复版） |
| Memcheck | M2 | 未运行（完整解封装未跑。仅关 varlat 对照 0/0） | — | — | 未运行，不填阴性 |
| Memcheck | M3 | on 1280 / 2。off 0 / 0 | 10 次完整解封装 | 消息编码函数第 714 行 | 检出（人工校准） |
| Memcheck | M4 | 32 / 1。常数时间实现 0 / 0 | 完整调用 | 比较内核第 9 行 | 检出（人工校准） |
| Memcheck | M5 | 三模式各 10 次，0 / 0 | 30 次完整解封装 | 完整调用链 | 未检出 |
| varlat（Valgrind 3.25.1） | M1 | 未运行（机制不在命中类别） | — | — | 未运行 |
| varlat | M2 | 未修复版 1280 / 3。已修复版 0 / 0 | 3 个热点 | 消息解码、压缩、向量压缩 | 检出（未修复版） |
| varlat | M3 | 未运行 | — | — | 未运行 |
| varlat | M4 | 未运行 | — | — | 未运行 |
| varlat | M5 | varlat=yes 0 / 0 | 10 次完整解封装 | 完整调用链 | 未检出 |

表 E1　统计与动态标记工具输出摘要。“未运行”表示该单元没有执行记录。

| 工具（版本） | 对象 | 关键输出 | 上下文 / 样本 | 位置 / 地址 | 结论 |
|---|---|---|---|---|---|
| Binsec（0.11.3 + Z3 4.15.3） | M1 | 未运行（无对应抽取内核） | — | — | 未运行 |
| Binsec | M2 | 未运行（$div$ 类来源不在已开检查项） | — | — | 未运行 |
| Binsec | M3 | 未运行 | — | — | 未运行 |
| Binsec | M4 | 脆弱实现 `insecure`，反例 `0x8049047`。常数时间实现 `bounded secure` | 32/32 与 16/16 路径 | 比较内核第 9 行 | 检出（人工校准） |
| Binsec | M5 | 重加密比较 1088/1088、消息编码 1/1 路径完成，无秘密相关控制流 | 路径完整 | 重加密比较与消息编码 | 未检出（有界） |
| Microwalk（源码自建 + Pin 3.28） | M1 | 未运行（官方镜像不可用，Pin 缺失） | — | — | 未运行 |
| Microwalk | M2 | 未运行 | — | — | 未运行 |
| Microwalk | M3 | 互信息 1.922 bit（全场最高）。off 0.722 bit | 5 个公开用例 | 两处秘密下标 load | 检出（人工校准） |
| Microwalk | M4 | 未运行 | — | — | 未运行 |
| Microwalk | M5 | 未运行 | — | — | 未运行 |
| Mona（上游脚本） | M1 | 未修复版无分离。已修复版 2/6 出现箱体分离 | 6 份输入 | 完整解封装 | 未检出。方向不符 |
| Mona | M2 | 未修复版无分离。s3 已修复版 1/8 出现分离 | 6 批 | 完整解封装 | 未检出。方向不符 |
| Mona | M3 | 未运行（未接入） | — | — | 未运行 |
| Mona | M4 | 未运行（未接入） | — | — | 未运行 |
| Mona | M5 | 未运行（未接入） | — | — | 未运行 |
| 反汇编（发行版 objdump） | M1 | `bt+jae` → `bt+setb+call cmov_int16` | 两版产物 | 消息编码函数 | 检出（未修复版） |
| 反汇编 | M2 | 三热点各 1 条 `div` → 0。消息编码 `bt/jae` 两个版本中的指令逐字节相同 | 两版产物 | 消息解码、压缩、向量压缩 | 检出。残留分支 |
| 反汇编 | M3 | 两条 `movzwl` 秘密下标 load | on/off 两版 | 静态偏移 `0x5CED/0x5D15` | 检出（人工校准） |
| 反汇编 | M4 | 3 条条件跳转中 1 条数据相关 | 脆弱实现/常数时间实现 | 比较内核 | 检出（人工校准） |
| 反汇编 | M5 | 无秘密相关分支。消息编码唯一跳转是循环回边。sk 检查为公开数据 | 完整产物 | 完整调用链 | 未发现 |

表 E2　形式化、访存与反汇编工具输出摘要。“未运行”表示该单元没有执行记录。

45 格中 31 格有真实产物，14 格未运行。未运行的原因包括工具未接入该模块、缺少运行依赖或该模块没有对应的抽取内核。不同工具的判据可能给出不同读法：RTLF 在 M2 两个版本均报告分布差异，dudect 的阳性落在已修复版，预注册 Welch+Holm 对两个版本均未拒绝。综合时还应结合 M2 与 M3 的定位结果，分别报告机制命中和计时判定。

工具输出只证明给定输入下的行为。Memcheck 的 0 命中不覆盖算术泄漏与缓存泄漏。Binsec 的 bounded secure 只覆盖给定入口、长度与秘密模型。Microwalk 的互信息只覆盖实际执行的用例。dudect 与 RTLF 只回答分布是否可区分。把这些边界写进表，是为了让“未运行”与“未检出”在版面上就分开，而不是靠读者猜。

# 附录 F　90 格排查全表

本附录给出 3 实现 × 5 步骤 × 6 类 = 90 格的逐格判定。来源列的数字含义：1 敏感分支/提前返回、2 可变时延指令/比较、3 数据相关循环、4 数据相关访存、5 失败路径、6 异常处理。

判定只取“存在 / 不存在 / 不适用 / 未检查”。全表合计：存在 5、不存在 75、不适用 10、未检查 0。人工夹具 M3、M4 不计入本表。“存在”格中的 M5 一格携带公开信息，按有界例外登记。

| 实现 | 步骤 | 来源 | 判定 | 函数 | 指令或行号 | 证据 |
|---|---|---|---|---|---|---|
| M1 | 解密 | 1 敏感分支/提前返回 | 不存在 | indcpa_dec / poly_tomsg | 常数回边 | A-17 |
| M1 | 解密 | 2 可变时延指令/比较 | 不存在 | poly_tomsg | 乘加、移位，无 `div` | A-17 |
| M1 | 解密 | 3 数据相关循环 | 不存在 | poly_tomsg | 32 × 8 常数上界 | A-17 |
| M1 | 解密 | 4 数据相关访存 | 不存在 | poly_tomsg | 固定顺序写 | A-17 |
| M1 | 解密 | 5 失败路径 | 不存在 | crypto_kem_dec | 本步不判定失败 | A-17 |
| M1 | 解密 | 6 异常处理 | 不适用 | — | 无异常机制 | A-17 |
| M1 | 哈希 | 1 敏感分支/提前返回 | 不存在 | hash_g / hash_h / rkprf | 定长调用 | A-17 |
| M1 | 哈希 | 2 可变时延指令/比较 | 不存在 | FIPS202 | 无比较 | A-17 |
| M1 | 哈希 | 3 数据相关循环 | 不存在 | FIPS202 | 24 轮固定 | A-17 |
| M1 | 哈希 | 4 数据相关访存 | 不存在 | FIPS202 | 常量下标 | A-17 |
| M1 | 哈希 | 5 失败路径 | 不存在 | FIPS202 | 不返回状态 | A-17 |
| M1 | 哈希 | 6 异常处理 | 不存在 | FIPS202 | 无 abort/断言 | A-17 |
| M1 | 重加密 | 1 敏感分支/提前返回 | 存在 | poly_frommsg | `0x16e bt` → `0x172 jae` | A-17、A-22 |
| M1 | 重加密 | 2 可变时延指令/比较 | 不存在 | poly_frommsg / 压缩 | 无比较动作 | A-17 |
| M1 | 重加密 | 3 数据相关循环 | 不存在 | poly_frommsg | 32 × 8 常数 | A-17 |
| M1 | 重加密 | 4 数据相关访存 | 不存在 | poly_frommsg | 固定顺序写 | A-17 |
| M1 | 重加密 | 5 失败路径 | 不存在 | indcpa_enc | 不返回错误 | A-17 |
| M1 | 重加密 | 6 异常处理 | 不存在 | indcpa_enc | 无异常机制 | A-17 |
| M1 | 比较 | 1 敏感分支/提前返回 | 不存在 | verify | 遍历 1088 B | A-17 |
| M1 | 比较 | 2 可变时延指令/比较 | 不存在 | verify | 累积异或，不短路 | A-17 |
| M1 | 比较 | 3 数据相关循环 | 不存在 | verify | 长度常数 1088 | A-17 |
| M1 | 比较 | 4 数据相关访存 | 不存在 | verify | 顺序下标 | A-17 |
| M1 | 比较 | 5 失败路径 | 不存在 | verify | 结果只作掩码 | A-17 |
| M1 | 比较 | 6 异常处理 | 不适用 | — | 不可失败 | A-17 |
| M1 | 隐式拒绝 | 1 敏感分支/提前返回 | 不存在 | dec / cmov | 无条件走完 | A-17 |
| M1 | 隐式拒绝 | 2 可变时延指令/比较 | 不存在 | cmov | `0x2b26 sete` → `0x2b37 call` | A-17 |
| M1 | 隐式拒绝 | 3 数据相关循环 | 不存在 | rkprf / cmov | 定长 32 B | A-17 |
| M1 | 隐式拒绝 | 4 数据相关访存 | 不存在 | cmov | 固定偏移、顺序下标 | A-17 |
| M1 | 隐式拒绝 | 5 失败路径 | 不存在 | dec | 单点返回，零跳转 | A-17 |
| M1 | 隐式拒绝 | 6 异常处理 | 不适用 | — | 无异常机制 | A-17 |
| M1 小计：存在 1、不存在 26、不适用 3、未检查 0（30 格） | | | | | | |

表 F1　M1 的 30 格排查。

| 实现 | 步骤 | 来源 | 判定 | 函数 | 指令或行号 | 证据 |
|---|---|---|---|---|---|---|
| M2 | 解密 | 1 敏感分支/提前返回 | 不存在 | dec / indcpa_dec | 常数回边 | A-18 |
| M2 | 解密 | 2 可变时延指令/比较 | 存在 | poly_tomsg | `0x36bf div %r9d` | A-18、A-22 |
| M2 | 解密 | 3 数据相关循环 | 不存在 | poly_tomsg | 32 × 8 常数 | A-18 |
| M2 | 解密 | 4 数据相关访存 | 不存在 | poly_tomsg | 顺序读写 | A-18 |
| M2 | 解密 | 5 失败路径 | 不存在 | crypto_kem_dec | 本步不判定失败 | A-18 |
| M2 | 解密 | 6 异常处理 | 不适用 | — | RNG 不在本步 | A-18 |
| M2 | 哈希 | 1 敏感分支/提前返回 | 不存在 | sha3 / shake | 定长调用 | A-18 |
| M2 | 哈希 | 2 可变时延指令/比较 | 不存在 | FIPS202 | 无比较 | A-18 |
| M2 | 哈希 | 3 数据相关循环 | 不存在 | FIPS202 | 24 轮固定 | A-18 |
| M2 | 哈希 | 4 数据相关访存 | 不存在 | FIPS202 | 常量下标 | A-18 |
| M2 | 哈希 | 5 失败路径 | 不存在 | FIPS202 | 不返回状态 | A-18 |
| M2 | 哈希 | 6 异常处理 | 不存在 | FIPS202 | 无 abort/断言 | A-18 |
| M2 | 重加密 | 1 敏感分支/提前返回 | 存在 | poly_frommsg | `0x3668 bt` → `0x366c jae` | A-23、A-22 |
| M2 | 重加密 | 2 可变时延指令/比较 | 存在 | poly_compress / polyvec_compress | `0x3523 div`、`0x38c7 div` | A-18、A-22 |
| M2 | 重加密 | 3 数据相关循环 | 不存在 | 压缩函数 | 常数上界 | A-18 |
| M2 | 重加密 | 4 数据相关访存 | 不存在 | 压缩函数 | 定长位移拼装 | A-18 |
| M2 | 重加密 | 5 失败路径 | 不存在 | indcpa_enc | 不返回错误 | A-18 |
| M2 | 重加密 | 6 异常处理 | 不存在 | indcpa_enc | 无异常机制 | A-18 |
| M2 | 比较 | 1 敏感分支/提前返回 | 不存在 | verify | 遍历 1088 B | A-18 |
| M2 | 比较 | 2 可变时延指令/比较 | 不存在 | verify | 累积异或 | A-18 |
| M2 | 比较 | 3 数据相关循环 | 不存在 | verify | 常数 1088 | A-18 |
| M2 | 比较 | 4 数据相关访存 | 不存在 | verify | 顺序下标 | A-18 |
| M2 | 比较 | 5 失败路径 | 不存在 | verify / cmov | 结果只作掩码 | A-18 |
| M2 | 比较 | 6 异常处理 | 不适用 | — | 不可失败 | A-18 |
| M2 | 隐式拒绝 | 1 敏感分支/提前返回 | 不存在 | dec / cmov | 无条件走完 | A-18 |
| M2 | 隐式拒绝 | 2 可变时延指令/比较 | 不存在 | cmov | 值屏障在位 | A-18 |
| M2 | 隐式拒绝 | 3 数据相关循环 | 不存在 | cmov / KDF | 32 B 与定长 | A-18 |
| M2 | 隐式拒绝 | 4 数据相关访存 | 不存在 | cmov | 固定偏移 | A-18 |
| M2 | 隐式拒绝 | 5 失败路径 | 不存在 | dec | 同一路径，返回 0 | A-18 |
| M2 | 隐式拒绝 | 6 异常处理 | 不适用 | — | 无异常机制 | A-18 |
| M2 小计：存在 3、不存在 24、不适用 3、未检查 0（30 格） | | | | | | |

表 F2　M2 的 30 格排查。重加密的“存在”格在未修复版与已修复版两侧均成立。

| 实现 | 步骤 | 来源 | 判定 | 函数 | 指令或行号 | 证据 |
|---|---|---|---|---|---|---|
| M5 | 解密 | 1 敏感分支/提前返回 | 不存在 | indcpa_dec | 常数回边 | A-21 |
| M5 | 解密 | 2 可变时延指令/比较 | 不存在 | poly_tomsg | 乘加、移位 | A-21 |
| M5 | 解密 | 3 数据相关循环 | 不存在 | indcpa_dec | 常数上界 | A-21 |
| M5 | 解密 | 4 数据相关访存 | 不存在 | indcpa_dec | 计数器下标 | A-21 |
| M5 | 解密 | 5 失败路径 | 不存在 | indcpa_dec | 仅分配失败清理 | A-21 |
| M5 | 解密 | 6 异常处理 | 不适用 | — | 无异常机制 | A-21 |
| M5 | 哈希 | 1 敏感分支/提前返回 | 不存在 | hash_g / hash_h / hash_j | 定长调用 | A-21 |
| M5 | 哈希 | 2 可变时延指令/比较 | 不存在 | FIPS202 | 长度驱动 | A-21 |
| M5 | 哈希 | 3 数据相关循环 | 不存在 | FIPS202 | 固定轮数 | A-21 |
| M5 | 哈希 | 4 数据相关访存 | 不存在 | FIPS202 | 常量下标 | A-21 |
| M5 | 哈希 | 5 失败路径 | 不存在 | FIPS202 | 不返回状态 | A-21 |
| M5 | 哈希 | 6 异常处理 | 不存在 | FIPS202 | 无 abort/断言 | A-21 |
| M5 | 重加密 | 1 敏感分支/提前返回 | 不存在 | poly_frommsg | 唯一跳转为循环回边 | A-21 |
| M5 | 重加密 | 2 可变时延指令/比较 | 不存在 | 压缩函数 | 乘加、位移 | A-21 |
| M5 | 重加密 | 3 数据相关循环 | 不存在 | poly_frommsg | 32 次定长 | A-21 |
| M5 | 重加密 | 4 数据相关访存 | 不存在 | poly_frommsg | 固定顺序 | A-21 |
| M5 | 重加密 | 5 失败路径 | 不存在 | indcpa_enc | 仅分配失败清理 | A-21 |
| M5 | 重加密 | 6 异常处理 | 不适用 | — | 无异常机制 | A-21 |
| M5 | 比较 | 1 敏感分支/提前返回 | 不存在 | ct_memcmp | 循环体内无跳转 | A-21 |
| M5 | 比较 | 2 可变时延指令/比较 | 不存在 | ct_memcmp | 无分支归约 | A-21 |
| M5 | 比较 | 3 数据相关循环 | 不存在 | ct_memcmp | 常数 1088 | A-21 |
| M5 | 比较 | 4 数据相关访存 | 不存在 | ct_memcmp | 顺序 64 B/轮 | A-21 |
| M5 | 比较 | 5 失败路径 | 不存在 | ct_memcmp | 返回不构造分支 | A-21 |
| M5 | 比较 | 6 异常处理 | 不适用 | — | 不可失败 | A-21 |
| M5 | 隐式拒绝 | 1 敏感分支/提前返回 | 存在（公开信息） | check_sk | `a404 call → a40e jne ad87` | A-21、A-22 |
| M5 | 隐式拒绝 | 2 可变时延指令/比较 | 不存在 | ct_cmov_zero | `ad60` 至 `ad85` 掩码 | A-21 |
| M5 | 隐式拒绝 | 3 数据相关循环 | 不存在 | KDF / cmov | 固定轮数与 32 B | A-21 |
| M5 | 隐式拒绝 | 4 数据相关访存 | 不存在 | KDF / cmov | 固定偏移 | A-21 |
| M5 | 隐式拒绝 | 5 失败路径 | 不存在 | dec | 成功与失败同路径 | A-21 |
| M5 | 隐式拒绝 | 6 异常处理 | 不适用 | — | 无异常机制 | A-21 |
| M5 小计：存在 1、不存在 25、不适用 4、未检查 0（30 格） | | | | | | |

表 F3　M5 的 30 格排查。唯一“存在”格判据是公开数据的哈希，按有界例外登记。

三张表合计 90 格：存在 5、不存在 75、不适用 10、未检查 0。五格“存在”分别是 M1 重加密的敏感条件分支、M2 解密的可变时延指令（除法）、M2 重加密的可变时延指令（除法）、M2 重加密的敏感条件分支（未修复版与已修复版两侧），以及 M5 隐式拒绝的公开 sk 检查。十格“不适用”全部是异常处理类与该步骤不可能失败的组合。

90 格按“存在 / 不存在 / 不适用 / 未检查”分别登记。M5 的“存在”格是公开 sk 完整性检查，判据是公开数据的哈希，按有界例外登记。排查同时发现 M2 官方修复产物中仍保留与 M1 同源的消息编码分支，这是全流程方法对补丁产物的超额发现。

五个“存在”格分别落在三条机制上：M1 的消息编码分支、M2 的三处秘密除法与消息

编码分支、M5 的公开 sk 检查。前四条与秘密相关，第五条只携带公开信息。把五格都写成“漏洞”会混淆可利用性，把第五条删掉又会掩盖一个有意的提前退出。表 F1 至表 F3 逐格保留函数与地址，是为了让读者能回到产物核对，而不是只看一个总数。

十格“不适用”的分布也要能对上：M1 三格、M2 三格、M5 四格，全部是异常处理类与该步骤不可能失败的组合。它们不是“没查”，也不是“查了没有”，而是该步骤没有这类机制可查。三种判定各自计数：“不存在”只用于已查无发现，“不适用”只用于无此类机制，“未检查”单列。这样“不适用”既不会并入阴性，也不会并入缺口。M3 与 M4 是人工校准，不计入 90 格，它们在正文与附录 E 单列，避免与真实实现混算。

# 附录 G　复现入口与命令

本附录的命令均在 x86_64 Linux 用户态实跑。不写 sysfs、不需要 sudo。实跑环境为 WSL2 虚拟化 Linux（clang 14、CMake 3.31、OpenSSL 3），与锁频采数所用的裸金属不是同一台机器。命令的通过数用于验证软件流程，不产生计时结论。完整锁频采集需要 x86_64 裸金属与频率写权限，不在本附录的实跑范围内。

| 目标 | 命令 | 预期输出 | 实测结果（权限 / 机器） |
|---|---|---|---|
| 普通构建与测试 | `make test` | 普通 CTest 全通过 | 57/57 通过，0 失败（普通用户。WSL2，clang 14） |
| 消毒器构建与测试 | `make sanitize` | ASan/UBSan CTest 全通过 | 57/57 通过，0 失败（普通用户。WSL2，clang 14。ASan 关闭泄漏检测并关闭地址随机化） |
| 平台配置 | `cmake -S . -B build/platform -DBASELINE_ENABLE_PLATFORM=ON -DA113_ARCHIVED_CASE_ONLY=OFF` | 配置成功，生成构建树 | 配置成功（普通用户，WSL2） |
| 平台构建 | `cmake --build build/platform` | 构建成功 | 构建成功（普通用户，WSL2） |
| 平台测试 | `ctest --test-dir build/platform --output-on-failure` | 平台 CTest 全通过 | 87/87 通过，0 失败（普通用户。vendor 未启用。quick 档启用后 88/88） |
| 一键 quick | `sh scripts/reproduce_all.sh quick --fetch` | 四配置构建与 CTest 全通过，统计演练完成 | 263/263 通过，0 失败（88 + 57 + 57 + 61。普通用户，WSL2） |
| vendor 取件 | `sh tools/fetch_all_vendors.sh` | 四棵 vendor 树就位并逐项 SHA-256 校验 | 4/4 就位，摘要校验通过（普通用户，WSL2） |
| full 档演练 | `sh scripts/reproduce_all.sh full --modules M2,M3,M4,M5 --control-root < 控制根 >` | 只打印采集命令，不执行 | 打印成功，退出码 0（普通用户，WSL2） |

表 G1　复现入口与命令。命令实跑通过。完整锁频采集需要 x86_64 裸金属与频率写权限。quick 档的 263 项是 platform 88 + ordinary 57 + sanitize 57 + historical-fixed 61，失败 0。

quick 档只做配置、构建、CTest 与小样本统计演练：统计演练每类 200 个样本、1 次重复，用于检查链路是否连通。它不锁频、不写 sysfs、不需要 sudo、不绑核，验证软件流程与测试入口，不产生任何模块的正式统计结论。full 档按冻结计划生成采集命令，默认只打印，加 `--execute` 才真正采集，且必须给出控制根。计时结论来自锁频采集，仍需裸金属与频率写权限。干净 clone 缺 vendor 与 third_party，M1 与 M2 的 vendor 树缺省不存在，先跑取件脚本。取件按固定摘要校验，不覆盖已有且摘要一致的树。

实跑环境的编译器与采数环境不同：本附录的命令在 WSL2 的 clang 14 与 CMake 3.22 下执行，锁频数据来自另一台 x86_64 裸金属的 clang 18.1.3。两边只共享源码与测试入口，不共享时间结论。把这一点写在表 G1 的表注里，是为了避免读者把 263 项通过当成 263 次计时实验。

命令失败时的排查顺序也写进复现说明：先看编译器与 CMake 版本，再看 vendor 是否就位，然后看 sanitizer 的运行库。ASan 在部分虚拟化环境需要关闭地址随机化或关闭泄漏检测才能跑完。这类环境差异只影响测试入口，不影响源码。复现者若在另一台机器上得到不同的通过数，应先核对配置项与编译器，而不是先怀疑实验结论。

## 附录 H　术语与缩略语

表 H1 给出全篇统一术语。术语首次出现时列出中文说明与英文全称，之后统一使用定稿词。

| 术语 | 英文 / 全称 | 含义 | 首见 |
|---|---|---|---|
| ML-KEM | Module-Lattice-Based Key-Encapsulation Mechanism | FIPS 203 标准化的密钥封装机制。本作品的算法标准为 ML-KEM-768。 | 1.1 |
| Kyber | CRYSTALS-Kyber | ML-KEM 的前身。本作品的 M2 是 pq-crystals 历史快照 Kyber768。 | 1.1 |
| FO | Fujisaki-Okamoto | 把公钥加密转换为 KEM 的变换。解封装中的重加密与比较由它规定。 | 1.1 |
| KEM | Key-Encapsulation Mechanism | 密钥封装机制。 | 1.1 |
| PQC | Post-Quantum Cryptography | 后量子密码。 | 1.1 |
| MDE | Minimum Detectable Effect | 最小可检测效应。给定 $\alpha$、功效、样本量与方差下可分辨的最小均值差。 | 2.4 |
| Holm | Holm-Bonferroni | 逐步下降的多重比较校正。本作品四个家族各取全族 $\alpha = 0.05$。 | 2.4 |
| RTLF | R-Time-Leak-Finder | 基于分位数与 bootstrap 的族外分布检验。$\alpha = 0.01$。 | 2.3 |
| dudect | Dude, is my code constant time? | 基于 Welch $t$ 的在线泄漏检测工具。最大 $t$ 不转成 $p$ 值。 | 2.3 |
| Binsec | Binsec/Rel | 二进制级关系符号执行与有界常数时间检查工具。 | 2.3 |
| Microwalk | MicroWalk | 基于动态二进制插桩的访存互信息分析工具。 | 2.3 |
| Valgrind | Valgrind | 动态分析框架。本作品使用 Memcheck 与 varlat 补丁。 | 2.3 |
| varlat | variable-latency patch | Valgrind 的可变时延指令检查补丁，用于除法操作数含秘密的情形。 | 2.3 |
| aarch64 | ARM 64-bit architecture | ARM 64 位架构。 | 4.11 |
| x86_64 | x86-64 | 本作品的主测量架构。 | 1.1 |
| ipd | initial public draft | FIPS 203 初始公开草案。M1 的对象口径。 | 1.3 |

表 H1　术语与缩略语。术语全篇统一。第一次出现给全称，之后只用定稿词。

# 参考文献

[1] NIST. Module-Lattice-Based Key-Encapsulation Mechanism Standard. FIPS 203, 2024. DOI: 10.6028/NIST.FIPS.203.

[2] R. Avanzi, J. Bos, L. Ducas, et al. CRYSTALS-Kyber algorithm specifications and supporting documentation. NIST PQC submission, 2021 (initial version 2018).

[3] Q. Guo, T. Johansson, A. Nilsson. A Key-Recovery Timing Attack on Post-Quantum Primitives Using the Fujisaki-Okamoto Transformation and Its Application on FrodoKEM. CRYPTO 2020, LNCS 12171, 359–386. DOI: 10.1007/978-3-030-56880-1_13.

[4] D. J. Bernstein, et al. KyberSlash: Exploiting secret-dependent division timings in Kyber implementations. IACR TCHES 2025(2): 209–234. DOI: 10.46586/tches.v2025.i2.209-234.

[5] A. Purnal. clangover. GitHub, 2024. https://github.com/antoonpurnal/clangover.

[6] GitHub Security Advisory GHSA-f2v9-5498-2vpp (CVE-2024-36405): liboqs Kyber control-flow timing leak when compiled with Clang. 2024.

[7] O. Reparaz, J. Balasch, I. Verbauwhede. Dude, is my code constant time? DATE 2017, 1697–1702. DOI: 10.23919/DATE.2017.7927267.

[8] M. Dunsche, M. Maehren, N. Erinola, R. Merget, N. Bissantz, J. Somorovsky, J. Schwenk. With Great Power Come Great Side Channels: Statistical Timing Side-Channel Analyses with Bounded Type-I Errors. USENIX Security 2024, 6687–6704. https://www.usenix.org/conference/usenixsecurity24/presentation/dunsche

[9] A. Langley. Checking that functions are constant time with Valgrind. 2010. https://github.com/agl/ctgrind.

[10] J. Wichelmann, A. Moghimi, T. Eisenbarth, B. Sunar. MicroWalk: A Framework for Finding Side Channels in Binaries. ACSAC 2018, 161–173. DOI: 10.1145/3274694.3274741.

[11] L. Daniel, S. Bardin, T. Rezk. Binsec/Rel: Efficient Relational Symbolic Execution for Constant-Time at Binary-Level. IEEE S&P 2020, 1021–1038. DOI: 10.1109/SP40000.2020.00074.

[12] G. Goodwill, B. Jun, J. Jaffe, P. Rohatgi. A testing methodology for side-channel resistance validation. NIST Non-Invasive Attack Testing Workshop, 2011.

[13] J. B. Almeida, M. Barbosa, G. Barthe, F. Dupressoir, M. Emmi. Verifying Constant-Time Implementations. USENIX Security 2016, 53–70.

[14] M. Azouaoui, O. Bronchain, C. Hoffmann, Y. Kuzovkova, T. Schneider, F.-X. Standaert. Systematic Study of Decryption and Re-encryption Leakage: The Case of Kyber. SAC 2021, LNCS 13203, 236–256, 2022. DOI: 10.1007/978-3-030-99766-3_11.

[15] J. Jancar, M. Fourné, D. De Almeida Braga, M. Sabt, P. Schwabe, G. Barthe, P.-A. Fouque, E. Acar. “They’re not that hard to mitigate”: What Cryptographic Library Developers Think About Timing Attacks. IEEE S&P 2022, 632–649. DOI: 10.1109/SP46214.2022.9833713.